Solution Manual for

Introduction to Technical Problem Solving with MATLAB

Jon Sticklen, PhD
Michigan State University

M. Taner Eskil, PhD
Michigan State University

Great Lakes Press, Inc.
St. Louis, MO Okemos, MI

PO Box 550 / Wildwood, MO 63040
(636) 273-6016
custserv@glpbooks.com
www.glpbooks.com

International Standard Book Number: **FILL IN**

Copyright © 2005 Great Lakes Press, Inc.
All rights are reserved.

No part of this publication may be reproduced in any form or by any means without prior written permission of the publisher.

All comments and inquiries should be addressed to:
Great Lakes Press, Inc.
c/o John Gruender, Editor
PO Box 550
Wildwood, MO 63040
jg@glpbooks.com
phone (636) 273-6016
fax (636) 273-6086

Library of Congress Control Number:

Printed in the U.S.A. by Sheridan Books, Inc., Ann Arbor, Michigan.

10 9 8 7 6 5 4 3 2 1

This book belongs to:

_____

_____

_____

Preface **3**

Solutions for Chapter 3

MATLAB Basics: Scalars **1**

Set A: Nuts and Bolts Problems for Scalars **1**
Set B: Problem Solving with MATLAB Scalars **14**

Solutions for Chapter 4

Saving Your Work in MATLAB **33**

Set A: Nuts and Bolts Problems for Saving Your Work **33**
Set B: Problem-Solving Using MATLAB User-Defined Functions **52**

Solutions for Chapter 5

Vector Operations **73**

Set A: Nuts and Bolts Problems for Vectors **73**
Set B: Problem Solving with MATLAB Vectors **91**

Solutions for Chapter 6

2 D Plotting and Using MATLAB Help **109**

Set A: Nuts and Bolts Problems for 2-D Plotting **109**
Set B: Problem Solving with MATLAB 2-D Plotting **153**

Solutions for Chapter 7

Arrays **191**

Set A: Nuts and Bolts Problems for Arrays **191**
Set B: Problem Solving with MATLAB Arrays **206**

Solutions for Chapter 8

More Flexibility: Introduction to Conditional and Iterative Programming **219**

Set A: Nuts and Bolts Problems for Programming Constructs **220**
Set B: Problem Solving Using Programming Constructs **237**

PREFACE

This Solutions Manual for the textbook **An Introduction to Technical Problem Solving with Matlab v. 7**, Sticklen and Eskil, Great Lakes Press, Inc., ISBN 1-881018-80-6. **The Solutions Manual should be available to instructors only; instructors are asked in the interest of all instructors to restrict access to this book**.

Problems in Chapters 3 to Chapter 8 at the end of each chapter are grouped into two sections. "Nuts and Bolts …" sections focus on the mechanics of the MATLAB capabilities of the chapter. The problems in "Nuts and Bolts …" are straightforward and should be considered *exercises* rather than full blown problems in that these problems are typically of the "turn the crank" variety. They should be assigned to allow students to exercise their base understanding of MATLAB capabilities.

The second grouping of problems at chapter ends are labeled "Problem Solving With …" These problems are aimed at giving students practice in *applying* the tools of MATLAB and may more accurately be termed *problems* in that students must determine what MATLAB capabilities should be applied in a given problem.

In this Solutions Manual problems from the main text are reproduced, then a solution is given. The problem appears basically exactly the way it does in the main text. The solution is set off between heavy bold, dashed lines, and the font of the answer is written in a bold, monospaced font.

In many cases, the problem solution will include notes on what students should "get out of" the problem, what instructors might assign as extensions to problems, and in some cases, alternative solutions to problems. In using solutions offered here, it is important that instructors realize there may be more than one way to solve given problems.

Questions, suggestions, and comments should be sent to Mr. John Gruender, Great Lakes Press, jg@glpbooks.com.

SOLUTIONS FOR CHAPTER 3
MATLAB BASICS: SCALARS

Problems in this chapter are solved by students without the ability to save their work in M files (functions/scripts) since saving MATLAB work is the topic of Chapter 4. In most cases students should be working with both the command window (to write and execute code lines) and the history window (to "re-run" code lines). To effectively work the *back and forth* between the command window and history window, students are (should be …) forced to start using MATLAB variables effectively.

Much of the what goes on in this chapter for students focuses on getting used to the MATLAB environment in the context of performing operations that are familiar: scalar operations.

Set A: Nuts and Bolts Problems for Scalars

Problem 3-A.1

Define a variable named **`angle`** to be 45°. Convert **`angle`** to radians using the following:

$$radians = degrees \times \left(\frac{\pi}{180}\right)$$

Store the result in the variable **`angleRad`**. Use the predefined variable **`pi`** in your calculation.

Using the History Window, repeat the calculation for **`angle`** = 60°, 75°, and 90°.

```
% PROBLEM 3-A.1
  clc, clear
  angle = 45                    % degrees
  angleRad = angle * (pi/180)   % radians

  % student then resets angle, and runs the line setting
  % angleRad from the history window
```

Problem 3-A.2

Suppose we have: an independent variable **x** and two dependent variables **y** and **z**. Here are the expressions to calculate **y** and **z**:

$$y = x - 5$$

$$z = 3y^2 - 16y + 74$$

Set **x** to be 5. Calculate the corresponding values of **y** and **z**.

Using the Command History Window, repeat the calculations for **x** = 7, 9, and 11.

```
This problem is a vehicle to refresh students on the concept of
dependent/independent variable – along with the continuing story
of scalar operations and using the command and history windows.

% PROBLEM 3-A.2
  clc, clear
  x = 5                         % indep var
  y = x - 5                     % dep var
  z = 3*y^2 - 16*y + 74         % dep var

  % student resets x, then recomputes y and z using the
  % history window
```

Problem 3-A.3

The area and the perimeter of a circle of radius **r** are found with the following expressions:

$$circleArea = \pi r^2$$

$$circlePerimeter = 2\pi r$$

Set the radius (**r**) to the value 5. Calculate the area and perimeter of a circle by entering the formulae above.

Now set the radius to be 7. Double-click on line commands in the Command History Window to recalculate the area and perimeter.

Set the radius to 9. This time, select both line commands by holding the *Ctrl* button and copy them to the Command Window. Press *Enter* to carry out both calculations.

```
% PROBLEM 3-A.3
  clc, clear
  r = 5                          % linear unit
  circleArea = pi*r^2            % corresponding area unit
  circlePerimeter = 2*pi*r       % same linear unit

  % student resets r, then recomputes using history window
```

Problem 3-A.4

What is a scalar? How does a scalar differ from a vector? How does it differ with an array (or a matrix)? Explain in scientific and MATLAB terms.

```
A scalar in MATLAB is a simple number; one number. A vector in
MATLAB is a linear (1-D) group of numbers. An array in MATLAB is
a table (2-D) of numbers. In science and engineering theory, a
vector is an entity that has both magnitude and direction, like
velocity. The corresponding scalar to the velocity vector is
called speed. There is a table like entity in science and
```

```
engineering theory called a tensor, but that is beyond the scope
of this book.
```

Problem 3-A.5

Having a magnitude and a direction, force is represented by vectors in science. Suppose we represented a force vector in the Command Window by defining its **x** and **y** components:

```
>> forceX = 3
>> forceY = 5
```

What type of MATLAB variables are *forceX* and *forceY*? Review the definitions of scalars, vectors and arrays in Section 3-2 to verify your answer.

```
Both forceX and forceY are scalars.
```

Problem 3-A.6

Using **angleRad** you defined in Problem 3-A.1, find the cosine and sine of **angle**. Store the results in variables **cosAngle** and **sinAngle**, respectively.

```
% PROBLEM 3-A.6
clc, clear
angle = 45                      % degrees
angleRad = angle * (pi/180)     % radians
cosAngle = cos(angleRad)
sinAngle = sin(angleRad)
```

Problem 3-A.7

Using variables **cosAngle** and **sinAngle** (Problem 3-A.6), define the following:

$$\alpha = 45°$$
$$z = \cos(\alpha)^2 + \sin(\alpha)^2$$

A well-known identity in trigonometry is that the sum of cosine-squared and sine-squared of an angle always equals one. If your result for **z** was not one, review Section 3-3 and redo the operations.

```
% PROBLEM 3-A.7
  clc, clear
  alpha = 45;                    % degrees
  angleRad = alpha * (pi/180)    % radians
  cosAngle = cos(angleRad)
  sinAngle = sin(angleRad)
  z = cosAngle^2 + sinAngle^2
```

Problem 3-A.8

Redefine the value of variable **angle** (Problem 3-A.6) as 65° and recalculate the value of **z** (Problem 3-A.7) using the Command History Window.

```
This is the standard operation for using the history window to
"store" solutions that can be re-run.
```

Problem 3-A.9

Create a variable named **z** and set it to seven. Calculate **4z** and store the result in **z**.

```
This exercise is very simple on the mechanical level. But the
point here is that using the assignment operator in MATLAB is
definitely not the same thing as equality in algebra.

% PROBLEM 3-A.9
  clc, clear
  z = 7
  z = 4*z
```

Problem 3-A.10

Assuming **x** is a defined variable in the MATLAB workspace, are the following scalar operations valid in MATLAB? If not, why not?

Hint: Remember, '**=**' is the assignment operator in MATLAB.

Part A `>> x + 2 = 4`

Part B `>> x = 4 - 2`

Part C `>> x = 4 - (-2)`

Part D `>> 5 = x`

Part E `>> 5 + 3 = 7 + 1`

```
Part A:  not valid because left hand side is not a variable (a
         container

Part B:  valid because left hand side is a variable

Part C:  valid because left hand side is a variable

Part D:  not valid because left hand side is not a variable

Part E: not valid because left hand side is not a variable
```

Problem 3-A.11

Solve the following problems with paper and pencil. When you are finished, execute the code in the Command Window to verify your answers. Use your understanding of the container model as you think through the questions.

Part A What do the variables **x** and **y** contain after executing the following lines?

```
>> x = 4
>> y = x
>> x = 6
```

Part B What does **x** contain after executing the following lines?

```
>> x = 2
>> x^2
```

Part C What does **x** contain after executing the following lines?

```
>> x = 2
>> x = 2 * x + 5
>> x = (x-3)/3
```

```
Part A: x contains 6, y contains 4

Part B: x contains 2

Part C: x contains 2
```

Problem 3-A.12

Set **z** to 5 in the Command Window. Set **k** and **m** using the expressions below.

Part A $k = e^{2z}$

Part B $m = \log(e^z)$

(The mathematical *log* function is "log to the base 10.")

```
% PROBLEM 3-A.12
  clc, clear
  z = 5
  k = exp(2*z)
  m = log10(exp(z))
```

Problem 3-A.13

One way to test for the sign of scalar **x** is to check if $\sqrt{x^2}$ equals x. Assume the calculation works:

$$x = \sqrt{x^2}$$

If so, then we can conclude that **x** is non-negative; otherwise **x** is negative. (A non-negative number is one that is positive or zero. The relationship above is NOT a MATLAB assignment statement.)

Successively set **x** to each of the following numbers and use the MATLAB relational equality operator to test whether **x** is non–negative: `{-5, 10, 35, -292, 0, 101}`.

```
% PROBLEM 3-A.13
  clc, clear
  x = -5
  x == sqrt(x^2)

  % now reset x=10, and "run" the second line from the history
  % window. Then reset x=35, and etc
```

Problem 3-A.14

One way to check if a scalar is an integer is to round the scalar and compare the result with itself.

The built-in MATLAB function **round** can be used for rounding a number. For instance, **round(5)** and **round(5.3)** will return 5. But **round(5.6)** will return 6.

Using the built-in function **round** and MATLAB relational operators, test if **x** is an integer for the following values of **x**: {-5.2, 10, 35.3, -292, 0, 101.99}.

This problem is very similar to the problem of testing for a negative number (Problem 3-A.13).

```
% PROBLEM 3-A.14
  clc, clear
  x = -5.2
  x == round(x)  % if x is integer, test is TRUE

  % now reset x=10, and run the test again... etc
```

Problem 3-A.15

(Be sure you have solved Problem 3-A.13 and Problem 3-A.14 before attempting this problem.)

Use relational and logical operators to do the following tests in MATLAB:

Part A Is 5 a non-negative integer?

Part B Is 7.7 a non-negative integer?

Part C Is 7.7 *not* a negative integer?

Part D Is 5 or 7.7 a negative integer?

```
% PROBLEM 3-A.15
```

```
% Part A
clc, clear
x = 5
x == sqrt(x^2)  &  x == round(x)

% Part B
clc, clear
x = 7.7
x == sqrt(x^2)  &  x == round(x)

% Part C
clc, clear
x = 7.7
~( x ~= sqrt(x^2)  &  x == round(x) )

% Part D
clc, clear
x = 5
y = 7.7
(  x ~= sqrt(x^2)  &  x == round(x)  )   | ...
   (  y ~= sqrt(y^2)  &  y == round(y)  )
```

Problem 3-A.16

Create expressions using variables **x** and **y** to answer the following questions. So you have explicit values for **x** and **y** to test your work, set **x=7** and **y=5**.

Part A Is **x** greater than 6 and less than 10? Here is another way to ask the same question: *Is* **x** *in the range (6,10)*?

Part B Is **x** greater than or equal to 10, or less than or equal to 6? Here is another way to ask the same question: *Is x not in the range (6, 10)*?

Part C Is (**x** is greater than 6) *and* (**y** is greater than **x** but less than 10)?

Part D Is (**x** is in the range (6,10)) but (**y** *not* in the range)?

Part E Is *x* or *y* in the range (6,10)?

A very common beginning MATLAB programming error is to represent a test on the range of a variable incorrectly. For example, if

```
we want to ask if x is in the range (2,4), then the common error
is to write
   2 < x < 4
This relational expression will always return 1: if 2 < x is
TRUE then the second test is 1 < 4, while if 2 < x is FALSE then
the second test is 0 < 4. In both cases the result is TRUE.

% PROBLEM 3-A.16
  clc, clear
  x = 7
  y = 5

  % Part A
  (6 < x)  &  (x < 10)

  % Part B
  (6 >= x)  |  (x >= 10)
  % alternative...
  %   ~( (6 < x)  &  (x < 10) )

  % Part C
  (x > 6) & (y > x) & (y < 10)

  % Part D
  ((6 < x)  & (x < 10)) & ~((6 < y)  & (y < 10))

  % Part E
  ((6 < x)  & (x < 10)) | ((6 < y)  & (y < 10))
```

Problem 3-A.17

Suppose that **x=4** and **y=2**. Compute the following expressions in MATLAB.

Part A $3(x-y)$

Part B $\frac{x}{2y}$

Part C $\frac{y^3}{(1-x^2)}$

Part D $\dfrac{y^3}{(1-x)^2}$

Part E $\left(1-\dfrac{y}{x}\right)^{-y}$

Part F $\dfrac{1}{\dfrac{1}{2y}-\dfrac{y+6}{x-2}}$

Most of these exercises are straight forward. However, students should take care with Part F. It is the one exercise part that is complex enough to really warrant decomposition.

```
% PROBLEM 3-A.17
  clc, clear
  x = 4
  y = 2

  % Part A
  3 * (x - y)

  % Part B
  x / (2*y)

  % Part C
  y^3 / (1 - x^2)

  % Part D
  y^3 / (1 - x)^2

  % Part E
  (1 - (y/x))^-y

  % Part F
  f1 = 1/(2*y)
  f2 = (y + 6)/(x - 2)
  1 / (f1 - f2)
```

Problem 3-A.18

Suppose that **x=4** and **y=2**. Compute the following expressions in MATLAB:

Part A $\sin\left(y-\frac{1}{x}\right)$

Part B $\sin\left(y-\frac{1}{x}\right)^2$

Part C $\sin\left(\left(y-\frac{1}{x}\right)^2\right)$

Part D $\tan\left(\ln\left(\frac{1}{x}+\frac{1}{y}\right)\right)$

Part E $e^{-2\sin(x)\cos(y)}$

```
% PROBLEM 3-A.17
  clc, clear
  x = 4
  y = 2

  % Part A
  sin(y - (1/x))

  % Part B
  sin(y - (1/x))^2

  % Part C
  sin( (y - (1/x))^2 )

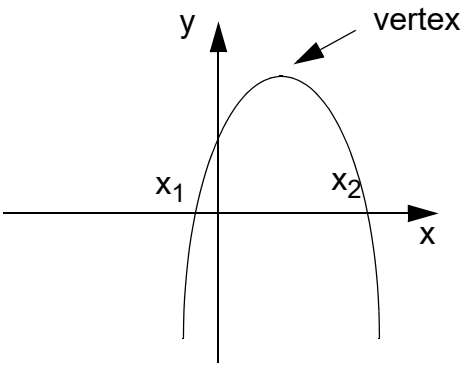
  % Part D
  tan(log((1/x) + (1/y)))

  % Part E
  exp(2*sin(x)*cos(y))
```

Set B: Problem Solving with MATLAB Scalars

Problem 3-B.1

A quadratic polynomial such as $y = 3x^2 + 2x - 6$ defines a parabola in the two-dimensional space. The vertex of a parabola is the point at the top if the curve is concave down or the point at the bottom if the curve is concave up.



For a quadratic polynomial $y = ax^2 + bx + c$, the x coordinate of the vertex and the peak value of the parabola can be found using this relationship:

$$x_{vertex} = -\frac{b}{2a}$$

$$y_{vertex} = ax_{vertex}^2 + bx_{vertex} + c$$

In the Command Window, set the values of the constants to be **a=1**, **b=3**, and **c=2**. Calculate the value of $\mathbf{x_{vertex}}$. Calculate $\mathbf{y_{vertex}}$.

Set **b=4**. Recalculate the values of $\mathbf{x_{vertex}}$ and $\mathbf{y_{vertex}}$ by selecting the appropriate command lines in the History Window and carrying them to the Command Window. Note that you do not need to redefine variables **a** and **c** in this case since they are already set to their original values.

Set **c=4** and recalculate $\mathbf{x_{vertex}}$ and $\mathbf{y_{vertex}}$. For which value of **b** did you calculate the vertex coordinates, 3 or 4?

This problem is again an exercise in using the history window to "save" commands for replaying. The answer for the last question posed is … Since the last value of b that was set was b = 4, the last calculation is for a=1, b=4. and c=4.

```
Code to implement an answer for this question is below. Note the
alternate expression for yVertex. If an instructor feels her
students should sharpen their algebra skills, then requiring the
substitution and manipulation to produce the alternate yVertex
would be a useful extension to the problem spec.


% PROBLEM 3-B.1
  clc, clear
  a = 1                                      % LINE 1
  b = 3                                      % LINE 2
  c = 2                                      % LINE 3
  xVertex = -b/(2*a)                         % LINE 4
  yVertex = a*xVertex^2 + b*xVertex + c      % LINE 5
    % substituted expression: yVertex = (-b^2/(4*a)) + c

  % bring LINE 2 back from History window, change b = 4,
  % execute

  % EXECUTE LINE 4 from Hist Win
  % EXECUTE LINE 5 from Hist Win

  % repeat for other settings of a, b, or c
```

Problem 3-B.2

The Newtonian gravitational force between two objects (like planets) in space is calculated using the following formula:

$$F_{gravitational} = -G\left(\frac{m_1 m_2}{d^2}\right)$$

Where **G** is the universal gravitational constant ($6.672 \times 10^{-11} N m^2 kg^{-2}$), $\mathbf{m_1}$ and $\mathbf{m_2}$ are the masses of the planets in kilograms, **d** is the distance between planets in meters, and the resultant force is in Newtons.

Using MATLAB, calculate the gravitational forces in the following examples:

Part A The Earth ($M_E = 5.97 \times 10^{24} kg$) and its moon ($m_M = 7.36 \times 10^{22} kg$), assuming an average distance of 3.84403×10^8 meters separating the Earth and its moon.

Part B The Earth and Sun ($m_S = 1.99 \times 10^{30} kg$), assuming an average distance of 1.495979×10^{11} meters between the Earth and the Sun.

Part C An Airbus A320 (74 tons) on the ground and the Earth, assuming the radius of the Earth is 6,380 km.

Part D An Airbus A320 at 40,000 feet and the Earth, assuming the radius of the Earth is 6,380 km.

Part E An Airbus A320 at 10,000 miles and the Earth, assuming the radius of the Earth is 6,380 km. (Assume the A320 was towed to that location.)

Parts A and B of this problem are relatively simple requiring application of what the "standard" use of command and history windows.

Parts C, D, and E however require as well an attention to physical units and an ability to convert units as needed. Students can usefully be given this problem with no heads-up about units. A good web site for obtaining conversion factors (as of this writing) is …

http://www.chemie.fu-berlin.de/chemistry/general/units_en.html

Students can find conversion factors there need in parts of this exercise:

- **1 ton = 907.1847 kg**
- **1 foot = 0.3048 m**
- **1 mi = 1609.344 m**

Code meeting the requirements of this problem is below.

```
% PROBLEM 3-B.2
  clc, clear
  G = 6.672 * 10^-11                % N m^2 kg^-2

  % Part A
  m1 = 5.97 * 10^24                 % kg, earth
  m2 = 7.36 * 10^22                 % kg, moon
  dist = 3.84403 * 10^8             % m, earth-moon
  F_partA = -G * m1 * m2 / dist^2   % N, earth-moon

  % Part B
  m1 = 5.97 * 10^24                 % kg, earth
  m2 = 1.99 * 10^30                 % kg, sun
```

```
dist = 1.495979 * 10^11            % m, earth-sun
F_partB = -G * m1 * m2 / dist^2    % N, earth-sun


% Part C
m1 = 5.97 * 10^24                  % kg, earth
  m2 = 74                          % tons, A320
m2 = m2 * 907.1847                 % kg, A320
  dist = 6380                      % km, earth-A320 ground
dist = dist * 10^3                 % m, earth-A320 ground
F_partC = -G * m1 * m2 / dist^2    % N, earth-A320 ground


% Part D
m1 = 5.97 * 10^24                  % kg, earth
m2 = 74 * 907.1847                 % kg, A320
  dist1 = 6380                     % km, earth-ground
  dist1 = dist1 * 10^3             % m, earth-ground
  dist2 = 4*10^4                   % ft, ground-A320 flight
  dist2 = dist2 * 0.3048           % m, ground-A320 flight
dist = dist1 + dist2               % m, earth-A320 flight
F_partD = -G * m1 * m2 / dist^2    % N, earth-A320 ground


% Part E
m1 = 5.97 * 10^24                  % kg, earth
m2 = 74 * 907.1847                 % kg, A320
  dist1 = 6380                     % km, earth-ground
  dist1 = dist1 * 10^3             % m, earth-ground
  dist2 = 10^4                     % mi, ground-A320 flight
  dist2 = dist2 * 1609.344         % m, ground-A320 flight
dist = dist1 + dist2               % m, earth-A320 flight
F_partE = -G * m1 * m2 / dist^2    % N, earth-A320 ground
```

Problem 3-B.3

Money saved in a bank account will compound with the interest according to the following formula:

$$newBalance = oldBalance \times \left(1 + \frac{interestRate}{100}\right)$$

The interest rate is given in percentage.

In the MATLAB Command Window, set **`oldBalance`** to 1,000 and **`interestRate`** to 6 percent. Calculate **`newBalance`**. This is the account balance after one year.

Store the value of **newBalance** in the variable **oldBalance** and drag the command line that calculates **newBalance** to the Command Window. Store the value of **newBalance** in **oldBalance** again. Describe what is now stored in **oldBalance**.

Use your Command History to calculate the balance after eight years. Find the answer in no more than two additional steps.

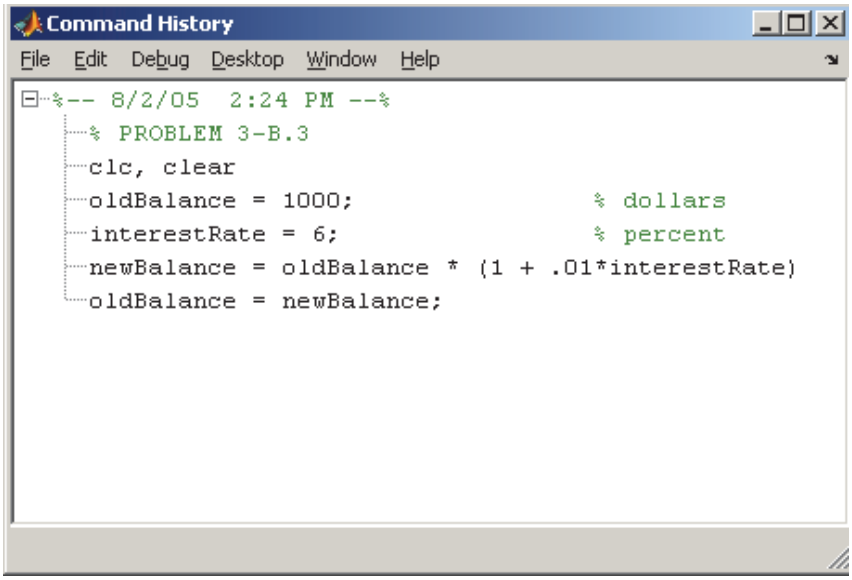
This problem should be easy conceptual for students except for the last "hook" – Find the answer in no more than two additional steps. The key to meeting that requirement is to know that multiple lines in the history window can be selected at once for execution or for dragging to the command window. After students attempt this problem for a time, a hint that multiple lines can be selected would be in order. (The method is to hold down <ctrl> when clicking another history window line.)

Code follows, then an indication explicitly of how the "two additional steps" requirement is met. Note that in the code, we start using semicolons with this problem to suppress command window echo when we do not need output.

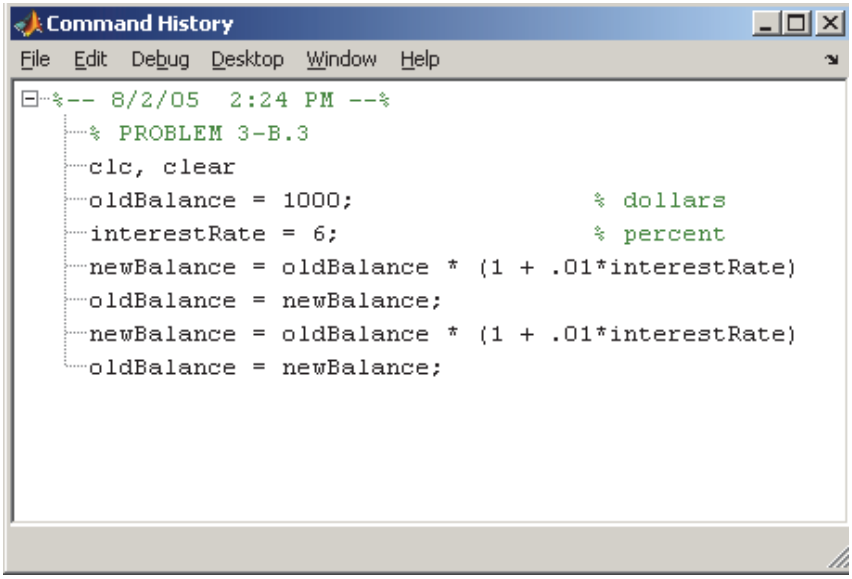
```
% PROBLEM 3-B.3
  clc, clear
  oldBalance = 1000;                % dollars
  interestRate = 6;                 % percent
  newBalance = oldBalance * (1 + .01*interestRate)
  oldBalance = newBalance;

  % Student executes the 5 lines above to get the value
  % after 1 year of interest is earned.
  % Then, the 4th line to get the value after 2 years, and
  % the 5th line to get the calculation ready for the next
  % pass through.
  % Continue doing this until the account balance for the
  % 8th year is computed. Select multiple lines in the
  % history window for execution to find the value for 8
  % years.
```

Executing the statements above will produce the following in the history window.

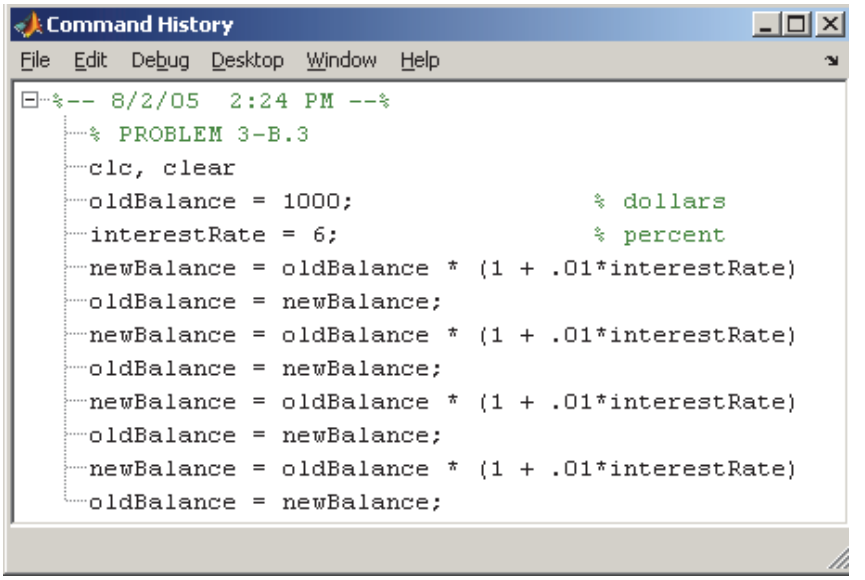


Selecting and executing the last two lines from the history window will produce the balance for the second year, and change the history window to the following:



Now we are to the point in the problem where we want to compute the value of the bank balance after eight years, and in only two steps.

Step 1: select the last four lines in the history window and execute them. This will give the balance after 4 years in the command window, and change the history window to the following:



Step 2: select the last eight lines in the history window and execute. This will produce the balance after eight years in the command window, as desired.

Problem 3-B.4

Redo Problem 3-B.3. To define balance, use this equation:

$$balance = balance \times \left(1 + \frac{interestRate}{100}\right)$$

Set **`balance`** to 1,000 and **`interestRate`** to 6 percent. Using these two variables and your Command History, calculate the account balance after eight years.

This problem is a redo of the preceding one with a different expression to compute balance. There is one difference - instead

```
of two commands to compute a new balance and reset the old bal-
ance to be the new (as in Problem 3-B.3), here the same effect
is obtained by using one command.

% PROBLEM 3-B.4
  clc, clear
  balance = 1000;                   % dollars
  interestRate = 6;                 % percent
  balance = balance * (1 + (interestRate/100))
```

Problem 3-B.5

Put values of three and seven in variables (containers) x and y, respectively. After assigning the values, switch the values of x and y without using any numbers. Be sure your code for the swap works by checking the results: check the values of x and y after your swap code is executed..

Hint: You can use an unlimited number of containers in MATLAB.

```
One of the reasons for using the container model of variables is
that the "visual thinking" that accompanies an application usu-
ally helps students to understand problems like this one. This
simple sounding problem is not simple for those who have not
programmed before. The reason largely lies in a lack of under-
standing for what an assignment statement accomplishes.

Code follows:

% PROBLEM 3-B.3
  clc, clear
  x = 3
  y = 7
  temp = x
  x = y
  y = temp
```

Problem 3-B.6

The table below shows the prediction of school administration for the percent of students who will be graduating and the number of new admissions for

each semester. The enrollment just prior to Fall, 2004 is 7,300. What is the expected enrollment in Fall 2006? (Do not worry about fractions of students in your result.)

You should make use of the History Window when you are doing the same calculation multiple times.

	Number of New Admissions	Predicted Graduation (%)
Fall 2004	1,200	11
Spring 2005	740	19
Summer 2005	145	6
Fall 2005	1,450	12
Spring 2006	810	21
Summer 2006	120	4

```
This problem has an intentional vaguness in it. The issue is
whether the graduation numbers should be derived from the size
of the class body excluding new admits, or if the graduation
numbers should be derived from the entire size of the class
body. There are logical arguments for both positions, but the
real point is that the implementor of a solution does not know
what the writter of the problem spec meant. This situation sets
up the possiblity of a discussion about clarrifying problem
specs if the instructor so desires.

The code below assumes the graduation rate is over all the
students (including new admits).

% PROBLEM 3-B.6
  clc, clear
  enroll = 7300
  numberNew = 1200;  gradPercent = 11;
  enroll = (enroll + numberNew) * (1 - (gradPercent/100))

  % This is all the code that is necessary. Executing this
  % code will find the beginning enrollment for Spring 2005.
  % Then drag the line from the history window that sets
  % numberNew and gradPercent, and reset these variables.
  % Finally in the history window execute the line setting
```

```
% the enrollment. Repeat for all six semesters in the table.
```

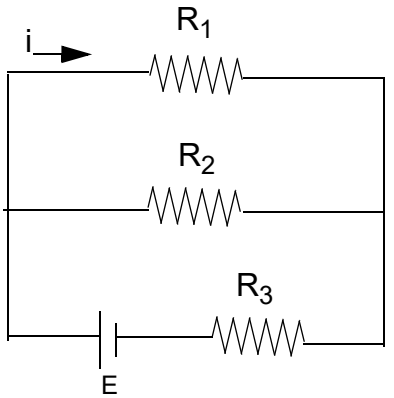
Problem 3-B.7

For the circuit setup shown in the figure, the current **i** (amperes) that flows through $\mathbf{R_1}$ can be found using this expression:

$$i = V\left(\frac{R_2}{R_1R_2 + R_1R_3 + R_2R_3}\right)$$

Where **V** is the electromotive force (volts) and $\mathbf{R_1}$, $\mathbf{R_2}$ and $\mathbf{R_3}$ are resistances (ohms) as shown in the diagram.

Suppose $\mathbf{R_1}$, $\mathbf{R_2}$, and $\mathbf{R_3}$ are 1, 2, and 4 ohms, respectively. Assume **V** is 20 volts. Find the current **i** that flows through $\mathbf{R_1}$. Pay attention to the precedence rules as you develop the MATLAB scalar expression for the relation above.



This problem is easier to solve if the expression above for current is decomposed by first computing the value of the denominator of the fraction in the expression for current above.

The ability to decompose expression to "bite sized chunks" will become increasing important for students as they face increasing complex expressions.

```
% PROBLEM 3-B.7
  clc, clear
  R1 = 1;           % ohm
  R2 = 2;           % ohm
  R3 = 4;           % ohm
  V = 20;           % volts
  denom = R1*R2 + R1*R3 + R2*R3;
  current = V * (R2 / denom)
```

You should do all the tests suggested in the following problems on the Command Window. You may be tempted to investigate the results visually and skip MATLAB tests that use relational and logical operators. However, engineering analysis requires writing code that works on varying inputs, much like how the built-in function **`sin`** works. You will need to do logical tests involving variables as you progress in MATLAB. You will learn how to write user-defined functions for MATLAB in the next chapter.

Problem 3-B.8

The *geometric mean* (average) of N numbers $\mathbf{x_1}$, $\mathbf{x_2}$, …, $\mathbf{x_N}$ is defined by the following equation:

$$geometricMean = \left(\prod_{i=1}^{N} x_i \right)^{\frac{1}{N}}$$

This can also be expressed as:

$$geometricMean = (x_1 \times x_2 \times \ldots \times x_n)^{\frac{1}{N}}$$

Set **`x1=7`**, **`x2=16`**, and **`x3=21`**. Find the arithmetic and geometric means of these three numbers. Use MATLAB to test if their arithmetic mean is greater than their geometric mean.

```
% PROBLEM 3-B.8
  clc, clear
  x1 = 7
  x2 = 16
  x3 = 21
  aMean = (x1 + x2 + x3) / 3
  gMean = (x1 * x2 * x3)^(1/3)
  aMean_greater_gMean = aMean > gMean
```

Problem 3-B.9

The roots of a quadratic polynomial, $0 = ax^2 + bx + c$, can be found with this equation:

$$x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

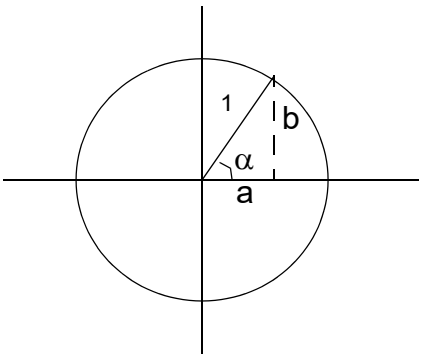
Use MATLAB to determine if both roots of the quadratic polynomial $0 = x^2 - 21x + 98$ are greater than ten.

An extension of this problem would be to give several sets of (a, b, c) and have students use the history window to resolve for the new values of the coefficients. Note of course that if (b^2 - 4ac) < 0, then the roots will involve complex numbers.

```
% PROBLEM 3-B.9
  clc, clear
  a = 1
  b = -21
  c = 98
  x1 = (-b + sqrt(b^2 - 4*a*c)) / (2*a)
  x2 = (-b - sqrt(b^2 - 4*a*c)) / (2*a)
  bothRootGreater10 = (x1 > 10) & (x2 > 10)
```

Problem 3-B.10

The unit circle is a useful tool in understanding trigonometric relationships for right triangles. A unit circle is a circle with a radius of one.



For every right triangle whose hypotenuse is 1, the adjacent and opposite side lengths are found with these formulae:

$$a = \cos(\alpha)$$
$$b = \sin(\alpha)$$

Applying the Pythagorean theorem, we find these equations:

$$a^2 + b^2 = 1$$
$$(\cos(\alpha))^2 + (\sin(\alpha))^2 = 1$$

Use MATLAB to verify this equality for α = 0, 30, 60, and 90 degrees.

The built-in trigonometric functions in MATLAB always take their arguments in radians. You will need to convert the given angles to radians.

```
% PROBLEM 3-B.10
  clc, clear
  alpha = 0                        % degrees
  alpha_rad = alpha * (pi/180);  % radians
  factors = (sin(alpha_rad))^2 + (cos(alpha_rad))^2;
  identityWorks = factors == 1

  % After executing all commands once, then select and move
  % all of them back to the command window from the history
  % window. Change the value of alpha. Press return to
  % reexecute the entire set of commands.
```

Problem 3-B.11

Jim argues that he is taller than Carla and Cindy. He is willing to put his money where his mouth is, but it turns out that Jim is taller than Carla or Cindy but not both. Jim, Carla, and Cindy are 5.9, 6.1, and 5.7 feet tall, respectively. Use MATLAB to verify the following:

Part A Jim is not taller than Carla and Cindy.

Part B Jim is taller than Carla or Cindy.

Part C Jim is taller than Carla or Cindy but not both.

Part C can be solved using the solutions from Part A and Part B. The problem spec does not demand a solution for Part C that uses solutions to Part A and Part B, but instructors can point out the possibility after students do this exercise.

```
% PROBLEM 3-B.11
  clc, clear
  jim = 5.9
  carla = 6.1
  cindy = 5.7

  % Part A
    partA_result = ~( (jim > carla) &  (jim > cindy) )

  % Part B
    partB_result = (jim > carla) | (jim > cindy)

  % Part C
    partC_result = ( (jim > carla) | (jim > cindy) ) & ...
                   ~( (jim > carla) & (jim > cindy) )
    % alternate
    % partC_result = partB_result & partA_result
```

Problem 3-B.12

Suppose every morning you look out the window and make a decision for how to get to work according to the following rules:

If it is rainy and the traffic is light, OR if you have fewer than 15 minutes to get to work, you take your car.

If it is rainy and the traffic is heavy, you take the train.

If it is rainy or snowy, you take your umbrella with you.

If it is not rainy and you have more than 15 minutes, you walk.

If it is snowy and the traffic is heavy, you call in sick.

This morning you look outside and see that it is not rainy (**`rainyDay=0`**) but snowy (**`snowyDay=1`**), and the traffic is heavy (**`heavyTraffic=1`**). You have 23 minutes to get to work.

Apply each rule using relational and logical operators of MATLAB and decide if you will take your car (**`drive`**), take the train (**`catchTrain`**), walk (**`walk`**), or call in sick (**`callAndCough`**), and if you will need to take your umbrella (**`takeUmbrella`**).

If the result you get does not make logical sense to you, examine the rules again and improve them. Run your example again and see if you get a more sensible result.

Converting the rules in the problem specification into MATLAB code will not be difficult for most students once they get the hang of it.

More difficult will be the consideration of the "goodness" of the result, and reflection on how to "fix" any logical problems. A context of "just common sense" will aid students in the task.

Code that implements the original problem spec follows:

```
% PROBLEM 3-B.12
  clc, clear
  rainyDay = 0;
  snowyDay = 1;
  heavyTraffic = 1;
  timeAvail = 23;                  % minutes

  drive = (rainyDay & ~heavyTraffic) | ...
          (timeAvail < 15)
  catchTrain = rainyDay & heavyTraffic
  walk = ~rainyDay & timeAvail > 15
  takeUmbrella = rainyDay | snowyDay
  callAndCough = snowyDay & heavyTraffic
```

When run, this code will produce the following results:

```
drive = 0
catchTrain = 0
walk = 1
takeUmbrella = 1
callAndCough = 1
```

The problem is that we are being told to both walk (i.e., go to work) and to stay home. Both results should not be indicated.

The problem lies with the third rule that tests if we should walk. The factor left out is that we should not walk if it is snowy. Putting this extra part (don't walk if its snowy) into the rule give us …

```
walk = ~rainyDay & ~snowyDay & timeAvail > 15
```

The results of running the code after this change is made are as follows:

```
drive = 0
catchTrain = 0
walk = 0
takeUmbrella = 1
callAndCough = 1
```

This is better, but there is still a problem: we are instructed to take our umbrella when we are not going to work. Fixing this problem would be easiest by reversing the order of the last two code lines, and making takeUmbrella depend on whether or not we are calling in sick.

```
% PROBLEM 3-B.12 b... augmented version
  clc, clear
  rainyDay = 0;
  snowyDay = 1;
  heavyTraffic = 1;
  timeAvail = 23;                    % minutes

  drive = (rainyDay & ~heavyTraffic) | ...
          (timeAvail < 15)
  catchTrain = rainyDay & heavyTraffic
  walk = ~rainyDay & ~snowyDay & timeAvail > 15
  callAndCough = snowyDay & heavyTraffic
  takeUmbrella = (rainyDay | snowyDay) & ~callAndCough
```

Problem 3-B.13

Suppose you are working in your company's accounting department. Your boss gives you a set of rules and asks you to calculate the salary raise for each employee. Here are the rules:

> If an employee has fewer than six years of experience and has evaluations that are positive OR has six or more years of experience and the evaluations are not positive, the employee will get an eight percent raise.
>
> If an employee has six or more years of experience and has evaluations that are positive, the employee will get a 12 percent raise.
>
> Otherwise, the employee will not get any raise.

Write two rules in MATLAB and test your rules with the data in the following table. Set **`salary=35000`**, and apply the MATLAB expressions that implement the company rules.

Employee Name	Years of Experience	Evaluations	Raise (%)
Ms. Moore	5	1	?
Ms. Tenenbaum	11	0	?
Ms. Costanza	3	0	?
Mr. Cane	9	1	?

Now condense your two rules into one that will return the desired percent raise. Test your new rule on the data table again.

This is the most challenging problem in this set. If students struggle with this problem, then the following "hint" should help: What is the value of x = 8 * (2 < 4)? This is making use of MATLAB representation of TRUE and FALSE as 1 and 0.

Answers for the four cases are:

Ms. Moore, 8%

```
    Ms. Tenenbaum, 8%
    Ms. Costanza, 0%
    Mr. Cane, 12%
```

Code meeting the original problem spec is as follows:

```
% PROBLEM 3-B.13
  clc, clear
  salary = 35000

  yrExp = 5, eval = 1
  percentRaise_8 = ...
           (((yrExp < 6) & eval)  |  ((yrExp >= 6) & ~ eval))
  percentRaise_12 = ((yrExp >= 6)& eval)
  raise = 0.08 * percentRaise_8 + 0.12*percentRaise_12
  newSalary = salary * (1 + raise)

  % Now use the history window to execute the rest of the
  % table values for yrExp and eval.
```

Code meeting the augmented problem spec – to fuse the two rules of the original spec into one rule – follows:

```
% ------------------------------------------------------------
% PROBLEM 3-B.13 augmented
  clc, clear
  salary = 35000
  yrExp = 5, eval = 1
  raise = 0.08 * (((yrExp < 6) & eval) | ...
                  ((yrExp >= 6) & ~ eval)) + ...
          0.12 * ((yrExp >= 6)& eval)
  newSalary = salary * (1 + raise)
```

SOLUTIONS FOR CHAPTER 4
SAVING YOUR WORK IN MATLAB

This problem is very important. WIthout the ability to store MATLAB work, it will be difficulty for freshman learners to leverage MATLAB. In the larger view, the introduction to computational functions in this chapter lays the ground work for much of what will come in engineering, spanning such disparate topics as modularity in mechanical design and the use of basic operations in chemical engineering. In a very real sense, the ability to effectively decompose and modularized conceptual thinking about man made artifact is a central facet of most engineering disciplines.

Set A: Nuts and Bolts Problems for Saving Your Work

Problem 4-A.1

By default, which folder does MATLAB save files in? Which folder will MATLAB search first to load or run saved files? Write your answer in a concise paragraph or two.

The answer in both cases is the current directory (or connected directory).

Problem 4-A.2

What is the purpose of the Current Directory Window? What does Current Directory mean in MATLAB? Write your answer in a concise paragraph or two.

```
The purpose of the Current Director window is to graphically
show the contents of the default directory - the directory where
MATLAB saves files unless told otherwise, and where it looks for
files first.
```

Problem 4-A.3

In Section 4-1, you created a new folder named **ML_work** under **My Documents**. Create a new folder under **ML_work** and name it **Ch4_Saving**. Make **Ch4_Saving** the current directory.

Put your solutions for the following problems in **Ch4_Saving**.

Problem 4-A.4

The area and the perimeter of a circle are found with the following expressions:

$$circleArea = \pi r^2$$
$$circlePerimeter = 2\pi r$$

In the MATLAB Command Window, set **r** to be 4 meters. Record the units as a comment in the command.

Now write a MATLAB Script file called **circlePerimeterArea** that computes the area and the perimeter for a circle of radius **r**. Put appropriate comments into your script file indicating the following:

- Its file location
- Its purpose
- Any variables which are assumed to be set when the script file runs
- The units of the assumed variables

Use Figure 4-4 as an example. All lines but the last end in semicolons. The last line produces an echo of the variables the script file produces because it has no ending semicolon. Your script file should be similar to the one shown in Figure 4-4.

Save your work.

Run **circlePerimeterArea**.

Change the radius to 7 meters in Command Window and rerun the script file. Test your results.

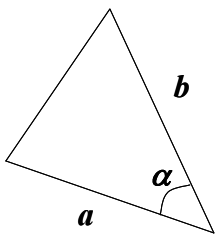
```
% file name => circlePerimeterArea.m
% Computes the area and perimeter of a circle of radius r
% Assumes that variable r is known when this script file
% runs.
% Assumed variables:
%     r                             % meters
% sample set up:
%   r = 7;                          % meters
% name of computed variables
%     circleArea                    % sq meters
%     circlePerimeter               % meters

  circleArea = pi * r^2             % sq meters
  circlePerimeter = pi * 2 * r      % meters
```

Problem 4-A.5

Suppose you know the lengths and the angle between two adjacent sides of a triangle, as shown in the sketch to the right. You can find the area of the triangle area from the following relation:

$$triangleArea = \frac{1}{2}ab\sin(\alpha)$$



In the Command Window, set side length **a** to be 5 ft, side length **b** to be 6 ft, and angle α to be 23°.

Enter **edit** to open MATLAB editor.

Write a script file to compute the area of the triangle using the relation above and store it in the variable **SAS_TriangleArea**. (The name comes from Side Angle Side computed area.)

The values of **a**, **b**, and α should not be set in your script file. Rather the script file will use the values of the variables you set in the Command Window. The trig built-in functions in MATLAB expect angles to be in radian units.

Store your script file giving it the name **computeSASarea**.

Run your script file, and test the results you obtain.

Reset **a**, **b**, and α to be 12 feet, 2 feet, and 78°, respectively.

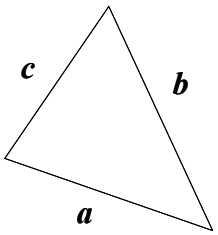
Run your script file again, and test your new results.

```
% file name => computeSASarea.m
% Computes the area of a triangle knowing two sides and the
% angle between the two sides.
% Assumed variables:
%     a, b: two sides of the triangle (ft)
%     alpha, the angle between a and b (deg)
% sample set up:
%    a = 12; b = 2;   % feet
%    alpha = 78;      % deg
% name of computed variables
%    SAS_TriangleArea

   alphaRad = alpha * (pi/180);   % deg
   SAS_TriangleArea = (1/2) * a * b * sin(alphaRad)  % sq ft
```

Problem 4-A.6

Suppose you know all the side lengths of a triangle, as shown in the sketch to the right. You can find the triangle area using Heron's formula:



$$triangleArea = \sqrt{S(S-a)(S-b)(S-c)}$$

S is the semi-perimeter and has the following value:

$$S = \frac{a+b+c}{2}$$

In the Command Window, set variables **a**, **b**, and **c**, to be 6 feet, 7 feet, and 3 feet, respectively.

Build a script file that computes the area of the triangle defined above, and stores the result in a variable named **SSS_TriangleArea**. Include a comment that explains your script file's purpose.

Save your script file; name it **computeSSSarea**.

Run your script file and test your results.

Reset **a**, **b**, and **c** to be 12 feet, 10 feet, and 15 feet, respectively. Run your script file again, and test your results.

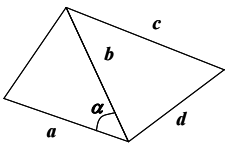
```
% file name => computeSSSarea.m
% Computes the area of a triangle the lengths of all three
% sides.
% Assumed variables:
%     a, b, c: all three sides of the triangle (ft)
% sample set up:
%   a = 12; b = 10; c=15;   % feet
% name of computed variables
%     SSS_TriangleArea

   S = (a + b + c) / 2;   % ft - the semiperimeter
   SSS_TriangleArea = sqrt(S * (S-a) * (S-b) * (S-c) ) % sq ft


% The expression above for SSS_TriangleArea can be
% rewritten as
%    1/4*(-(a+b+c)*(a-b-c)*(a-b+c)*(a+b-c))^(1/2)
```

Problem 4-A.7

Using the MATLAB script files **computeSASarea.m** and **computeSSSarea.m** that you wrote in Problem 4-A.5 and Problem 4-A.6, find the area of the shape shown in the sketch to the right.



First, in the Command Window, set the variables **a**, **b**, **c**, **d**, and α to be 5 feet, 6 feet, 7 feet, 11 feet, and 23°, respectively.

Create a new script file called **twoTriangles.m** whose purpose is to call **triAreaSAS** and **triAreaSSS** and add the results to determine the total area of the object in the sketch above. Set a variable named **sumBothTriangles** to hold the resulting total area.

Be careful with the names of variables you used in **triAreaSAS** and **triAreaSSS**. You may have to make changes in **triAreaSAS** and **triAreaSSS** to ensure consistent variable naming.

Put comments in **twoTriangles** that explains the purpose of your script file, and the names and meanings of variables used.

Run **twoTriangles** to test your result.

Now reset **a**, **b**, **c**, **d**, and α to be 3 feet, 5 feet, 7 feet, 15 feet, and 32°, respectively. Run **twoTriangles** again and test your new result.

This exercise has several points that should be emphasized. First, note that a "call" to a script file from another script is simply the name of the script. The script file name must not be used with an assignment statement to set a variable – as we will shortly do with function calls. Students most likely to trip on this point are students who have prior programming experience.

Secondly, if students take this simple sounding problem seriously, they will discover (probably) that using scripts to call other scripts can get tedious because the names of internal variables must be known. This point is made in the text of this chapter, but from a freshman learner's viewpoint, it is somewhat hard to grasp.

Thirdly, at instructor option, this exercise can be used to help students start thinking about how to test MATLAB code. The comments at the end of the script below suggest setting up a square and using it for test.

Finally, a variation on this exercise is to give the word description given in the comments of the code below as the problem spec, and require the students to make a sketch like that shown in the actual problem spec.

```
% file name => twoTriangles.m
% Computes the area of a rhombus constructed of two
% triangles: T1 and T2. For T1 we know two sides a and b,
% and the angle alpha which a makes with b. For T2, the side
% b is common with T1, and we know the other two sides c and
% d. Areas of T1 and T2 are found using other scripts, then
% the area of the rhombus is found by adding T1 and T2.
% Assumed variables:
%     a, b, c, d as described above (ft)
%     alpha as described above (deg)
% sample set up:
%     a=3; b=5; c=7; d=15;   % feet
```

```
%     alpha=32;              % deg
% name of computed variables
%     sumBothTriangles
% USES SCRIPTS
%    computeSASarea and computeSSSarea

  % Use computeSAS area to compute area of triangle formed
  % by a-ALPHA-b. Note the variables expected by computeSAS
  % area are compatible with the varaibles expected by
  % twoTriangles.
    computeSASarea

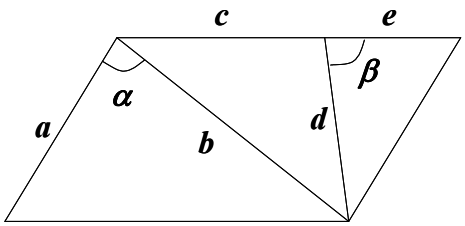
  % Use computeSSSarea to compute area of triange formed by
  % b-c-d. Note that computeSSSarea expects the three sides
  % to be named a-b-c. Thus a, b, and c will have to be
  % reset before calling computeSSSarea.
    a = b;
    b = d;
    c = d;
    computeSSSarea

  % Now just add the results.
    sumBothTriangles = SAS_TriangleArea + SSS_TriangleArea

  % A good test is to set
  %     a=10; b=10*sqrt(2); c=10; d=10;   % feet
  %     alpha=45;              % deg
  % The area should be 100 sq ft since the figure would be a
  % square of side 10. This does not prove the scirpt is
  % correct; it is just one test point.
```

Problem 4-A.8

Consider the parallelogram in the sketch to the right.



In the Command Window set variables **a**, **b**, **c**, **d**, **e**, α, and β to be 7 feet, 8 feet, 5 feet, 6 feet, 3 feet, 66°, and 85°, respectively.

Write a new script that calculates the area of the parallelogram. Use the scripts you wrote for Problem 4-A.5 and Problem 4-A.6. Name your calling script file **areaParallelogram**.

Put the result for the total area of the parallelogram in a variable named **sumAreaParallelogram**.

Keep the variables' names consistent across all scripts in your solution. You may need to make changes in scripts you wrote earlier.

Run **areaParallelogram** and test your result for the current settings of the problem variables.

Reset **a**, **b**, **c**, **d**, **e**, α, and β to be 3.5 feet, 4 feet, 2.5 feet, 3 feet, 1.5 feet, 66°, and 85°, respectively.

Run **areaParallelogram** again, and test your new result.

```
This exercise is similar to the A.7, but adds yet more complex-
ity. Students will find this problem very tedious because of the
need to hold on to the values of variables when the script is
first called, while at the same time reset variables to corre-
spond to the internal names of the called scripts.

The point to this exercise is call attention to the problems of
using scripts: the need to know the internal names of variables.
% file name => areaParallelogram.m
% Computes the area of a parallelogram using scripts for
% computing the area of a triangle using SAS and for
% computing the area of a triangle using SSS. See the
% graphic in the problem spec.
% Assumed variables:
%     a,b,c,d,e  linear measurements per the problem spec
%                diagram,  all in feet
%     alpha, beta  angles for the problem spec diagram, both
%                  in degrees
% sample set up:
%     a=3.5; b=4; c=2.5; d=3; e=1.5;    % feet
%     alpha=66; beta=85;                % deg
% name of computed variables
%     sumAreaParallelogram              % sq ft
% USES SCRIPTS:
%     computeSASarea and computeSSSarea

%  In order to use the scripts from before, the internal
%  names of variables have to match. To make that clear, the
%  first step will be to save the variables this scripts in
%  distinctive names.
    aTOP = a; bTOP = b; cTOP = c; dTOP = d; eTOP = e;
    alphaTOP = alpha; betaTOP = beta;

% Set up variables to use SAS to get the area of the
% triangle with alpha in it.
    a = aTOP;
```

```
    alpha = alphaTOP;
    b = bTOP;
    computeSASarea
    area_Alpha = SAS_TriangleArea;

% Set up variables to use SSS to get the area of the
% triangle with with sides b, c, d
    a = bTOP;
    b = cTOP;
    c = dTOP;
    computeSSSarea
    area_ThreeSides = SSS_TriangleArea;

% Set up varaibles to use SAS to get the area of the
% triangle with beta in it
    a = dTOP;
    alpha = betaTOP;
    b = eTOP;
    computeSASarea
    area_Beta = SAS_TriangleArea;

% total area is just the sum of the three parts computed
% above
    sumAreaParallelogram = area_Alpha + area_ThreeSides + ...
                                area_Beta

% A good preliminary test is
%     a=10; b=10*sqrt(2); c=10; d=10; e=0;   % feet
%     alpha=45; beta=90;                     % deg
% The answer should be 100 sq ft. (square of side 10 ft)
```

Problem 4-A.9

Answer the following questions in the context of workspaces:

- What do the terms *global workspace* and *local workspace* mean?
- How can you retrieve the value of a variable used in a function from the Command Window?
- Can you edit a function workspace from the Command Window in the normal mode of operation?
- Can you edit a script workspace from the Command Window in the normal mode of operation?

Answer all of the questions in one or two concise paragraphs.

Problems A.7 and especially A.8 are aimed the idea that scripts are difficult to use because they rely on the global workspace. This exercise asks students to bring together the concepts of the global workspace and the local workspace as used by functions in MATLAB.

Problem 4-A.10

Suppose you write the following functions (in 2 separate DOT-M files) for finding the sum-of-squares of two numbers:

```
ss1:   function [sqSum] = squareSum(a,b)
ss2:     sqA = square(a);
ss3:     sqB = square(b);
ss4:     sqSum = sqA + sqB;
```

```
sq1:function [sq] = square(a);
sq2:     temp = a;
sq3:     sq = temp^2;
```

Then, you test **squareSum** by running it from the MATLAB Command Window:

```
cw1:   >> clear;
cw2:   >> temp = 3^2 + 5^2;
cw3:   >> finalValueSqSum = squareSum(3,5)
```

Fill in the following table with the values of variables while MATLAB is running. Remember each function has its own workspace. Enter ***Unk*** if a variable is undefined (not in the workspace currently active).

The table with answers supplied is below.
Students should work this exercise through without using MATLAB first. Then using MATLAB and "red flag stops" check their work.

location of probe	a	b	sqA	sqB	sq	temp	finalValueSqSum
in Cmnd Wndw: **after cw2, before cw3**							
in **squareSum**: **after ss1, before ss2**							
in **square**: **1st call, after sq3**							
in **squareSum**: **after ss2, before ss3**							
in **square**: **2nd call, after sq3**							
in **squareSum**: **after ss3, before ss4**							
in **squareSum**: **after ss4**							
in Command Window: **after cw3**							

location of probe	a	b	sqA	sqB	sq	temp	finalValueSqSum
in Cmnd Wndw: **after cw2, before cw3**	UNK	UNK	UNK	UNK	UNK	34	UNK
in **squareSum**: **after ss1, before ss2**	3	5	UNK	UNK	UNK	UNK	UNK
in **square**: **1st call, after sq3**	3	UNK	UNK	UNK	9	3	UNK
in **squareSum**: **after ss2, before ss3**	3	5	9	UNK	UNK	UNK	UNK
in **square**: **2nd call, after sq3**	5	UNK	UNK	UNK	25	5	UNK
in **squareSum**: **after ss3, before ss4**	3	5	9	25	UNK	UNK	UNK
in **squareSum**: **after ss4**	3	5	9	25	UNK	UNK	UNK
in Command Window: **after cw3**	UNK	UNK	UNK	UNK	UNK	34	34

Problem 4-A.11

Write a MATLAB function that takes an angle in degrees and converts it to radians. Use this conversion relation:

$$angleInRadians = angleInDegrees \times \frac{\pi}{180}$$

Name your function **deg2rad**. (The name comes from "convert degrees to radians.") A function must minimally include comments indicating the function's purpose, input variables used by the function, and output variables used by the function.

Test your function with the following equation:

```
>> angleRad = deg2rad(45)
```

```
function angRad = deg2rad(angDeg)
% converts an angle from degrees to radians
% INPUT: angDeg, scalar, an angle in degrees
% OUTPUT: angRad, scalar, equivalent angle in radians
% sample call:
%     angleRad = deg2rad(45)

  angRad = (pi/180) * angDeg;  % radians
```

Problem 4-A.12

Write a MATLAB function that takes an angle in degrees and returns the cosine of the angle. In your function, do the following:

- Call **deg2rad** to convert the angle to radians.
- Find and return the cosine of the angle.

Name your function **cosDeg.m**.

Call your function from the MATLAB Command Window to test its operation.

```
>> cosAngle = cosDeg(74)
```

Include appropriate comments in your function.

```
function cosVal = cosDeg(angDeg)
% returns the cosine of an angle given in degrees
% INPUT: angDeg, scalar, angle in degrees
% OUTPUT: cosVal, scalar, the cosine of angDeg
% calls:
%   deg2rad
% sample call:
%     cosAngle = cosDeg(74)

  angRad = deg2rad(angDeg);
  cosVal = cos(angRad);

  % Alternative:
```

```
%   cosVal = cos(deg2rad(angDeg));
```

Problem 4-A.13

Modify your solution to Problem 4-A.12 so your function returns the angle in radians and its cosine.

Use ***SaveAs*** under the file menu in the MATLAB editor to create a copy of **`cosDeg`**; name the new copy **`radCosD`**. Test your function by calling it from MATLAB Command Window:

```
>> [angleRad, angleCos] = radCosD(74)
```

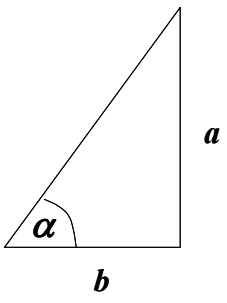
```
function [angRad, cosVal] = radCosD(angDeg)
% given an angle input in degrees, compute the equivalent
% number of radians and the cosine
% INPUT: angDeg, scalar, angle in degrees
% OUTPUT: cosVal, scalar, the cosine of angDeg
%         angRad, scalar, the equivalent number of radians
% calls:
%   deg2rad
% sample call
%    [angleRad, angleCos] = radCosD(74)

  angRad = deg2rad(angDeg);
  cosVal = cos(angRad)
```

Problem 4-A.14

Refer to the sketch to the right. For right triangles, the tangent of angle α is defined as the ratio of the opposite side length to the adjacent side length. The equation for the sketched triangle is:

$$Tangent(\alpha) = \frac{a}{b}$$



Write a MATLAB function that takes sides **a** and **b** (both in meters) and returns the tangent of α, as shown in the sketch. The order of the inputs to your function is important.

Name your function **tanTriangle.m**.

Test your function from the MATLAB Command Window:

```
>> tanAlpha = tanTriangle(5,3)
```

```
function tanVal = tanTriangle(oppositeSide, adjacentSide)
% compute the tangent of an angle in  triangle given the
% adjancent leg and the opposite leg of the triangle
% INPUT: oppositeSide, scalar, length of opposite leg (m)
%        adjacentSide, scalar, length of adjacent leg (m)
% OUTPUT: tanVal, scalar, the value of the tangent
% sample call:
%   tanAlpha = tanTriangle(5,3)

    tanVal = oppositeSide / adjacentSide;
```

Problem 4-A.15

Modify your solution for Problem 4-A.14 so the function returns the tangent and the value of the angle (in degrees).

Name your function for this problem **tanAndAngle**. Test your function by calling it from the Command Window and testing the result, use the following:

```
>> [tanAlpha, alphaInDeg] = tanAndAngle(5,3)
```

```
function [tanVal, angVal] = ...
                    tanAndAngle(oppositeSide, adjacentSide)
% compute the tangent of an angle in  triangle given the
% adjacent leg and the opposite leg of the triangle, and
% the value of the angle
% INPUT: oppositeSide, scalar, length of opposite leg (m)
%        adjacentSide, scalar, length of adjacent leg (m)
% OUTPUT: tanVal, scalar, the value of the tangent
%         angVal, scalar, value of the angle (deg)
% sample call:
%   [tanAlpha, alphaInDeg] = tanAndAngle(5,3)
```

```
tanVal = oppositeSide / adjacentSide;
angValRad = atan(tanVal);               % rad
angVal = (180/pi) * angValRad;          % deg
```

Problem 4-A.16

Write a MATLAB function that takes five scalar inputs and returns the arithmetic and geometric mean of the five numbers.

The mathematical definitions of the simple arithmetic mean of N numbers (a_1, a_2, …, a_i, …, a_N) and the geometric mean of the same N numbers are these two equations:

$$arithmeticMean = \left(\sum_{i=1}^{N} a_i \right) \times \frac{1}{N}$$

$$geometricMean = \left(\prod_{i=1}^{N} a_i \right)^{\frac{1}{N}}$$

(Look up geometric mean using a search engine such as Google if you don't understand the Π symbol.)

Name your function **arithGeomAvg**. Test your function by executing the following:

```
>> [arithM, geomM] = arithGeomAvg(5,3,8,5,7)
```

A useful question might be asked about an extension to this problem: Could the used to solve this problem also be used to find the arithmetic and geometric means for a set of 32 numbers? How about for a set of 5321 numbers? The discussion that follows can foreshadow the next chapter of the textbook, which focuses on MATLAB vectors.

```
function [aMean, gMean] = arithGeomAvg(n1, n2, n3, n4, n5)
% compute the arithmetic and geometric means for 5 numbers
% INPUT: n1-n5, scalar, simple numbers
```

```
% OUTPUT: aMean, scalar, the arithmetic mean of the inputs
%         gMean, scalar, the geometric mean of the inputs
% sample call:
%    [arithM, geomM] = arithGeomAvg(5,3,8,5,7)

   aMean = (n1+n2+n3+n4+n5)/5;
   gMean = (n1*n2*n3*n4*n5)^(1/5);
```

Problem 4-A.17

Which type of error is easier to detect a syntax or a runtime error? Why?

Write your answer in one or two concise paragraphs.

```
A syntax error is typically much easier to detect than a runtime
error. Syntax errors are trapped by MATLAB because a syntax
error is caused by a misuse in some way of the language of
MATLAB. A runtime (or logic) error on the other hand involves no
misuse of the MATLAB language, but rather a flaw in the logic of
the code.
```

Problem 4-A.18

Identify and correct the syntax errors in the following commands. After you believe you see the problem, and the correction you need, then type the original error and your correction into the Command Window to see if you are correct.

Part A y = 5(8-5)^2/7

Part B tan45 = sin[45(pi/180)]/cos[45(pi/180)];

Part C sinSquare45 = sin^2(45*pi/180)

Part D EricAge + 7 = 41

Part E (4.7) =< (10/4)

Part F (5/2) = (10/4)

```
Part A:   forgot multiplication sign after 5…
          y = 5*(8-5)^2/7

Part B:   multiple errors: (1) sine and cosine take round
          parenthesis, not square brackets, (2) forgot
          multiplication sign in both sine and cosine terms…
          tan45 = sin(45*(pi/180))/cos(45*(pi/180));

Part C:   raising a sine factor is accomplished by putting the
          <caret> *after* the entire sine term …
          sinSquare45 = sin(45*pi/180)^2

Pard D:   the left hand side of a assignment statement can
          include a variable name or variable names only. No
          operators or functions are allowed on the left hand
          side. It is difficult to "guess" at what the author of
          the code line in Part D meant. When faced with this
          kind of situation, a reasonable response is to simply
          ask the author what operation was intended.

Part E:   wrong order for the LESS THAN OR EQUAL TO symbols.
          (4.7) <= (10/4)

Part F:   the equality relational operator is a double equal
          symbol.
          (5/2) == (10/4)
```

Problem 4-A.19

The total interest on an initial investment of **M** dollars compounded annually at **r**% interest is given by …

$$interestEarned = M\left[\left(1+\frac{r}{100}\right)^{k}-1\right]$$

Use the following function to calculate the total interest and overall balance:

```
function [interest,balance]=bankAccount(M, r, k)
% This function calculates the account balance
% compounded annually over the interest rate,
% after a specified number of years
% Inputs: Initial investment: M (dollars)
%         percent interest rate: r (percent)
```

```
%          number of years: k (yrs)
% Outputs: interest earned: interest (dollars)
%          bank account balance: balance (dollars)
  interest = M((1+(r/100))^k-1);
  balance = M+interest*k;
```

Create a copy of the function above using the MATLAB editor and save it to your local machine.

Find and fix the syntax error(s).

Detect the runtime error(s) and fix them.

There is one syntax error: the command that computes "interest" is missing a multiplication sign.

There are two runtime errors. First the "r" variable stands for the interest rate as a percent, but the command computing the "interest" treats "r" as a decimal.

Second, the command for "balance" is conceptually wrong.

The two lines of operating code in the function should be …

```
interest = M*((1+r)^k-1);
balance = M+interest;
```

Problem 4-A.20

The area of a triangle for which all side lengths are known can be found using Heron's formula …

$$triangleArea = \sqrt{S(S-a)(S-b)(S-c)}$$

S is called the semi-perimeter and has the following value:

$$S = \frac{a+b+c}{2}$$

A novice programmer writes the following function to implement Heron's formula:

```
function area = SSSTriArea(a,b,c)
% This function calculates the area of a triangle
% for which the side lengths are known.
% Inputs: Side lengths of triangle: a (ft),
%            b(ft), c(ft)
% Output: Area of triangle: area (sq ft)
  A = sqrt(S)*(S-a)*(S-b)*(S-a);
  S = a+b+c;
```

Create this function and save to your hard disk.

Find and fix the syntax error(s).

Using the MATLAB debugging tool, find the runtime error(s), and fix them.

Try your function on a right triangle with side lengths 3, 4, and 5 feet using the following command line:

```
>> triArea = SSSTriArea(3,4,5)
```

Calculate this triangle's area by hand as well to test your answer.

The syntax error is that S is not computed before it is used. The order of the line computing A and S should be reversed.

One runtime error is that the command computing S is missing a divisor of 2.

A second runtime error is in the command computing A: the SQRT should run across the right hand side of the command, not be applied only to S.

A third runtime error is that the output variable "area" is not set in the function.

A fourth runtime error is that the term in the calculation of the area (S-a) is duplicated. One of the (S-a) terms should be changed to (S-c).

The two lines from "SSSTriArea" should read as follows:

```
S = (a+b+c) / 2;
area = sqrt(S*(S-a)*(S-b)*(S-c));
```

Set B: Problem-Solving Using MATLAB User-Defined Functions

Problem 4-B.1

You are designing a storage silo for corn. Your design uses a right prism with equilateral triangular bases. In your first try, each side of the triangular end plates has a length of 30 meters and the height of the silo is 10 meters. This silo design holds enough corn to meet requirements.

Make a rough sketch of the first silo design. (If you are unsure what a right prism is, look it up using Google.)

On further consideration, you see that the barn that houses the silo is short on floor space. Suppose that to make the base of the silo have a smaller footprint, you redesign a taller and skinnier silo but with the constraint that it must hold the same amount of corn as the first design. You decide that the redesigned silo should have a footprint only 75 percent of the first design but hold an equal amount of corn.

Make a rough sketch of the second silo design.

Your problem: Determine the dimensions of the second (and final) version of the silo. (Your solution will include the length of a side of triangular end caps and the height of the prism.)

To solve this problem, write the following **script files**:

- **`eqTriangleArea`**, whose purpose is to calculate the area of an equilateral triangle given the length of a side (in meters)

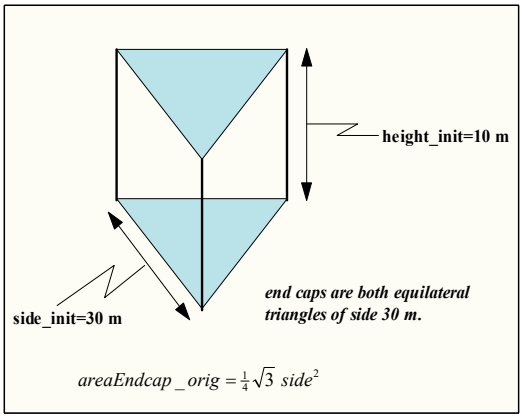
- **`eqTriangleSide`**, whose purpose is to compute the side of an equalaterial triangle given its area (in square meters)

- **`siloProblem`**, whose purpose is to compute the solution to this problem using the two other script files (This is the top-level script file.)

  The comments in a script file must include the variable's names that must be set before the script is run and the names of variables set by action of the script. If you don't know the mathematical formula you need to solve this problem, look up equilateral triangle using Google.

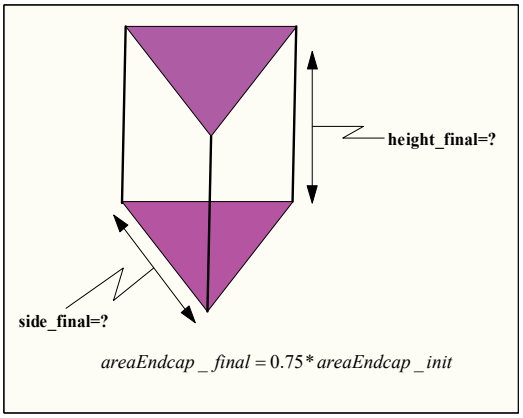
  Be sure to test your implementation.

A rough sketch of the original design for the silo and the final design are to the right.



Note that the sketches not only capture the geometry of the problem specification, but they also capture the target of the problem solving to be undertaken.

Many freshman learners have difficulty in translating a "word spec" into meaningful constructs, and in particular in simply figuring out what operations are being asked for in a problem specification. Using sketches as a way of clarifying the problem can prove a useful aid to many students.



A major point of this exercise is to drive home the point that making even simple programs with scripts can be quite tedious because of the need to keep variable names internal to each script in mind while writing other scripts. There are only three scripts in the problem spec of this problem. Yet even at this scale, students are likely to have some difficulty.

Notice the way that variables which must be set prior to the main script running are echoed to the command window at the beginning of the main script, and how the variables there are to be computed are also echoed to the command window at the end of the main script. Although this is useful, it is important for students to understand that the central action of scripts is to set variables in the workspace.

Scripts that implement the problem spec of this exercise follows.

```
% file name => siloProblem.m
% Computes the end cap sides and height of a silo per
% problem spec.
% Assumed variables:
```

```
%    height_init, scalar, the original silo's height (m)
%    side_init, scalar, the original silo's end cap side (m)
% sample set up:
%     height_init = 10;      % m
%     side_init = 30;        % m
% name of computed variables
%     height_final, scalar, height for final design of silo
%                           (m)
%     side_final, scalar, length of end cap for final
%                         design of silo (m)

     clc
     % display the variables that are supposed to be set
        disp('values for variables required to start')
        height_init, side_init

     % do the computations
     eqTri_side = side_init;                 % m
     eqTriangleArea
     area_init = eqTri_area;
     volume_init = area_init * height_init; % cu m

     % Final volume is supposed to be the same as first
     % design volume. Final area of the end cap is supposed
     % to be 75% of the initial area of the end cap. Thus
     % the final height must be 1/.75 times the original
     % height.

     height_final = (1/0.75) * height_init;
     area_final = 0.75 * area_init;
     eqTri_area = area_final;
     eqTriangleSide
     side_final = eqTri_side;

     % display the computed variables
     disp(' ')
     disp('values for computed variables')
     height_final, side_final
```

```
% file name => eqTriangleArea.m
% Computes the area of an equilateral triangle given a side
% Assumed variables:
%     eqTri_side, scalar, the length of a side in meters
% sample set up:
%     eqTri_side = 10;       % meters
% name of computed variables
%     eqTri_area             % sq meters

      eqTri_area = (sqrt(3)/2) * eqTri_side^2;  % sq meters
      % found using Google then  MathWorld
```

```
% file name => eqTriangleSide.m
% Computes the side of an equilateral triangle given its area
% Assumed variables:
%     eqTri_area, scalar, the area of an equilateral triangle
% sample set up:
%     eqTri_area = 8.66;        % sq meters
% name of computed variables
%     eqTri_side                % meters

      eqTri_side = sqrt(eqTri_area * 2 / sqrt(3));  % meters.
```

Problem 4-B.2

Redo Problem 4-B.1 to the same problem specification as before, but implement your solution with user-defined functions rather than scripts.

Create the following:

- **eqTriangleAreaFunc** that inputs side length of an equilateral triangle, computes the area of the triangle, and outputs the area
- **eqTriangleSideFunc** that inputs the area of an equilateral triangle, computes the length of a side the triangle, and outputs the side length
- **siloProblemFunc** that takes no inputs, computes a solution to the problem, and outputs the solution

Test your implementation.

Write a one- or two-paragraph comparison of using scripts in MATLAB versus using user-defined functions.

This exercise completes the story started in the previous problem, B.1. This is precisely the same problem spec, but now functions are to be used rather than scripts. This point can be further underlined by asking students to extend their solution to allow the "percentage of footprint" to be a variable of the problem.

Code implementing the problem spec using functions is below.

```
function [height_final,side_final] = ...
                    siloProblemFunc(height_init, side_init)
% Computes the end cap sides and height of a silo per
% problem spec.
% INPUTS height_init, scalar, the original silo's
%                              height (m)
%        side_init, scalar, the original silo's
%                            end cap side (m)
% OUTPUT: height_final, scalar, height for final design of
%                               silo (m)
%         side_final, scalar, length of end cap for final
%                               design of silo (m)
% sample call:
%     [h_fin,s_fin] = siloProblemFunc(10, 30)
% calls:
%    eqTriangleAreaFunc, eqTriangleSideFunc

     area_init = eqTriangleAreaFunc(side_init);   % sq m
     volume_init = area_init * height_init;       % cu m

     % Final volume is supposed to be the same as first
     % design volume. Final area of the end cap is supposed
     % to be 75% of the initial area of the end cap. Thus
     % the final height must be 1/.75 times the original
     % height.

     height_final = (1/0.75) * height_init;       % m
     area_final = 0.75 * area_init;               % sq m
     eqTri_area = area_final;
     eqTriangleSide
     side_final = eqTriangleSideFunc(area_final); % m


function eqTri_area = eqTriangleAreaFunc(eqTri_side)
% Computes the area of an equilateral triangle given its
% side length
% INPUT: eqTri_side, scalar, side of triangle (m)
% OUTPUT: eqTri_area, scalar, area of triangle (sq m)
% sample call
%     area = eqTriangleAreaFunc(20)

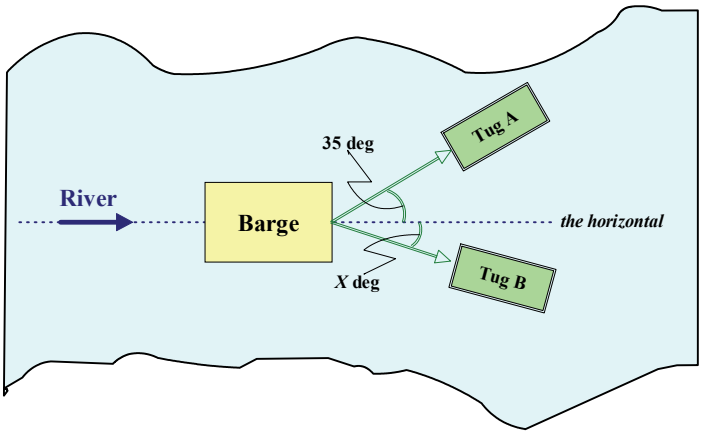
    eqTri_area = (sqrt(3)/2) * eqTri_side^2; % sq m



function eqTri_side = eqTriangleSideFunc(eqTri_area)
% Computes the side of an equilateral triangle given its area
% INPUT: eqTri_area, scalar, area of an equilateral
%                            triangle (sq m)
% OUTPUT: eqTri_side, scalar, triangle side (m)
% sample call:
%    area = 300;
%    side = eqTriangleSideFunc(area)
```

```
eqTri_side = sqrt(eqTri_area * 2 / sqrt(3));  % meters
```

Problem 4-B.3



The sketch at the right shows two tugboats pulling a barge in a river. Tugboats A and B pull the Barge with 2,000 and 4,000 lbs. forces, respectively. The sketch shows Tugboat A's direction of pull, 35 degrees from the horizontal in the specific case shown. Finally, the sketch shows Tugboat B pulling the barge at an unknown angle, X degrees, in the negative direction from the horizontal.

The goal of the two tugboats is to pull the barge straight down the river, which means along the horizontal. Part of the force exerted by each is along the horizontal while part of the force exerted by each is along an axis that is 90 degrees rotated from the horizontal. In order for the combined pull of the two tugboats to move the barge straight down the river, the pull of Tugboat A in the vertical direction must be as large as the pull of Tugboat B in the vertical but opposite in direction.

Write a Matlab function that calculates the angle labeled *X* in the sketch.

Your function should take three inputs: the force exerted by Tugboat A (in lbs), the force exerted by Tugboat B (in lbs), and the angle Tugboat A's pull makes with the horizontal (in degrees). Your function should compute and output the angle that Tugboat B's pull makes with the horizontal (in degrees) to satisfy the goal of moving the boat straight down the river. In addition to the standard comments, include in your function any comments about assumptions you must make to solve this problem. Call your function **`angle_TwoTugboats`**.

Test your function by executing the following:

```
>> angB = angle_TwoTugboats(2000, 4000, 35)
```

```
For most freshman learners, understanding this problem will be
dependent on understanding the diagram given in the problem
spec.

The code below is an implementation for one solution.


function tugB_angle = ...
       angle_TwoTugboats(tugA_force, tugB_force, tugA_angle)
% Compute the angle of Tugboat B in the diagram of the
% problem spec.
% INPUT: tugA_force, scalar, force A exerts (lbs)
%        tugB_force, scalar, force B exerts (lbs)
%        tugA_angle, scalar, angle A makes to horizontal
%                                                    (deg)
% OUTPUT: tugB_angle, scalar, angle B makes to horizontal
%                                                    (deg)
% sample call:
%     angB = angle_TwoTugboats(2000, 4000, 35)

% The problem spec set the only requirement to be that the
% barge must go "straight" down the river - i.e., along the
% horizontal. This means that the component of the force of
% TugA along the "vertical" the must be equal but opposite to
% TugB along the "vertical." (Vertical would be 90 degrees
% to the horizontal line marked in the problem spec.)

    tugA_rads = tugA_angle * (pi/180);           % radians
    tugA_vertical = tugA_force * sin(tugA_rads); % lbs
    sin_tugB_rads = tugA_vertical / tugB_force;  % lbs
    tugB_rads = asin(sin_tugB_rads);             % radians
    tugB_angle = tugB_rads * (180/pi);           % deg
```

Problem 4-B.4

The transient behavior of RLC circuits (containing resistors, inductors, and capacitors) plays an important role in the design of electronic components. Suppose an open RLC circuit has an initial charge of q_0 (in coulombs) stored within a capacitor. When the circuit is closed the charge flows out of the capacitor, leaving a charge q (in coulombs) after a time t given by the following relation:

$$q = q_0 e^{-\left[\frac{R}{2L}\right]t} \cos\left(\left[\frac{1}{LC} - \left(\frac{R}{2L}\right)^2\right]^{0.5} t\right)$$

R is the resistor in the circuit, **L** is the inductor in the circuit, **C** is the capacitor in the circuit, $\mathbf{q_0}$ is the initial charge of the capacitor in the circuit, **t** is time since the circuit has been closed, and **q** is the charge remaining on the capacitor at time **t**. The standard units for **R**, **L**, **C**, **q** (and $\mathbf{q_0}$), and **t** are ohms, henries, farads, coulombs, and seconds, respectively.

Write a MATLAB function that takes five inputs for an RLC circuit: **R** (in ohms), **L** (in henries), **C** (in farads), $\mathbf{q_0}$ (in coulombs), and **t** (in seconds). Your function should compute and output one variable: **q**, the charge remaining in the capacitor at time **t**.

Name your function **q_RLC**.

Perform an initial test of your function by executing the following:

```
>> R = 100;       % ohms
>> L = 9.2;       % henries
>> C = 0.0025;    % farads
>> q0 = 1000;     % columbs
>> chargeRemaining_00 = q_RLC(R,L,C,q0,0)
```

Your answer should be the value of **q** at time equal to zero, i.e., 1,000 coulombs. (Why is this a reasonable initial test?)

Now run your function **q_RLC** for **t=0.1**, **t=0.2**, **t=0.3**, and **t=0.4** seconds.

Create a plot of *Charge versus Time* for the five value pairs you have.

How would you characterize the relation between charge remaining and time? Write a concise paragraph describing your characterization.

This exercise is aimed at providing freshman learners experience in writing a relatively complex MATLAB expression. The key lesson that should be learned is to decompose a complex expression, write expressions for each of the resulting fragments, then compose.

The function below meets the problem specs. A second function below will create a plot that is part of the spec, but note that the plot is specified, not the tool used to create the plot.

Students familiar with EXCEL can use it to create the plot, or as a fall back, a hand drawn plot will suffice.

The relation between charge remaining and time is an exponential decay.

```
function q = q_RLC(R, L, C, q0, t)
% compute the charge in an RLC circuit
% INPUT: R, scalar, the circuit resistance (ohms)
%        L, scalar, the circuit inductance (henries)
%        C, scalar, the circuit capacitance (farads)
%        q0, scalar, the initial circuit charge (coulombs)
%        t, scalar, time since the circuit was closed (sec)
% OUTPUT: q, scalar, charge remaining in the circuit
%                    after t seconds
% sample call:
%     qRem = q_RLC(100, 9.2, 0.0025, 1000, 0)

    eTerm = exp(-R*t/(2*L));
    cosTerm = cos( sqrt((1/(L*C)) - (R/(2*L))^2 ) * t);
    q = q0 * eTerm * cosTerm;
```

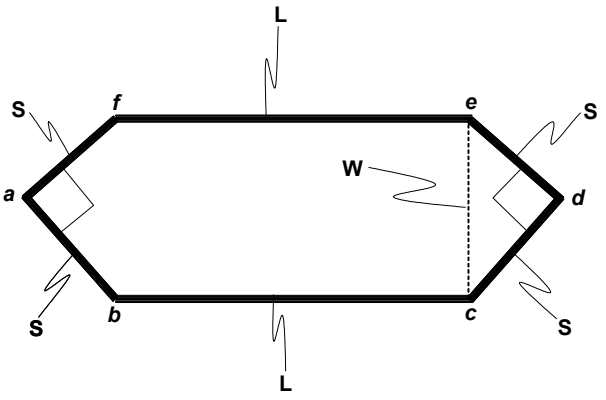
```
function q_RLC_plot
% makes the plot speced in Problem B.4

   t = 0:.1:.4;
   for i = 1:length(t)
      q(i) = q_RLC(100, 9.2, 0.0025, 1000, t(i));
   end

   figure
   plot(t, q, '--rs','LineWidth',2,...
      'MarkerEdgeColor','k',...
      'MarkerFaceColor','g',...
      'MarkerSize',10)
   xlabel('time (sec)',  'FontWeight', 'bold');
   ylabel('charge remaining (coulombs)', 'FontWeight', 'bold');
   grid on
   titleStr = ['q = q0 * exp(-R*t/(2*L)) * ', ...
               'cos( sqrt((1/(L*C)) - (R/(2*L))^2 ) * t)'];
   title(texlabel(titleStr), 'FontSize', 14)
```

Problem 4-B.5

A fence is needed around the field sketched at the right. The field consists of a rectangle (**bcef**) and two triangles (**abf** and **cde**). The necessary fence is indicated by the heavy line in the sketch.



The rectangular part of the field has dimensions **W** (meters) by **L** (meters). The triangles are the same except for a rotation; both are right isosceles triangles with two sides of length **S** and one side of length **W**.

The total area of the field is **A** square meters.

Your job is to calculate the fence cost required for the field. From land records, you know the value of **A**. From measurement, you know the value of **S**. And you are given the cost per linear meter of the fence.

Work out a solution to this problem on paper. Then implement your solution by developing MATLAB functions. Do not do this problem by writing one function. Decompose the problem by writing the functions that compute the following:

1. The length of **W** (meters) given the length of **S** (meters). Name this function **`wGivenS`**.
2. The area of a right isosceles triangle given the length of one of the isosceles sides (in meters). Name this function **`area_rtIsTri`**.
3. The side length of a rectangle given its area and width: **`length_rec`**.
4. Top-level function. The cost of the fence given the area of the field **A**, the length of an isosceles side **S**, and the unit cost of the fence. Name this function **`priceFenceTri`**.

Once you have your functions written and the syntax bugs deleted, be certain you test your solution with several reasonable inputs.

```
This exercise should focus the student on issues of decomposi-
tion to make problem solving modular - both to make implementa-
tions be reusable and to enable more straightforward solutions.

A solution follows.

function cost = priceFenceTri(fieldArea, isSide, unitCostFence)
% Compute the cost of the fence around a field specified in
% the problem description.
% INPUT: fieldArea, scalar, the total area of field (sq m)
%        isSide, scalar, length of an isosceles side (m)
%        unitCostFense, scalar, unit cost of fence (%/m)
% OUTPUT: cost, scalar, the total cost of a fence around the
%                       perimeter of the field ($)
% sample call:
%   cost = priceFenceTri(100, 10, 2.25)
% calls:
%    area_rtIsTri, wGivenS

  areaInEnds = 2 * area_rtIsTri(isSide);
  areaInRect = fieldArea - areaInEnds;
  lengthRec = length_rec(areaInRect, wGivenS(isSide));
  totalPerimeter = 2*lengthRec + 4*isSide;
  cost = totalPerimeter * unitCostFence;

function side = length_rec(area, width)
% Compute the side length of a rectangle given its
% area and width.
% INPUT: area, scalar, the area of the rectangle (sq m)
%        width, scalar, the width of the rectangle (m)
% OUTPUT: side, scalar, the width of the rectangle (m)
% sample call:
%    s = length_rec(100, 20)

   side = area / width;    % m
function area = area_rtIsTri(isocSide)
% compute area of a right isosceles triangle given the
% length of one of the isosceles sides (in meters)
% INPUT: isocSide, scalar, isosceles side of triangle (m)
% OUTPUT: area, scalar, area of triangle (sq m)
% sample call:
%    A = area_rtIsTri(10)
% calls:
%     wGivenS

  base = wGivenS(isocSide);
  area = base * (base/2) / 2;      % sq m


function baseSide = wGivenS(isocSide)
```

```
% computes the length of the base of a right isosceles
% triangle given the length of an isosceles side
% INPUT: isocSide, scalar, length of an isosceles side (m)
% OUTPUT: baseSide, scalar, length of the base (m)
% sample call:
%     base = wGivenS(10)

   % Construct a line from d that cuts W in half. This line
   % will make a right angle with W, by symmetry. Also by
   % symmetry, the angle edc will be bisected, thus it will
   % be an
   baseSide = sqrt(2) * isocSide;
```

Problem 4-B.6

Suppose you write the following function to find the square of the ratio of two numbers:

```
function divsq = ratioSquare(x1, x2)

% This function finds the square of the ratio of
% two numbers
% Inputs x1 and x2, simple numbers
% Output divsq is the square of the ratio x1/x2
    divsq = x1/x2^2;
```

Then, you run **ratioSquare** in Matlab Command Window:

```
>> x1 = 640;
>> x2 = 320;
>> divSquared = ratioSquare(x2,x1)
```

What should the values of **x1**, **x2** and **divSquared** be after the computation is over? What value do you get when you run the function?

Fix any runtime errors in **ratioSquare** and retry.

```
The runtime error is in the misuse of the precedence of MATLAB.
In order to get the desired result, the last line of the
function should read
```

```
divsq = (x1/x2)^2;
```

Problem 4-B.7

You write the following function to add three numbers at a time:

```
function [result] = adding(i1,i2,i3);
% This function returns the sum of its three
% scalar inputs
% Inputs: i1, i2, i3 ...  are three real numbers
% Outputs: result is the sum of the three numbers
   temp = i1 + i2;
   result = temp + i3;
```

Then you call **adding** from the Matlab Command Window:

```
>> a = 2
>> temp = 3
>> c = adding(a, a, a)
```

When execution of **adding** completes, what is the value of **temp**?

Solve the problem by hand first, then write the function and run it to test your answer.

The simple exercise is to underline for students the concepts of the workspace, and in particular the workspace of a function. The value of "temp" after "adding" is executed is 3. The variable named "temp" inside the function is different from the variable "temp" defined in the global workspace.

Problem 4-B.8

You write the following function to find the average of three numbers:

```
function ave = myAverage(x1, x2, x3)
% This function finds the average of three real
% numbers
% Inputs: x1, x2, x3 are all real numbers
```

```
% Output: ave is the arithmetic average of the
%         three inputs
  ave = sumInputs/3;
```

You call **myAverage** from the Command Window as follows:

```
>> x1 = 5; x2 = 7; x3 = 2;
>> sumInputs = x1+x2+x3;
>> average = myAverage(x1,x2,x3)
```

What will MATLAB return?

Write a paragraph explaining what is wrong with **myAverage**.

The function call will result in a syntax error. The root of the error is that "sumInputs" is defined in the global workspace, but then it is used in the workspace of "myAverage."

Problem 4-B.9

The solutions for values of x in the quadratic polynomial $y = ax^2 + bx + c$ can be found this way:

$$x_1 = \frac{-b + \sqrt{b^2 - 4ac}}{2a}$$

$$x_2 = \frac{-b - \sqrt{b^2 - 4ac}}{2a}$$

Write a Matlab function called **qRootsGreaterTen** that inputs coefficients a, b, and c. Your function should return a single output as follows:

- If $\mathbf{x_1}$ and $\mathbf{x_2}$ are real numbers greater than 10, then output **1**.
- Otherwise, output **0**.

(Hint: the square root of a negative number is not a real number. In your function, first compute the values for $\mathbf{x_1}$ and $\mathbf{x_2}$. Then, to compute the output for **qRootsGreaterTen**, form a logical expression that tests to see if both are greater than 10 and makes certain that the values are real numbers.)

```
There is a MATLAB built-in function "imag" that will extract the
imaginary part of a complex number, and thus can be used to test
if a number is real. However, using the hint given in the prob-
lem spec should suffice.

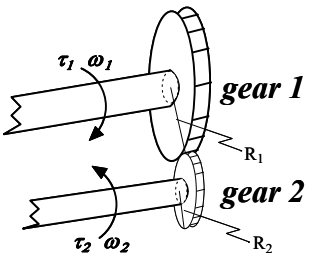
Code follows.

function greater10 = qRootsGreaterTen(a,b,c)
% Computes the two roots of a quadratic, and if both roots
% are real numbers and greater than 10, then return 1, else
% return 0.
% INPUT: a,b,c... scalars, the coefficients in
%           f(x) = ax^2 + bx + c
% OUTPUT: greater10, either 0 or 1
% sample call:
%    x = qRootsGreaterTen(1, 2, 3)

  sqRtTerm = b^2 - 4*a*c;
  x1 = (-b + sqrt(sqRtTerm)) / (2*a);
  x2 = (-b - sqrt(sqRtTerm)) / (2*a);
  greater10 = (sqRtTerm >= 0) & (x1 > 10) & (x2 > 10);
```

Problem 4-B.10

Automobile transmissions are used to reduce the torque (the turning force on a shaft) while increasing the angular velocity (how rapidly the shaft spins), or vice versa. Consider the simplified transmission model shown in the rough sketch at the right.



The sketch shows two gears hooked together, or coupled. Real transmissions typically have many more than two gears. Whether simple or complex, systems of gears can be understood by noting the distance that any one gear must rotate. For example, if Gear 1 has a radius half as big as Gear 2, then when Gear 1 completes one complete rotation, Gear 2 will complete only 180 degrees of rotation. This intuition leads to a relation between the angular speed of Gear 1, its radius, and the angular speed of Gear 2 and its radius. Assuming that the teeth on Gear 1 and Gear 2 have the same spacing, the relation is the following:

$$\frac{R_1}{R_2} = \frac{\omega_2}{\omega_1}$$

ω is the angular speed and ***R*** is the radius of a gear.

Write a function that inputs the radius of Gear 1, the angular speed of Gear 1, and the desired angular speed of Gear 2. The function should compute and output the radius that Gear 2 to meet the design goal for the gear system (that is, the desired angular speed of Gear 2). Name your function **`gear2_rotSpeed`**.

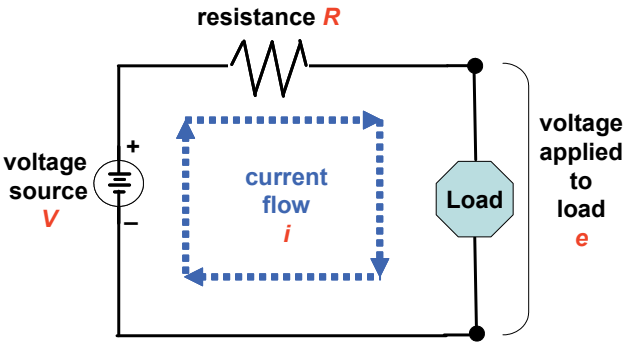
This problem is straightforward. The only difficulty for a freshman learner will be to read and understand the problem specification. Hence this problem is more focused on issues of problem spec understanding than on issues of MATLAB syntax.

Code follows.

```
function gear2_requiredRad = ...
         gear2_rotSpeed(gear1Rad, gear1AngSp, gear2DesAngSp)
% For a two-gear system, computes the output gear radius
% required to produce a given output gear angular speed,
% also given input gear radius and angular speed.
% INPUT: gear1Rad, scalar, input gear radius
%        gear1AngSp, scalar, input gear angular speed
%        gear2DesAngSp, scalar, output gear desired ang sp
% OUTPUT: gear2_requiredRad, scalar, output gear radius
% sample call:
%    rad1 = 10; angSp1 = 300; angSp2 = 10;
%    rad2 = gear2_rotSpeed(rad1, angSp1, angSp2)

  gear2_requiredRad = gear1Rad * (gear1AngSp/gear2DesAngSp);
```

Problem 4-B.11



In the sketch to the right, an electric circuit is shown. The circuit consists of a source of electric potential (*V* volts) that creates an electric current (***i*** amps) through a resistance (***R*** ohms) and some load (like an electric motor). The purpose of the resistance ***R*** is to drop the voltage to some desirable level **e** required to make the load work properly.

The voltage applied across the load, ***e***, can be determined from this equation:

$$e = V - iR$$

Write a MATLAB function named **`resistanceGivenV_I`** that computes the value of ***R*** given ***V***, ***i***, and ***e***.

Use your function to compute the correct design value for the resistance using these values:

- The electric potential ***V*** is 30 volts
- The current flowing ***i*** is 5 amps
- The desired voltage ***e*** applied to the load is 8 volts

This problem is again focused primarily on reading the problem spec. Once understood, the implementation of a solution is straightforward.

Code follows.

```
function R = resistanceGivenV_I(V, i, e)
% For an voltage and resistance circuit, computes the
% resistance R across a load given the current through the
% load i, the voltage across the load e, and the driving
% voltage V.
% INPUT: V, scalar, voltage driving the circuit (V)
%        i, scalar, current through the load (amps)
%        e, scalar, voltage across the load (V)
```

```
% sample call:
%      voltsBat = 30; current = 5; voltsDrop = 8;
%      R = resistanceGivenV_I(voltsBat, current, voltsDrop)

      R = (V - e) /  i;
```

Problem 4-B.12

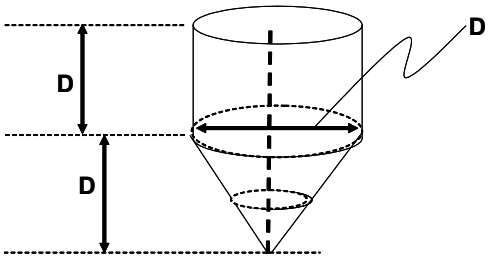
A container is made up of a cylinder and a cone at the bottom end. The cylinder height and the cone height are equal to the cylinder diameter.

Make a sketch of the container. Be sure to label your sketch and understand the physical situation with which you are working.

Write a MATLAB function that takes a requested container volume as input and returns the cylinder diameter. Name your function **`cylDiameter`**.

This problem is like the preceding problems: the issue for student learners is to understand the problem spec.

A sketch similar to that at the right should be completed by students. Following that, the problem should be straightforward.



Code follows.

```
function D = cylDiameter(vol)
% Consider a container that is a cylinder of diameter D and
% height D with a right circular cone attached to one end.
% The cone has a height also of D. Given the volume of the
% container, this function computes D.
% INPUT: vol, scalar, the area of the container
% OUTPUT: diam, scalar, D (in the above desc)
% sample call:
%      statedVol = 100;
%      required_diam = cylDiameter(statedVol)

    % vol as function of D
    % vol = (1/3)*pi*(D/2)^2*D  +  pi*(D/2)^2*D
    % vol = pi*(D/2)^2*D * (4/3)
```

```
%   invert...
D = ((3/pi) * vol)^(1/3);
```

Problem 4-B.13

You are testing the following trig identity:

$$(\sin(\alpha))^2 + (\cos(\alpha))^2 = 1, \text{ for all } \alpha$$

You write a function called **pythagorasForUnitCircle** and run it for 0, 30, 60, and 90 degrees. Unfortunately, the program produces large numbers of error messages. You conclude the identity is incorrect. Your conclusion is obviously wrong, and an error message indicates bad MATLAB syntax.

Study your code below carefully. Determine the syntax and/or runtime error(s) and fix them.

Run the code with several inputs to test the improved version of **pythagorasForUnitCircle**:

```
function trueOrFalse = pythagorasForUnitCircle(angle)
% This function is written to verify the identity
%                 cos^2(angle)+sin^2(angle) = 1
% Input: angle... an angle in degrees
% Output: trueOrFalse ... 1 if identity is verified,
% 0 otherwise

   sincos = cos^2(angle)+sin^2(angle);
   trueOrFalse = (sincos=1);
```

There are two syntax errors in the function: (1) the test on equality in the last line should be "==" and (2) the cosine and sine squared terms in the next to last line should be of form like cos(x)^2.

There is also one runtime error: the angle is input in degrees and must be converted to radians. The two lines of code should read as follow:

```
angle = angle * (pi/180);              % radians
sincos = cos(angle)^2 + sin(angle)^2;
```

```
trueOrFalse = (sincos=1);
```

Problem 4-B.14

You need to write a MATLAB function to compute ***L(x)*** given by the following equation (where $0 < x < 1$):

$$L = 100\left(\frac{x}{0.6}\right)^{0.625}\left(\frac{1-x}{0.4}\right)^{-1.625}$$

The function you write as a first draft is the following:

```
function L = computeL(x)
% This function calculates L(x) per the problem spec
% Input: x
% Output: L

   L = 100 * x/0.6^0.625 * 1-x/0.4^-1.625;
```

Find and correct the errors in your first draft.

Although this is the last problem in this chapter, it is straightforward. The code below corrects the errors, which are caused by unintended application of precedence rules.

```
function L = computeL(x)
% This function calculates L(x) per the problem
% spec
% Input: x
% Output: L

%   L2 = 100 * x/0.6^0.625 * 1-x/0.4^-1.625;

    firstTerm = (x/0.6)^0.625;
    secondTerm = ((1-x)/0.4)^-1.625;
    L = 100 * firstTerm * secondTerm;

% Or in one line...
%    L = 100 * (x/0.6)^0.625 * ((1-x)/0.4)^-1.625;
```

SOLUTIONS FOR CHAPTER 5
VECTOR OPERATIONS

The problems in this chapter in part are exemplars of what distinguishes MATLAB: a very strong ability to perform multiple operations in one clear line of code by using vector operations. Students should try to keep in mind as they work these problems that a good deal of the material in this chapter can be understood as a generalization of scalar operations.

Students typically have difficulty with content addressing, using the find built-in function to identify the locations in vectors whose contents satisfy set conditions. It is important especially that they master this technique in order to make good use of the power offered in MATLAB.

Set A: Nuts and Bolts Problems for Vectors

Problem 5-A.1

Create a row vector **a1** consisting of the numbers in the ordered set {1 5 7 22 12} using the square bracket operator.

```
a1 = [1 5 7 22 12]

OR

a1 = [1, 5, 7, 22, 12]
```

Problem 5-A.2

Create a row vector **a2** consisting of the numbers in the ordered set {1 2 3 4 5} using the square bracket operator.

```
a2 = [1 2 3 4 5]

OR

a2 = [1, 2, 3, 4, 5]
```

Problem 5-A.3

Create a row vector **a3** consisting of the numbers in the ordered set {1 2 3 4 5} using the colon operator.

```
a3 = 1:1:5

OR

a3 = 1:5
```

Problem 5-A.4

Create a row vector **a4** consisting of the numbers in the ordered set {1 4 7 10} using the square bracket operator.

```
a4 = [1, 4, 7, 10]

OR
```

```
a4 = [1 4 7 10]
```

Problem 5-A.5

Create a row vector **a5** consisting of the numbers in the ordered set {1 4 7 10} using the colon operator.

```
a5 = 1:3:10
```

Problem 5-A.6

Create a row vector **a6** starting with the element 1, ending with the element 10, with five equally spaced points in the vector. Use **linspace**.

```
a6 = linspace(1,10,5)

Its important that students understand the problem spec "equally
spaced points." This matches the definition of linspace. If
however, the problem spec had been for "equal length intervals"
then the solution would have been linspace(1,10,6).
```

Problem 5-A.7

Create a row vector **a7** starting with the element 1, ending with the element 10, with five equally spaced points in the vector. Use the colon operator.

```
numPts = 5
numIntervals = numPts - 1
legnthTotalInt = 10 - 1
lengthBetweenPoints = legnthTotalInt / numIntervals
```

```
a7 = 1:lengthBetweenPoints:10

The point of this exercise and the one that follows is to show
students that the colon operator fits for some operations, but
not for others. The colon operator is designed for operations
when the length between points and the final point in a vector
under construction are known. The "easy" MATLAB function to use
for this exercise is linspace.
```

Problem 5-A.8

Create a row vector **a8** starting with the element 1, ending with the element 10, with five equally spaced points in the vector. Use the square bracket operator.

```
a8 = [1, 3.25, 5.5, 7.75, 10]
```

Problem 5-A.9

Build a row vector **a9** that has ten elements, each with the value 1. Use the MATLAB built-in function **ones**.

```
a9 = ones(1,10)
```

Problem 5-A.10

Build a row vector **a10** that has five elements, each with the value 33. Use the built-in function **ones**.

```
a10 = 33 * ones(1,5)
```

Problem 5-A.11

In MATLAB, define variable **x** as follows:

```
>> x = [3, 9, 4, 99, 2]
```

Set a variable **y** to be the number of elements in **x** – use **length**.

```
y = length(x)
```

Problem 5-A.12

In MATLAB, define variable **x** as follows:

```
>> x = [3, 9, 4, 99, 2]
```

Set a variable **y** to be the first and fourth elements of **x**.

```
y = x([1,4])

Students many times will miss the square bracket and try to use
x(1,4). The error being made is to not understand that x([1,4])
is really a two step operation: first create an index vector,
then use the index vector to address the desired elements of x.
```

Problem 5-A.13

In MATLAB, define variable **x** as follows:

```
>> x = [3, 9, 4, 99, 2]
```

Set a variable **y** to be the first, second, and third elements of **x**. Use the colon operator in your solution.

```
y = x(1:3)

Note that the colon operator can easily be confused by students
with the colon symbol use to access all elements of a vector (or
array).
```

Problem 5-A.14

In MATLAB, define variable **x** as follows:

```
>> x = [3, 9, 4, 99, 2]
```

Set a variable **y** to be the third through the last element of **x** so your solution works no matter how many elements are in **x**.

```
y = x(3:end)

The use of "end" inside an index is important in MATLAB because
it allows general solutions – the phrase "works no matter how
many elements …" should be a tip off to students in this
exercise to use "end."
```

Problem 5-A.15

In MATLAB, define variable **x** as follows:

```
>> x = [3, 9, 4, 99, 2]
```

Set a variable **y** to be the next-to-last and the last element of **x** so your solution works no matter how many elements are in **x**.

```
y = x(end-1:end)

A common error students make on this exercise is to write x(end-
1, end) again because of a misunderstanding of indexing.
```

Problem 5-A.16

In MATLAB, define variable **x** as follows:

```
>> x = [3, 9, 4, 99, 2]
```

Change the second element of **x** to be 3.

```
x(2) = 3
```

Problem 5-A.17

In MATLAB, define variable **x** as follows:

```
>> x = [3, 9, 4, 99, 2]
```

Change the second element of **x** to be 102 and the fourth element of **x** be 205.

```
x(2) = 102
x(4) = 205

OR

x([2,4]) = [102,205]
```

```
The second solution in which both elements are changed in one
MATLAB operation is preferable, but it also is not obvious to
most beginning students.
```

Problem 5-A.18

In MATLAB, define variable **x** as follows:

```
>> x = [3, 9, 4, 99, 2]
```

Change the last element of **x** to be 999 so that your solution works no matter the length of **x**.

```
x(end) = 999

Using an index of "end" is relatively straightforward for
students for READ operations. It is far less intuitive at the
outset for WRITE operations.
```

Problem 5-A.19

In MATLAB, define variable **x** as follows:

```
>> x = [3, 9, 4, 99, 2]
```

Change the next-to-last element of **x** to be 55 and the last element to be 65 so that your solution works no matter the length of **x**.

```
x(end-1) = 55
x(end) = 65

OR

x([end-1:end]) = [55, 65]
```

```
The second solution requires that students understand that when
replacing a subvector, the size of the subvector to be replaced
must match the length of the replacement vector.
```

Problem 5-A.20

In MATLAB, define variable **x** as follows:

```
>> x = [3, 9, 4, 99, 2]
```

Delete the next to last element of **x** so it will leave **x** as a vector with only four elements.

```
x(end-1) = [];

Students many times mistake deleting an element with setting an
element to zero.
```

Problem 5-A.21

In MATLAB, define variable **x** as follows:

```
>> x = [3, 9, 4, 99, 2]
```

Change **x** into a column vector.

```
x = x'
```

Problem 5-A.22

Create a column vector **y** whose elements are {2, 40, 1} using the square bracket operator.

```
y = [2,40,1]
y = y'

OR

y = [2,40,1]'
```

Problem 5-A.23

Using the **x** vector result from Problem 5-A.21 and the **y** vector result from Problem 5-A.22, create a column vector **z** whose first five elements are from **x** and whose next three elements are from **y**.

```
z = [x;y]

A typical student problem with this exercise is to try [x,y].
```

Problem 5-A.24

Change the **z** vector result from Problem 5-A.23 to a row vector.

```
z = z'
```

Problem 5-A.25

Define **x** as follows:

```
>> x = [3; 9; 4; 5; 2]
```

Assume that values in **x** are angles in radians. Create a vector **y** such that each element in **y** is the sine of the corresponding element in **x**.

```
y = sin(x)
```

Problem 5-A.26

Define **x** as follows:

```
>> x = [3; 9; 4; 5; 2]
```

Create a vector **y** so each element y_i has the value e^{x_i} for the corresponding i^{th} element.in **x**.

```
y = exp(x)
```

Problem 5-A.27

Define **x** as follows:

```
>> x = [3; 9; 4; 5; 2]
```

Create a variable **s** which holds the sum of the elements of **x**.

```
s = sum(x)
```

Problem 5-A.28

Define **x** as follows:

```
>> x = [3; 9; 4; 5; 2]
```

Create a variable **myMax**, which is the maximum value of all elements in **x**.

```
myMax = max(x)
```

Problem 5-A.29

Define **x** as follows:

```
>> x = [3; 9; 4; 5; 2]
```

Create a variable **myMean** which is the arithmetic mean of all scalar elements in **x**.

```
myMean = mean(x)
```

Problem 5-A.30

Define **x** as follows:

```
>> x = [3; 9; 4; 5; 2]
```

Create a variable **mySort**, which contains all values in **x** in sorted order.

```
mySort = sort(x)
```

Problem 5-A.31

In MATLAB, define variables **x** and **y** as follows:

```
>> x = [3; 9; 4; 5; 2]
```

```
>> y = [2; 1; 4; 3; 2]
```

Create a new vector **z** in which each element is the sum of the corresponding elements in **x** and **y**.

```
z = x + y
```

Problem 5-A.32

In MATLAB, define variables **x** and **y** as follows:

```
>> x = [3; 9; 4; 5; 2]

>> y = [2; 1; 4; 3; 2]
```

Create a new vector **z** in which each element is twice the value of the corresponding element in **x**, added to three times the corresponding value in **y**.

```
z = 2*x + 3*y
```

Problem 5-A.33

In MATLAB, define variables **x** and **y** as follows:

```
>> x = [3; 9; 4; 5; 2]

>> y = [2; 1; 4; 3; 2]
```

Create a new vector **z** in which each element is the product of the corresponding elements in **x** and **y**.

```
z = x .* y
```

```
A common student error is to not include the DOT to indicate a
cell-by-cell multiplication.
```

Problem 5-A.34

In MATLAB, define variables **x** and **y** as follows:

```
>> x = [3; 9; 4; 5; 2]

>> y = [2; 1; 4; 3; 2]
```

Create a new vector **z** in which each ith element in **z** is $(x_i)^{y_i}$.

```
z = x .^ y
```

Problem 5-A.35

In MATLAB, define variables **x** and **y** as follows:

```
>> x = [3; 9; 4; 5; 2]

>> y = [2; 1; 4; 3; 2]
```

Create a new vector **z** in which each element is three times the value of the corresponding value in **x**, divided by five times the value of the corresponding value in **y**.

```
z = 3*x ./ (5*y)

The multiplication operations need not use the DOT because one
side is a scalar. When introduced to cell-by-cell operations,
students have a tendency to use a DOT for every multiplication,
division, and exponentiation. It will be helpful later if
students now use the DOT versions as needed. That is, if
```

```
students now think through when to use the DOT and when it is
not necessary, they will be in much better shape later.
```

Problem 5-A.36

In MATLAB, define variables **x** and **y** as follows:

```
>> x = [3; 9; 5; 5; 1]

>> y = [2; 1; 4; 6; 2]
```

Are any elements in **x** such that corresponding elements of **y** are twice as big?

```
any_twiceX_equal_Y = any(x == 0.5*y)
```

Problem 5-A.37

In MATLAB, define variables **x** and **y** as follows:

```
>> x = [3; 9; 5; 5; 1]

>> y = [2; 1; 4; 6; 2]
```

Are all elements in **x** such that corresponding elements of **y** are twice as big?

```
all_twiceX_equal_Y = all(x == 0.5*y)
```

Problem 5-A.38

In MATLAB, define variables **x** and **y** as follows:

```
>> x = [3; 9; 5; 5; 1]

>> y = [2; 1; 4; 6; 2]
```

Find the indices for all elements in **x** such that corresponding elements of **y** are twice as big.

```
ndx = find(x == 0.5*y)

Although this is a simple exercise combining a relational, cell-
by-cell comparison and the find operator, this is an important
exercise to be certain students understand. The concept of
"index" is not straightforward for many freshman learners of
MATLAB.
```

Problem 5-A.39

In MATLAB, define variables **x** and **y** as follows:

```
>> x = [3; 9; 5; 5; 1]

>> y = [2; 1; 4; 6; 2]
```

Find the indices for elements in **x** so the corresponding elements of **y** are twice as big as the element in **x** or the corresponding elements of **y** are less than **x**.

```
ndx = find( (2*x == y) | (y < x) )

The inner parentheses around the relational tests are
technically not needed. However, readability is increased by
including them.
```

Problem 5-A.40

In MATLAB, define variables **x** and **y** as follows:

```
>> x = [3; 9; 5; 5; 1]

>> y = [2; 1; 4; 6; 2]
```

Find the indices for all elements in **x** so neither the corresponding elements of **y** are twice as big as the element in **x** nor the corresponding elements of **y** are less than **x**.

```
ndx = find( ~(  (2*x == y) | (y < x ) )  )

A useful additional exercise is to ask students what the
relationship should be between the answer for this exercise and
the previous exercise. And why. Depending on time and the level
of students, a discussion of deMorgan's rules could also follow
from this exercise.
```

Problem 5-A.41

In MATLAB, define variables **x** and **y** as follows:

```
>> x = [3; 9; 5; 5; 1]

>> y = [2; 1; 4; 6; 2]
```

Find the locations and values of all the elements in **x** that have values between 4 and 10.

```
ndx = find(  (4 < x) & (x < 10)  )
values = x(ndx)

This type of question ("find values of X between …") beginning
students typically answer with a phrase like 4 < x < 10, which
appears to be a more direct translation of the problem spec. To
straighten students out on this point is to emphasize to them
the way MATLAB processes expressions, and in particular the way
that in the absence of an applicable precedence rule MATLAB
processes left to right. The issue also underscores the value of
using additional parentheses to help in "understanding."
```

Problem 5-A.42

In MATLAB, define variables **x** and **y** as follows:

```
>> x = [3; 9; 5; 5; 1]

>> y = [2; 1; 4; 6; 2]
```

Set all the elements in **x** that have values between 4 and 10 to the value of the corresponding value in **y**.

```
ndx = find(  (4 < x) & (x < 10)  )
x(ndx) = y(ndx)
```

Problem 5-A.43

In MATLAB, define variables **x** and **y** as follows:

```
>> x = [3; 9; 5; 5; 1]

>> y = [2; 1; 4; 6; 2]
```

Set all the elements in **y** that have values greater than or equal to 3 to two times their initial value.

```
ndx = find(y >= 3)
y(ndx) = 2 * y(ndx)
```

Problem 5-A.44

In MATLAB, define variables **x** and **y** as follows:

```
>> x = [3; 9; 5; 5; 1]

>> y = [2; 1; 4; 6; 2]
```

For all the elements in **x** such that *either* the corresponding elements of **y** are twice as big as the element in **x** *or* the corresponding elements of **y** are less than **x**, set those elements in **x** to the number 99.

```
ndx = find(  (2*x == y) | (y < x)  )
x(ndx) = 99
```

Problem 5-A.45

The relationship $dist = \frac{1}{2}at^2$ gives the distance traveled (**dist**) by a particle starting from standstill, undergoing uniform acceleration (**a**) after some time (**t**). Create a vector **t** having 100 equally spaced values ranging from 0 to 50 seconds. Assume that **a** is 100 m/s/s. Compute a vector **dist** with the same number of elements as **t**, and for which each element is the distance traveled for the corresponding time of travel in **t**.

```
t = linspace(0,50)          % sec
a = 100                     % m/s/s
dist = 0.5 * a * t.^2       % m

It is unlikely that students will put in units unless doing so
is emphasized, along with the reason that inserting units is
important.
```

Set B: Problem Solving with MATLAB Vectors

Problem 5-B.1

The melting temperature is a good indicator to discover the identity of an unknown substance. Suppose we perform four independent experiments and find melting temperatures of four samples to be [848 854 851 852] ^{o}C. A second round of experiments gives the following results: [850 853 849 851] ^{o}C.

Store the results of the two rounds of experiments in vectors **firstRoundResults** and **secondRoundResults**, respectively.

Now define a new vector **allResults** by concatenating **firstRoundResults** and **secondRoundResults**. Check the lengths of all three vectors and verify the length of **allResults** is the sum of the lengths of **firstRoundResults** and **secondRoundResults**.

```
firstRes = [848 854 851 852];           % deg C
secondRes = [850 853 849 851];          % deg C
allRes = [firstRes,secondRes];          % deg C
lengthsOK = (length(firstRes) + length(secondRes) ) ...
                      == length(allRes)
```

Problem 5-B.2

Part A Create a vector **x** using the square bracket method that has the following elements:

{0 3 6 9 … 81 84 82 80 78 76 … 0}

Part B Create a vector **x1** using the colon operator that has the following elements:

{0 3 6 9 … 81 84}

Part C Create a vector **x2** using the colon operator that has the following elements:

{82 80 78 76 … 0}

Part D Create a vector **x_AnotherWay** that is identical to **x** by concatenating your result from **Parts B** and **C**.

```
x = [0 3 6 9 12 15 18 21 24 27 30 33 36 39 42 45 ...
     48 51 54 57 60 63 66 69 72 75 78 81 84 ...
     82 80 78 76 74 72 70 68 66 64 62 60 58 56 ...
     54 52 50 48 46 44 42 40 38 36 34 32 30 ...
     28 26 24 22 20 18 16 14 12 10 8 6 4 2 0];

x1 = 0:3:84;
x2 = 82:-2:0;
x_AnotherWay = [x1, x2];
```

```
The point of this problem is to sensitize students to the
possibilities for creating specified long vectors by leveraging
the patterns that may be embedded in the specification.
```

Problem 5-B.3

What does MATLAB return when you execute the following line in the Matlab Command Window?

```
>> x = [.7:5 linspace(1,6,6)]
```

Figure this out with paper and pencil first, then run the command in MATLAB to verify your result.

```
The meta-level of this simple exercise is to encourage students
to figure out code lines mentally.
```

Problem 5-B.4

Use two methods to create a vector with 30 elements, where each element is the number 30. Use built-in functions **`zeros`** in one method and **`ones`** in the other method.

```
oneMethod = 30 + zeros(1,30)
secondMethod = 30 * ones(1,30)

The point of this exercise is for students to realize that there
is more than one way to build a vector. A follow on exercise
would be to ask students to test to be sure the two methods
yield the same result. One such test would be … all(oneMethod ==
secondMethod)
```

Problem 5-B.5

Create the following vectors using the colon operator and the built-in function **`linspace`** methods:

Part A A vector whose elements are all odd numbers between 1 and 191

Part B A vector whose elements are 99 evenly spaced numbers between 5 and 148

```
partA_colon = 1:2:191
partA_linspace = linspace(1,191,((191-1)/2)+1)

Students should note: The colon operator is the easier way to
solve Part A.


partB_colon = 4:(148-4)/(99-1):148
partB_linspace = linspace(4,148,99)

Students should note: the linspace operator is the easier way to
solve Part B.

This problem also gives the instructor an opportunity to
emphasize the importance of using containers for accuracy. You
will observe that many freshman learners will be inclined to
calculate the interval size and use the number they see on the
screen in the colon operator. However, this truncated value of
the interval size will result in a vector that ends with
146.5318 instead of 148.
```

Problem 5-B.6

We would like to display a temperature given in ^{o}F on three temperature scales, ^{o}F, ^{o}C, and ^{o}K, in a vector of length 3. The conversion from ^{o}F to ^{o}C and from ^{o}C to ^{o}K are given as follows:

$$tempCelsius = \frac{tempFahrenheit - 32}{1.8}$$

$$tempKelvin = tempCelsius + 273.15$$

Write a MATLAB function that accepts a scalar for temperature in Fahrenheit degrees as input and outputs a vector of degrees in ^{o}F, ^{o}C, and ^{o}K.

```
function outTempsV = convertFahrenheit(degF)
% converts temperature in Fahrenheit to Celsius and Kelvin
% INPUT: temperature in Fahrenheit
% OUTPUT: outTempsV, 3 elecment vector [f, c, k]
%              where f is the input Fahrenheit temperature
%                    c is the equivalent Celsius temperature
%                    k is the equivalent Kelvin temperature
% sample call:
%   allTemps = convertFahrenheit(212)

  degC = (degF - 32) / 1.8;
  degK = degC + 273.15;
  outTempsV = [degF,degC,degK]
```

Problem 5-B.7

Two vectors, **x** and **y**, are given as follows:

```
x = [8 4 6 5 9 3]

y = [1 2 3 4 5 6]
```

Fetch every other element in **x** starting with the second element. Use the fetched elements to replace the last three elements of **y**.

```
y(end-2:end) = x(2:2:end)
```

Problem 5-B.8

Write a MATLAB function that takes a positive, non-zero integer **N** and a vector **V** as inputs and produces a single output vector such that every N^{th} element in **V** is set to the value -99.

For example, if **N=3**, and **V=[1 2 3 4 5 6 7]**, the output returned would be **[1 2 -99 4 5 -99 7]**.

```
function V = changeEachNth(N, V)
% modifies a vector input such that every Nth element
% is set it -99, where
% N is also an input
% INPUTS: N a positive integer
%         V a numerical vector (row or column)
% OUTPUTS: V a modified version of the input where each Nth
%              element is set to -99

  V(N:N:end) = -99;
```

Problem 5-B.9

Find the values of **z** in the following questions with paper and pencil first. When finished, execute the commands to verify your answers.

```
Part A    x = linspace(2,20,10);    z = x([3:3:length(x)])

Part B    x = [7 3 4 2 35 23];      z = [x([4 5]), x([2 1])]

Part C    z = [1:10];               z([6:length(z)]) = z([1:5])

Part D    z = [4 8 2 1];            z([1, length(z)]) = [1 4]
```

```
Again, this exercise is aimed at encouraging students to "think
like a computer" in order to understand code lines.
```

Problem 5-B.10

Use vector **x=[4 8 2 1]**. Write a MATLAB function that takes no inputs and outputs a shuffled version of **x**. The function will shuffle the elements in vector **x** by performing the following steps:

1. Switch the first and last elements of **x**.
2. Switch the first two elements with the last two elements in **x**.

For the value for **x** above, the output of your function should be **[2 4 1 8]**.

```
function V = shuffleInput(V)
% (a) swap first and last elements of V, (b) swap first two
%   elements with last two elements
% INPUT: V a vector of at least 4 elements
% OUTPUT: modified version of V per problem spec

  temp = V(1);
  V(1) = V(end);
  V(end) = temp;

  temp = V(1:2);
  V(1:2) = V(end-1:end);
  V(end-1:end) = temp;
```

Problem 5-B.11

Write a MATLAB function that accepts two integers, **a** and **b**, and outputs a column vector that is composed of every other number between **a** and **b**.

```
function everyOtherV = buildEveryOtherV(a, b)
% creates a vector composed of every other integer between a,b
% INPUTPS: a - integer
%          b - integer
% OUTPUT: everyOtherV - column vector of integers starting with
%                       a, and including every other integer until
%                          b is reached or exceeded

  everyOtherV = (a:2:b)';
```

Problem 5-B.12

Write a MATLAB function that accepts two scalars, **a** and **b**, and an integer **c**, and outputs a column vector that is composed of **c** evenly spaced numbers between **a** and **b**.

```
function evenlySpacedV = buildVector(a, b, c)
% creates a column vector composed c evenly spaced numbers
% between a and b, with simple use of linspace function
% INPUTPS: a - scalar
%          b - scalar
%          c - a positive integer
% OUTPUT: evenlySpacedV - column vector of integers starting
%                         with a, ending with b, and including a
%                          total of c points

  evenlySpacedV = (linspace(a,b,c))'
```

Problem 5-B.13

Create the following vectors using first **linspace** or the colon operator, then the transpose operator.

Part A A column vector of all even numbers between (and including) 0 and 10

Part B A column vector of five evenly spaced numbers between (and including) 2 and 10

Part C

$$\begin{bmatrix} 4 \\ 7 \\ 10 \\ 13 \\ 16 \end{bmatrix}$$

```
partA = 0:2:10
partA = partA'
```

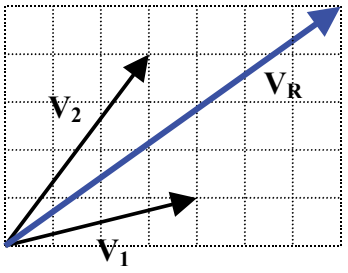
```
partB = linspace(2,10,5)
partB = partB'

Although Part A and Part B are simple, the point is to ask the
student to actively choose between two tools: colon operator and
linspace.


partC = 4:3:16
partC = partC'
```

Problem 5-B.14

A two-dimensional physics vector can be represented in MATLAB by a collection of its *x* and *y* components. For example, V_1 and V_2 in the figure below could be represented as (4,1) and (3,4), respectively. The summation of two vectors is carried out by adding the components of vectors independently. In the example below, V_3, which is the sum of V_1 and V_2, is calculated as (4+3, 1+4). If vectors represent forces, a summation of vectors is said to be the Resultant Force.



Write a MATLAB function that accepts two arbitrary forces in a two-dimensional space and returns the resultant force.

```
function resultantF = sumForces(force_1, force_2)
% compute vector sum of two vectors
% INPUT: force_1, a (MATLAB) vector with N elements representing
%                 an N-dimensional physical vector
%        force_2, a (MATLAB) vector with N elements representing
%                 an N-dimensional physical vector
```

```
%        (N is the same for force_1 and force_2)
% OUTPUT: resultantF, a N element (MATLAB) vector representing
%                     the vector sum of the two inputs

  resultantF = force_1 + force_2;
```

Problem 5-B.15

When an object of mass *m* attached to a spring from above is free, two relations for the two forces are acting on the spring: the gravitational force giving rise to the weight of the mass (pulling the mass downward) and the force exerted by the spring (pulling the mass upward). At rest, the two forces must be equal to each other in magnitude but opposite in sign.

$$F_{gravitational} = -mg$$
$$F_{spring} = -k_{spring}x$$
$$\therefore mg = -k_{spring}x$$

The first relation gives the force due to gravity (the weight). The second relation gives the restoring force due to the spring, where k_{spring} is the spring constant, i.e., a constant of proportionality that is descriptive for one given spring and where *x* is the length that the weight stretches the spring. The third relation puts it all together since the spring force and the gravitational force are equal in magnitude but opposite in direction.

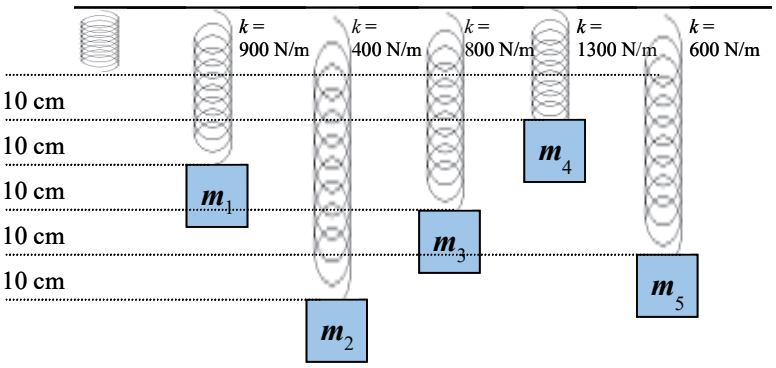
We will take the units for the relation $mg = -k_{spring}\,x$ to be the following: unit for mass (**m**) is kilograms, unit for displacement (**x**) is meters, unit for the spring constant (**k**) is Newtons/meter. The value of **g**, the acceleration due to gravity, is 9.81 m/s/s.

The potential energy stored in the string due to this elongation is known to be (in Newton-meters):

$$potentialEnergy = \frac{kx^2}{2}$$

Write a MATLAB function that accepts a vector of extension distances (**x**) and a vector of corresponding spring constants (**k**). Your function should return a vector of object masses and a vector of potential energies stored in each spring.

Test your function with data for the five spring/mass examples in the figure below. Your function should work for any number of spring-mass systems.




```
function [massV, potentialEnergyV] = springsCompute(extDistV,
sprConstV)
% compute suspended mass and potential energy values for
% a set of springs given corresponding static extension
% distances and spring constants
% INPUTS: extDistV, a vector of extension distances (m)
%         sprConstV, a vector of spring constants (N/m)
% OUTPUTS: massV, a vector of mass values for each spring (kg)
%         potentialEnergyV, a vector of potential energy values
%                           stored in each spring at the static
%                            extension position (N-m)
  g = -9.81;     % accel due to gravity (m/s/s),
                 % minus sign expressed direction is "down"

  massV = -(sprConstV .* extDistV) / g;                  % (kg)
  potentialEnergyV = 0.5 * (sprConstV .* extDistV.^2);   % (N-m)

% sample call
%   D = [20 50 30 10 40]; S = [9 4 8 3 6] * 10^2;
%   [masses, potentialEnergyStored] = springsCompute(D, S)
```

Problem 5-B.16

Suppose you have a catalogue of automobiles such as in the following table:

ID	Color (0 - black, 1 - blue, 2 - red)	Price($)
1002	0	24,000
2176	1	18,000
3201	2	36,000
4204	1	26,000
5512	2	35,000
6309	0	29,000
8841	1	31,000
9004	0	38,000

Write a MATLAB function that will input a vector for **ID**, a vector for **Color**, a vector for **Price**, and return the ID numbers of black cars that are priced under $30,000.

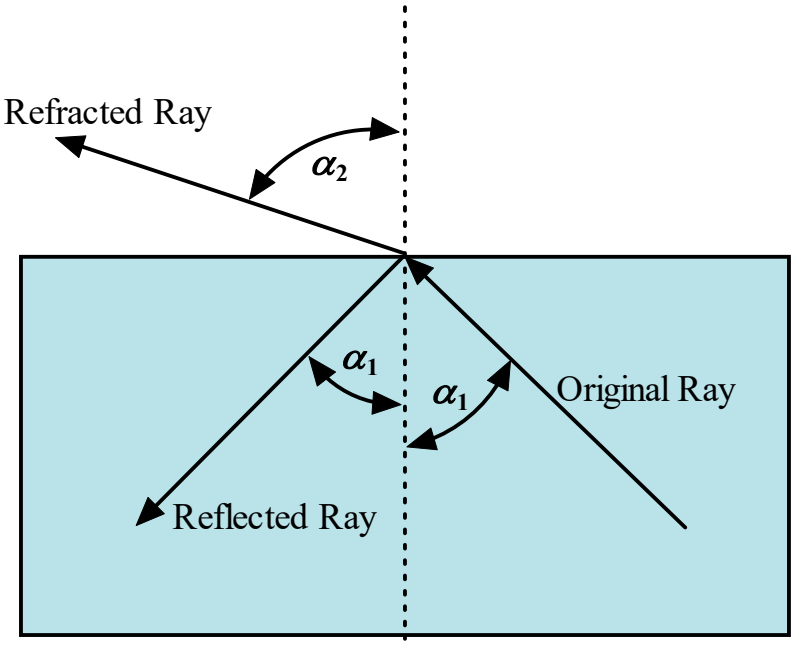
```
function selectIDs = chooseCars(idV, colorV, priceV)
% chooses cars from a DB such that the color of the car is black
% and the price of the car is under $30k
% INPUTS: idV, a vector of N elements containing ID numbers
%         colorV, a corresponding vector of N elements containing
%                 the color of the car, numerically coded
%                 0=black 1=blue 2=red
%         priceV, a corresponding vector of N elements containing
%                 the price of the car
% sample call
% id = [1002 2176 3201 4204 5512 6309 8841 9004];
% color = [0 1 2 1 2 0 1 0];
% price = 10^3 * [24 18 36 26 35 29 31 38];
% idsSelected = chooseCars(id, color, price)

  ndx = find(  (colorV == 0) & (priceV < 30000)  );
  selectIDs = idV(ndx);
```

Problem 5-B.17

When a light source is submerged in a pond and pointed toward the surface of the pond, either reflection or refraction occurs when the light beam crosses from under the water to above the water. Whether refraction or reflection

takes place depends on the incidence angle α_1 of the light beam. The physical situation is shown in the following diagram:



When α_1 is smaller than *the critical angle*, the light beam will undergo *both* refraction and reflection. That is, some of the light will be passed out of the water and into the air, but its direction will change (refraction), and some of the light will be bounced from the surface back into the water (reflection).

Refraction is described according to Snell's law in the following relation:

$$n_1 \sin(\alpha_1) = n_2 \sin(\alpha_2)$$

where n_1 is the index of refraction of water (1.33), and n_2 is the index of refraction of air (1.00029), and the angles are shown in the diagram above.

If angle α_1 is larger than the critical angle, only internal reflection occurs, and the angle of reflection will be the same as the incidence angle as shown in the diagram above.

The critical angle for the water-air boundary is 49°. Write a Matlab function that accepts a vector of incidence angles (α_1, in degrees) and returns a vector of reflection or refraction angles (in degrees) for each incidence angle. Assume that for angles smaller than the critical angle only refraction occurs. Try your function with incidence angles from 0 to 90° with increments of 5°.

Given the vector of angles, find which angles are greater than critical angle first. Calculate refraction angles and replace some refraction angles with reflection angles.

```
This problem has a purposely embedded flaw that some students
may detect, or that the instructor may want to start a "hunt"
for. The critical angle can be determined directly from Snell's
Law and the properties of the materials - in particular the
index of refraction for the materials on either side of the
interface. The value given for the critical angle in the problem
spec is an approximation. A more accurate value for the material
properties given is 48.772 degrees. A student could find this
"difficulty" with the problem spec by careful debugging. For
example, with a test value of 48.9 degrees, the value for the
bounce angle will be a complex number IF one assumes the criti-
cal angle is 49 degrees.

function bounceAngleV = bounceLight(incidenceAngleV)
% compute how light bounces across a water/air boundary
%     (uses Snell's Law)
% INPUT: incidenceAngleV, a vector of incidence angles in deg
% OUTPUT: bounceAngleV, a corresponding vector of angles, each
%                     of which a light ray travels along after
%                      going across the interface

 % problem constants
   criticalAngle = 49 * pi/180;                  % radians
   indexWater = 1.33;
   indexAir = 1.00029;

 % convert incidenceAngleV to radians
   incidenceAngleV = incidenceAngleV * pi/180;   % radians

 % find locations where angle > critical angle
   ndx_overCritical = find(incidenceAngleV > criticalAngle);

 % convert all incidence angles assuming refraction
   bounceAngleV = asin( (indexWater/indexAir) * ...
                                        sin(incidenceAngleV) );

 % replace values for cases of reflection only
   bounceAngleV(ndx_overCritical) = ...
                           incidenceAngleV(ndx_overCritical);

 % convert answer back to degrees
   bounceAngleV = bounceAngleV * (180/pi);             % degrees

% sample call
```

```
bouncingAngles = bounceLight([0, 30, 45, 55, 75, 90])
```

Problem 5-B.18

Write a function that will take vectors **A** and **B** as input, compare and update the two vectors according to the following rules, and output the updated vectors **A** and **B**.

1) If an element in **A** is smaller than the corresponding element in **B**, set the element in **A** to zero.

2) If an element in **A** is greater than the corresponding element in **B**, set the element in **B** to zero.

3) If an element in **A** is equal to the corresponding element in **B**, set the element in **A** to zero and set the element in **B** to zero.

```
A common problem that students have in trying to implement this
seemingly straightforward problem lies in *not* making a copy of
the two input vectors at the outset.

function [updatedA, updatedB] = change_A_B(A,B)
% modified two input vectors by applying modification rules
% given in problem specification
% INPUT: A, vector with N elements
%        B, vector with N elements
% OUTPUT: updatedA, vector with N elements, modified A vector
%         updatedB, vector with N elements, modified B vector

  % create a copy of A and B
    updatedA = A; updatedB = B;

  % apply rule A
    ndx = find(A < B); updatedA(ndx) = 0;

  % apply rule B
    ndx = find(A > B); updatedB(ndx) = 0;

  % apply rule C
    ndx = find(A == B); updatedA(ndx) = 0; updatedB(ndx) = 0;


% Sample Call
```

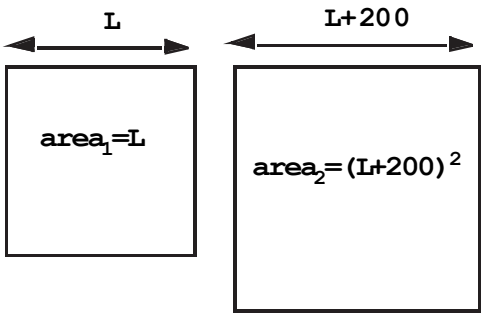
```
% [changedA, changedB] = change_A_B([1 5 10], [2 3 10])
```

Problem 5-B.19

A farmer has a square shaped field with side length **L** ft. He then buys another field which is square, but its side length is **L+200** ft. He knows that the total area of the two fields is between 110 acres and 120 acres. (1 sq. mi = 640 acres; 5280 ft = 1 mi)

Write a MATLAB function that accepts a two-element vector for the range of total area (in acres) as input (**[110, 120]** in this case), computes a vector of possible values of **L** (in feet), and returns a vector of possible values of **L**. Consider integer values of **L** between 100 ft and 5,000 ft in increments of 100 ft.

A drawing of the situation can help students to visualize the problem.



The total area of the two fields is known to satisfy the following:

$$\text{TotalArea} = L^2 + (L + 200)^2$$

$$\text{lowerLimit} \le \text{TotalArea} \le \text{upperLimit}$$

where in the example lowerLimit=110 acres and upperLimit=120 acres.

One solution is the following function.

```
function satisfyingL = findFiedLength(lowerLimit, upperLimit)
% compute satisfying values of L producing a total area between
% lowerLimit and upperLimit, with the constraint that the total
% area is composed of contributions from two square fields:
%   one of the fields has sides of length L feet
%   the second has sides of length (L + 200) feet
% INPUTS: lowerLimit, scalar lower limit of area in Acres
%         upperLimit, scalar upper limit of area in Acres
% OUTPUT: satisfyingL, vector of L vals satisfying constraints
%                      Values in units of feet
%                    Acceptable values selected from a vector of
%                    possibilities running from 100:100:5000 ft
%                      (The second argument here sets the
%                      "resolution" of the output vector.)

  % convert the limits to sq ft
  %   {A Acres / [640 A/sq mi]}  => SqMi sq mi
  %   SqMi sq mi / (1 mi/5280 ft)^2 => SqFt sq ft
  %   thus (A Acres) * (1/640) * 5280^2 => SqFt sq ft
    limits = [lowerLimit, upperLimit];    % Acres
    limits = limits * 5280^2 / 640;       % sq ft

  % set up a test vector of L values, compute the total area of
  % both fields with each value of L,  then find the areas that
  % satisfy the problem constraints, and finally identify the
  % corresponding values, L
    L = 100:100:5000;                                  % ft
    resultingArea = L.^2 +  (L + 200).^2;              % sq ft
    ndx=find((limits(1) <= resultingArea) & ...
                (resultingArea <= limits(2)));
    satisfyingL = L(ndx);


% sample call
% satisfyingLengths = findFiedLength(100, 120)
```

SOLUTIONS FOR CHAPTER 6
2 D PLOTTING AND
USING MATLAB HELP

This chapter is meant to be a breather for students from facets of MATLAB that are very new to them. In particular, its meant as a targeted way to bring together what has been learned about the MATLAB environment, expressions, and vector operations, and then to incrementally extend so as to enable 2-D plotting.

You will find in the notes and solutions below that some problems, particularly in Set B, require students to use MATLAB HELP to look up "new" (to them) features of MATLAB. Although **sprintf** is touched on in the text, students will need to refresh their memories on how to use it. In addition, in some of the problems of Set B, **texlabel** can be used to advantage to annotate graphs. **texlabel** is not described in the text; students will need to use MATLAB HELP to see how to use it. Students should be encouraged to do "professional plots" for the problems in this chapter. A reasonable way to convey what that means is to tell students "suppose you have a job and you are turning in the plot to your supervisor."

You will also find that some problems, again largely in Set B, are purposefully open-ended or not totally well defined. Both of these problem characteristics are aimed at helping students grow into independent problem solvers.

Set A: Nuts and Bolts Problems for 2-D Plotting

Problem 6-A.1

Use **`ezplot`** to plot the following function in the closed interval $[-2\pi, 2\pi]$:

$$f(x) = \sin(x^2)$$

Write a function that creates this plot. Name the function **Prob6_A_1**.

```
function Prob6_A_1
% plot sin(x^2) in [-2pi, 2pi] using explot
% No input or output variables.

  ezplot('sin(x^2)', [-2*pi, 2*pi])
  grid on

    % notice that labeling is done automatically by ezplot
```

Problem 6-A.2

Use **ezplot** to plot the following function in the closed interval $[0, A\pi]$:

$$f(x) = \sin(x)e^{-0.1x}$$

A is the parameter that you set.

Write a function that creates this plot. Your function will take one input which is a value for **A**. Name the function **Prob6_A_2**.

Call your function with **A=2**. Write a short paragraph describing the plot you produced.

Call your function with **A=8**. Write a short paragraph describing the second plot you produced.

What lesson is to be learned from this problem?

The point of the comparison over the two different ranges of x is that for the interval [0, 2*pi] the damping of the exponential term is not apparent. But over the larger interval [0, 8*pi] the damping is clear. The general point is that care must be taken when characterizing a plot.

Some students may notice, or the instructor may nudge them towards noticing, that the f(x) in this problem is really two term: an oscillation and a damping term.

A function meeting this problem spec is below.

```
function Prob6_A_2(A)
% use ezplot to graph sin(x)*exp(-0.1x) in [0, A*pi]
% INPUT: A, a scalar number
% OUTPUT: none

  ezplot('sin(x)*exp(-0.1*x)', [0, A*pi])
```

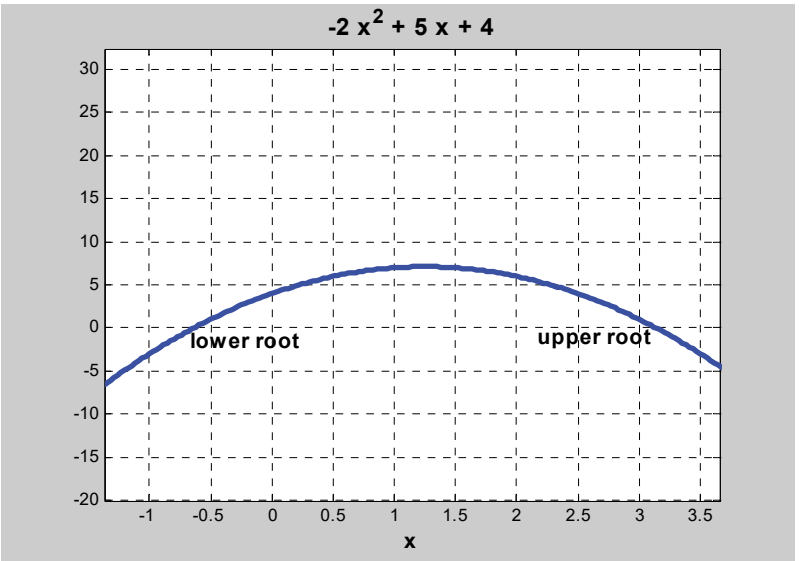
Problem 6-A.3

Use **`ezplot`** to plot the following polynomial and graphically determine its two roots:

$$f(x) = -2x^2 + 5x + 4$$

Turn the grid on and enlarge the graph around the roots to increase accuracy. Search in MATLAB Help to learn how to zoom a 2-D graph.

Label the roots in the plot using **`text`** or **`gtext`**. Save the figure and call it **`Prob6_A_3.fig`**.



```
A reasonable figure for this problem is shown above. The code to
produce this graph is as follows.

ezplot('-2*x^2 + 5*x + 4')
grid on

% at this point the student would "zoom" the chart to better
% show the roots - i.e. the zero-crossing

gtext('lower root')
gtext('upper root')


The figure above is enhanced with the graph editor to make the
functional line thicker and make the text be larger and bold.
```

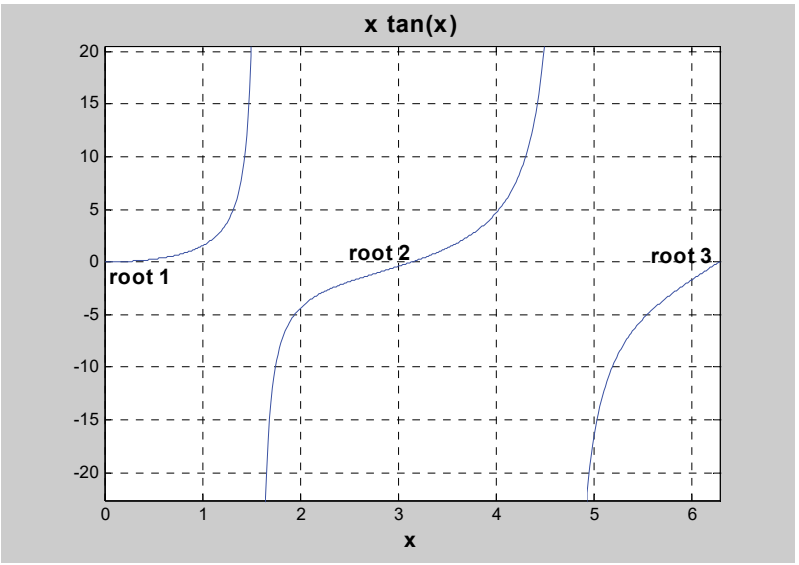
Problem 6-A.4

Use **`ezplot`** to determine the roots graphically of the following equation in the interval $[0, 2\pi]$:

$$x\tan(x) = 9$$

Use the grid and zoom facilities of MATLAB for more accurate answers.

Label the roots in the plot using **`text`** or **`gtext`**. Save the figure and call it **`Prob6_A_4.fig`**.



Problem 6-A.5

Use **`ezplot`** to plot the following implicit function in the interval $[-6, 6]$:

$$f(x, y) = 0 = (-2x^2y^2) + 5x^2y - 4xy^2 + 16xy + 4x + 8y + 16$$

Set the limit for the y-axis to $[-9, 9]$.

Create a function that makes the plot. Name the function **`Prob6_A_5`**.

Students typically have some difficulty in understanding what an "implicit plot" is, and how to read it. It would be useful in association with this problem if some directed exercise were initiated along the lines of … What is the value of Y to make the value of F be 0 if the value of X is 2? In the F for this problem, this type of additional exercise not only helps students to understand implicit graphs, but also introduces/ strengthens understanding of multi-valued functions.

```
One solution function is below.

function Prob6_A_5
% use ezplot to create an implicit plot of
% f(x,y)=0 = (-2x^2y^2) + 5x^2y - 4xy2 + 16xy + 4x + 8y + 16
% Show the y-axis from [-9, 9]
% No input or output variables

F='(-2*x^2*y^2) + 5*x*2*y - 4*x*y^2 + 16*x*y + 4*x + 8*y + 16';
ezplot(F, [-2*pi, 2*pi , -9, 9])
          % the problem spec set the Y limits, but not the
          % X limits. X limits were set to reflect the standard
          % MATLAB default
grid on
title(F)  % explicitly doing a title is a good idea in this
          % example because the full function will be
          % truncated otherwise
```

Problem 6-A.6

Use **ezplot** to plot the following implicit expression of two variables:

$$x^2 + y^2 = 1$$

Turn the grid on. Adjust the plot range so that your plot is easily readable. Add a title to your plot and label the axes.

Name the function that creates the plot **Prob6_A_6**.

```
This problem introduces one more step in using ezplot for
implicit plots of functions of two variables: the functional
form to be given to ezplot must be equal to zero. Asking addi-
tional student work on this problem would help make the point;
by changing the problem specification to a different constant
students will get a better feel for the situation.

function Prob6_A_6
% uses ezplot to plot x^2 + y^2 = 1
% No input or output variables

  F = 'x^2 + y^2 - 1';
  ezplot(F)
  grid

  title('x^2 + y^2 = 1')
```

```
    % simple change in title from that created by ezplot
    % not essential

  % axis already labeled appropriately by ezplot

  % zoom should be done by student after function runs
```

Problem 6-A.7

Graphically determine the value of $\cos(t)$ when $\cosh(t) = 3$.

Write a function, **Prob6_A_7**, that takes two inputs: **Lower** and **Upper**. These are the bounds on **t** used in the making the plot. Experiment with calls to your function until you can easily see the graphical solution to this problem. In your function, use **ginput** to capture the value of the graphical solution, and return the answer as an output variable from your function.

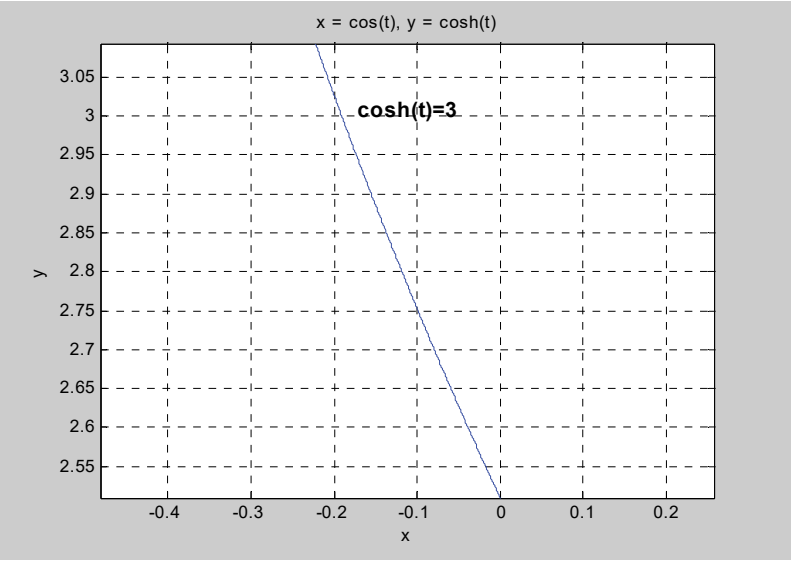
```
This exercise extends the student to the third identified use
for ezplot: plotting two functions of the same independent vari-
able. Again, it is important that students understand the inter-
pretation of the graph once its created.

A function that solves this problem is below, followed by a
sample version of the generated plot.

function Prob6_A_7(lower, upper)
% uses ezplot to create a plot to find the value of cos(t)
% when cosh(t)=3.
% INPUTS: lower, a scalar, the lower limit of the range of t
%         upper, a scalar, upper limit on t
% OUTPUTS: none

  F1 = 'cos(t)';    F2 = 'cosh(t)';
  ezplot(F1, F2, [lower, upper])
  grid on
  gtext('cosh(t)=3')

  % by experimentation, reasonable values for the lower and
  % upper are lower=pi/2   upper=pi/1.75
```



Problem 6-A.8

Graphically determine the value of *f(t)* given by:

$$f(t) = \sin(t)e^{-0.1t}$$

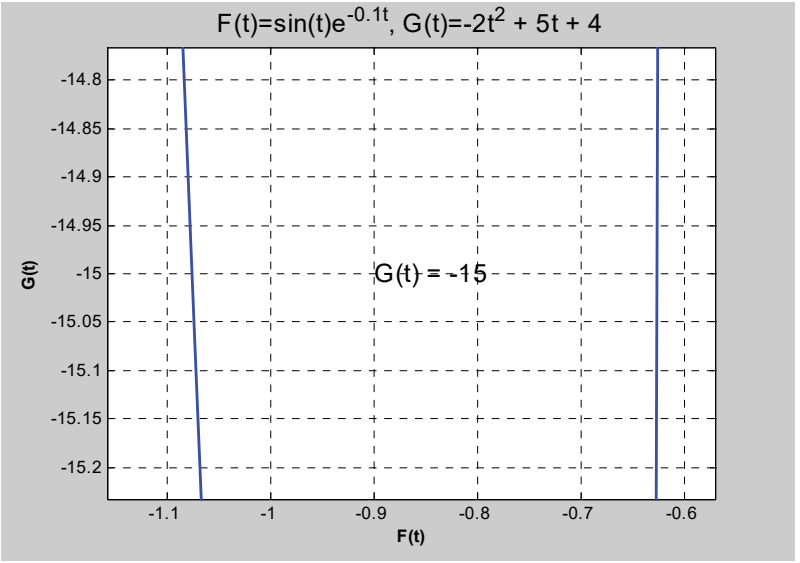
when

$$g(t) = -2t^2 + 5t + 4 = -15$$

Use the same general method you used for Problem 6-A.7. Name your function for this problem **`Prob6_A_8`**. To annotate your plot look up in MATLAB HELP and use the built-in function **texlabel** to create text in your plot.

Be careful with this problem. You should return multiple answers, not a single answer.

A reasonable chart can be obtained by first finding starting limits for the independent variable of the problem, followed by using the ZOOM capability of MATLAB. A completed chart is below followed by a function meeting the problem specification.




```
function Prob6_A_8(lower, upper)
% uses ezplot to create a plot to find the value of
%    f(t) = sin(t) exp(-0.1t) when
%      g(t) = -2t^2 + 5t + 4 = -15
% INPUTS: lower, a scalar, the lower limit of the range of t
%         upper, a scalar, upper limit on t
% OUTPUTS: none
% sample call
%    Prob6_A_8(-2*pi, 2*pi)

  F = 'sin(t) * exp(-0.1*t)';
  G = '-2*t^2 + 5*t + 4';
  ezplot(F, G, [lower, upper])
  grid on
  text(-0.9,-15, 'G(t) = -15', 'FontSize', 14)

  xlabel('F(t)', 'FontWeight', 'bold')
  ylabel('G(t)', 'FontWeight', 'bold')

  titleStr = 'F(t)=sin(t)e^(-0.1t), G(t)=-2t^2 + 5t + 4';
  title(texlabel(titleStr), 'FontSize', 14)

  % values for upper and lower of [-2*pi, 2*pi] follow by
  % approriate use of ZOOM will enable the two answers to be
```

```
% found.
% values are (app) -1.8 and -6.1
```

Problem 6-A.9

Use **ezplot3** to create a 3-D curve for the following relationship in the interval $[-9\pi, 9\pi]$:

$$x = e^{0.1t}\cos(t)$$
$$y = e^{0.1t}\sin(t)$$
$$z = t$$

Write a MATLAB function that creates the specified function. Name your function **Prob6_A_9**. In your function, call **rotate3d** so you can explore the plot from different viewing angles.

(You will need to look up **ezplot3** and **rotate3d** in MATLAB Help.)

```
function Prob6_A_9
% plots an expanding helix in 3-D that is given by the
% relations below. Uses ezplot

  F1 = 'exp(0.1*t) * cos(t)';
  F2 = 'exp(0.1*t) * sin(t)';
  F3 = 't';

  ezplot3(F1, F2, F3, [-9*pi, 9*pi], 'animate')
       % the animate parm is an add on that students will
       % enjoy

  rotate3D
```

Problem 6-A.10

The data table below shows power dissipation for varying magnitudes of electric current in a circuit.

Current (amperes)	0	5	10	15	20	25
Power Dissipation (watts)	0	175	700	1575	2800	4375

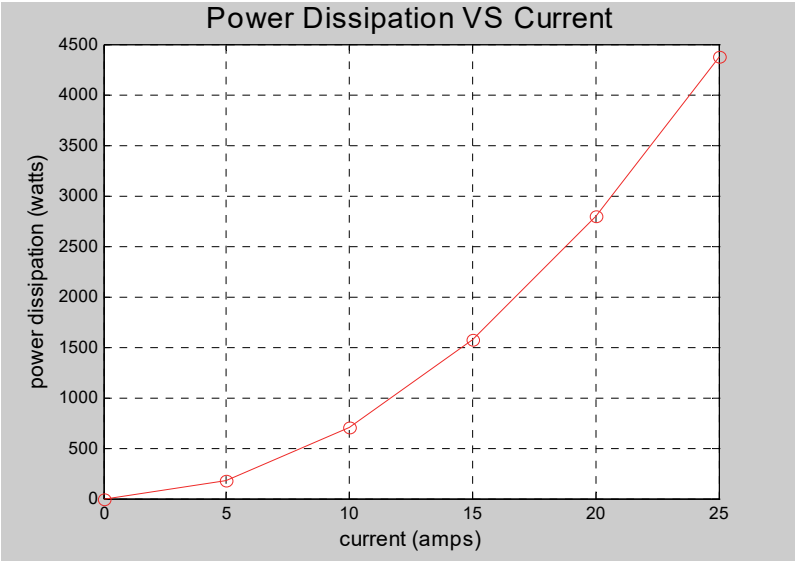
Plot the data. Mark the data points with circles and connect them with a red solid line. Don't forget to label the *x* and *y* axes and to create a title for your graph.

Save the figure you generate as **`Prob6_A_10.fig`.**

The code to generate the required plot is:

```
cur = [0 5 10 15 20 25];                % amps
powerDis = [0 175 700 1575 2800 4375]; % watts
plot(cur, powerDis, '-ro')
title('Power Dissipation VS Current', 'FontSize', 16)
xlabel('current (amps)', 'FontSize', 12)
ylabel('power dissipation (watts)', 'FontSize', 12)
grid on
```

The chart produced by the above code is shown below.



Problem 6-A.11

Steel is often tempered before it can be used for construction. Tempering relieves the internal stresses, reducing the brittleness while softening the steel. Tempering is carried out by heating the steel to a specific temperature, holding the temperature for a required length of time, and cooling the steel. The following table gives the temperature of a steel plate as measured during a tempering process.

Hours	0.0	0.5	1.0	1.5	2.0	2.5	3.0	3.5	4.0	4.5	5.0	5.5	6.0	6.5
^{o}F	22	221	356	414	412	416	415	408	291	126	87	42	35	22

Plot the data by marking the data points with triangles and connecting them with magenta solid lines. Label the *x* and *y* axes and title your graph. Set the font size for axis labels and the title to 14 and 16, respectively.

Save the figure you generate as **`Prob6_A_11.fig`**.

There are several points to note about this problem.

First, the type of triangle is not specified. A student will find (under Line Spec in HELP for example) that there are four types of triangles for data markers. The code below will specify the "upward pointing" triangle - specified with a caret symbol.

Second, this problem is technically no more difficult than Problem 6_A_10, just above. That is, the code is almost exactly a carbon copy. Yet because there is more verbiage here, more explanation of the physical system being plotted, freshman students typically find this problem harder than the one before. Some work with students so that they see the need to cut through the words and get to what the problem spec really is will help.

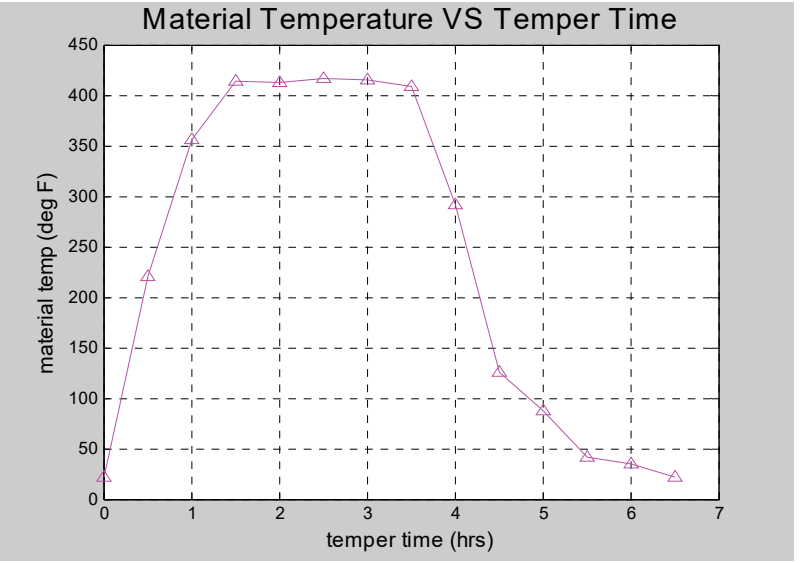
Third, it would be a useful exercise to ask students to "explain in words" the chart that results. That is, explain the chart in terms of the physical process that is in play.

The code to generate the required plot is:

```
hrs = [0.0 0.5 1.0 1.5 2.0 2.5 3.0 3.5 4.0 4.5 5.0 5.5 6.0 6.5];
                    % hrs
defF = [22 221 356 414 412 416 415 408 291 126 87 42 35 22];
                    % degrees F
```

```
plot(cur, powerDis, '-ro')
title('Power Dissipation VS Current', 'FontSize', 16)
xlabel('current (amps)', 'FontSize', 12)
ylabel('power dissipation (watts)', 'FontSize', 12)
grid on
```

The chart produced by the above code is shown below.



Problem 6-A.12

Hybrid vehicles combine more than one source of power to achieve higher power (e.g., locomotives) or longer duration of operation (e.g., submarines). Regenerative braking is one engineering method to enable a gasoline engine/ electric battery hybrid vehicle to achieve higher traction at lower speeds and higher range for a given storage battery size. When a car is moving, it has an amount of energy dependent on how fast it is moving: the car's kinetic energy. When the car is stopped by the driver applying the brakes, the kinetic energy must be dissipated. What happens is that the energy of motion is converted into heat energy by the brakes, and the brakes become hot. In advanced automotive design, regenerative braking is used to store some of the energy of motion as the brakes are applied; not all the kinetic energy is wasted in heating the brakes, rather some is stored in an electrical battery.

An automotive company is in the process of designing a new gasoline engine/ electric battery hybrid vehicle. From computer simulation using batteries with different storage capacities, data in the following table are found.

Bat. Capacity (ampere-hours)	10	20	30	40	50	60	70	80	90	100	110	120
Range (mi)	330	365	379	384	389	394	398	399	400	401	399	395

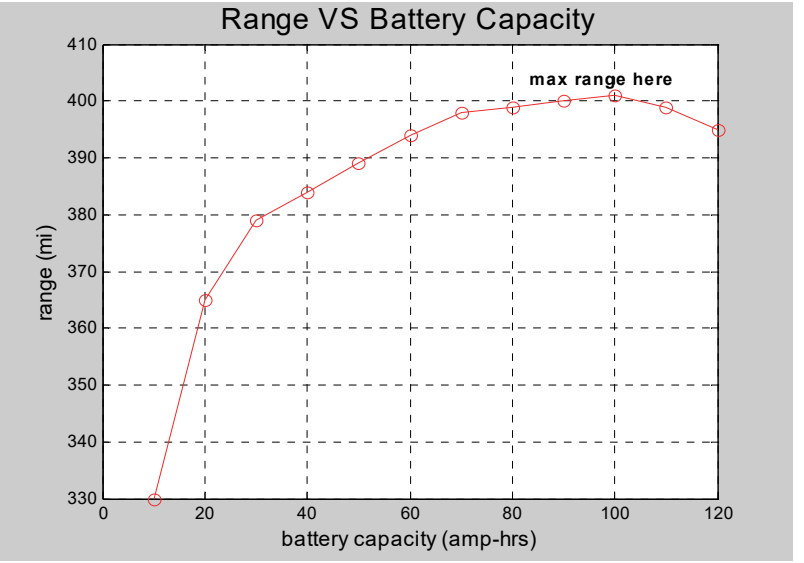
Plot vehicle range versus battery capacity. Determine an optimum value for battery capacity visually. Save your plot as **`Prob6_A_12.fig`**. On your graph, mark the optimum battery capacity using the **`gtext`** function.

What factors can you suggest that would make an optimum value, i.e., why isn't the relationship between battery capacity and range monotonically increasing? Put your answer into one or two concise paragraphs.

The code to generate the required plot is:

```
batCap = [10,20,30,40,50,60,70,80,90,100,110,120];    % amp-hrs
range = [330,365,379,384,389,394,398,399,400,401,399,395]; % mi
plot(batCap, range, '-ro')
title('Range VS Battery Capacity', 'FontSize', 16)
xlabel('battery capacity (amp-hrs)', 'FontSize', 12)
ylabel('range (mi)', 'FontSize', 12)
grid on
gtext('max range here', 'FontWeight', 'bold')
```

The chart produced by the above code is shown below.




```
The "word" part of this question can be as open ended as the
instructor wants it to be. The short answer can be along the
lines that increasing battery capacity implies increasing weight
for the car, and that for this data set, increasing battery
capacity beyond that required to produce a 100 mile range starts
to require heavier and heavier batteries, and range begins to
decline.
```

Problem 6-A.13

The volume of a cone can be calculated using the following equation:

$$coneVolume = (\pi r^2)\left(\frac{coneHeight}{3}\right)$$

r is the radius of the base circle. Write a MATLAB function that inputs a vector of base radii and a scalar cone height and plots the cone volume as a function of base radius. Be sure you label the chart appropriately. Name your function **Prob6_A_13**.

Try your function with 0.1, 0.2,..., 10 feet for the radii. Make the cone height five feet.

Hint: If you are getting the error message 'Matrix dimensions must agree', go back to Chapter 5 and refresh your memory of cell-by-cell operations.

A function that satisfies this problem spec is as follows:

```
function Prob6_A_13(radV, height)
% plots the growth in a cone's volume with increasing base
% radius, for a given cone height
% INPUT: radV, vector of values for the cone radius
%        height, scalar value for cone height
% OUTPUT: none
% generates a plot of VOL vs RAD (for fixed height)

  volV = (pi*radV .^ 2) * (height/3);
  plot(radV, volV, '-k')
  title('Cone Vol VS Base Radius (fixed height)','FontSize', 16)
  xlabel('radius (ft)', 'FontSize', 12)
  ylabel('volume (cu ft)', 'FontSize', 12)
  legend(['height is ', num2str(height)])
  grid on

% sample call
%  radii = 0.1:0.1:10;  % ft
%  oneHeight = 5;       % ft
%  Prob6_A_13(radii, oneHeight)
```

Problem 6-A.14

Create a function, **`Prob6_A_14`**, that plots the following mathematical function in the range $a < x < b$ using **`plot`**:

$$f(x) = \frac{1}{xe^{-0.04/x}}$$

Test your function for the case **`a=1, b=2`**.

This problem is a standard example of using MATLAB to make a functional plot. A function to accomplish the task is below.

A variation on the problem can be started by asking student to change a to 0.2, and replot. Then change to 0.1 and replot. The

```
graphs of course will get more and more "steep" on the left hand
side of the plot. If the student now tries to plot with a set to
0.01, the plot is relatively useless. The question that may be
posed to students is – how can we make this plot and get a
useful view of the function? One answer is along the lines of
changing the variable on Y-axis to be the log of the original
value. The MATLAB function semilogy can then be introduced to
students.


function Prob6_A_14(a, b)
% plots specified math function in range (a, b)
% INPUT: a, scalar, lower limit for range of function plot
%        b, scalar, upper limit for range of function plot
% OUTPUT: none
% generates a plot

  x = linspace(a,b);
  F = 1 ./ (x .* exp(-0.04./x));
  plot(x, F, '-k')
  title('F(x) = 1 / (x * exp(-0.04/x))', 'FontSize', 16)
  xlabel('x', 'FontSize', 12)
  ylabel('F(x)', 'FontSize', 12)
  grid on

% sample call
%  lower = 1;
%  upper = 2;
%  Prob6_A_14(lower, upper)
```

Problem 6-A.15

Mathematical function **S(x)** is defined as follows:

$$S(x) = 5\left(\frac{x}{0.2}\right)^{0.7}\left(\frac{1-x}{0.2}\right)^{-0.7}$$

Create a function, **Prob6_A_15** in the range $[a, b]$ using **plot**.

```
This problem is another example of standard numerical graphing
of a function.

There are three points to note however. First, be sure students
pick up the point exercised in the use of the title build-in
function in the code below. The issue is how to get a
superscript into a title. The answer is that a caret must
proceed EVERY character that one wants to appear as a
```

superscript. This appears in MATLAB HELP and having students find this for themselves is a good exercise.

Second, if the following function is called for example with Prob6_A_15(1, 2) the following warning will appear:

```
Warning: Imaginary parts of complex X and/or Y arguments
ignored.
> In Prob6_A_15 at 12
```

The reason of course is that the term including (1-x) will be negative in the range (1,2). The range for x that will produce non-complex numbers is (0,1). This again presents a possible learning activity for students. Even though complex numbers are not a topic in the text, understanding mathematically the rudiments of what is going on in this problem is of interest.

Third, in the code below, note the manner in which the expression for S(x) is decomposed. This goes back to good programming practice to enable efficient debugging. Students should be reminded of this.

```
function Prob6_A_15(a, b)
% plots specified math function in range (a, b)
% INPUT: a, scalar, lower limit for range of function plot
%        b, scalar, upper limit for range of function plot
% OUTPUT: none
% generates a plot

  x = linspace(a,b);
  S1 = (x ./ 0.2).^0.7;
  S2 = ((1 - x) ./ 0.2).^-0.7;
  S = 5 * S1 .* S2;
  plot(x, S, '-k')
  title('S(x) = 5 * (x/0.2)^0^.^7 * ((1-x) / 0.2)^-^0^.^7', ...
        'FontSize', 16)
  xlabel('x', 'FontSize', 12)
  ylabel('F(x)', 'FontSize', 12)
  grid on

% sample call
%  lower = 0;
%  upper = 1;
%  Prob6_A_15(lower, upper)
```

Problem 6-A.16

A function **G(x,y)** of two independent variables is defined as follows:

$$G(x, y) = \frac{x(7.14 - 0.26y)^{2.1}}{y^{y}}$$

Write a function, **`Prob6_A_16`**, that plots **`G(x,y)`**.

Since we have not studied plotting functions of two variables, you will have to figure out a path to show a number of plots that can give the sense of **`G`**.

Refer to Figure 6-7. In it, you plotted a family of curves in one window. Use the methods you learned to produce Figure 6-7 to produce a family of G-plots for the following:

$$2 < x < 4$$
$$y = 2, y = 3, y = 4$$

Call the family of plots **Family_fixedY**. Refer to Figure 6-9. In it, you plotted several curves, each in its own window pane. Use the methods you learned to produce Figure 6-9 to produce a second window pane in the same figure and in it create a second family of G-plots this time for the following:

$$2 < y < 4$$
$$x = 2, x = 3, x = 4$$

Call the family of plots **Family_fixedX**. Your function **`Prob6_A_16`** should input the following arguments to generalize from the above situation. The inputs to **`Prob6_A_16`** should be the following:

> **`lowerLimit`** with the lower limit on **`x`** for **Family_fixedY** and the lower limit on **`y`** for **Family_fixedX**
>
> **`upperLimit`** with the upper limit on **`x`** for **Family_fixedY** and the upper limit on **`y`** for **Family_fixedX**

Your function **`Prob6_A_16`** should produce three curves in both families, **Family_fixedY** and **Family_fixedX**. The fixed values should be **`lowerLimit`**, **`(lowerLimit+upperLimit)/2`**, and **`upperLimit`**.

This is a fairly complicated problem at first reading, but if students will systematically understand what is being asked, it will not be too difficult for them. The key is systematic reading of the problem specification, and applying the *right* MATLAB tool for each part of the problem as needed.

A function which meets the specs of the problem is as follows.

```
function Prob6_A_16(lowerLimitV, upperLimitV)
```

```
% plot G = (x * (7.14 - 0.26*y)^2.1 ) / y^y
%   as 2D plots,
%       PLOT 1: with y constant at (0, 0.5, 1)
%       PLOT 2: with x constant at (2,3,4)
% INPUT: lowerLimitV, a two element vector
%             [<lowerLimitOnX_plot1, <lowerLimitOnY_plot2]
%        upperLimitV, a two element vector
%             [<upperLimitOnX_plot1, <upperLimitOnY_plot2]
% OUTPUT: none
% generate a graph
% sample call
%  Prob6_A_16([2, 5], [4, 7])

   figure

 % generate plot 1 in a subplot
   subplot(2,1,1)
   hold on
   x = linspace(lowerLimitV(1), upperLimitV(1));
   y = [lowerLimitV(2), ...
       (lowerLimitV(2)+upperLimitV(2))/2, ...
       upperLimitV(2)];
   G1 = (x .* (7.14 - 0.26*y(1)).^2.1 ) ./ (y(1)).^y(1);
   G2 = (x .* (7.14 - 0.26*y(2)).^2.1 ) ./ (y(1)).^y(2);
   G3 = (x .* (7.14 - 0.26*y(3)).^2.1 ) ./ (y(1)).^y(3);
   plot(x, G1, '-r')
   plot(x, G2, '--g')
   plot(x, G3, '-.b')
   legend(['y = ', num2str(y(1))],...
          ['y = ', num2str(y(2))],...
          ['y = ', num2str(y(3))] )
   hold off
   title( 'G(x,y) = (x * (7.14-0.26*y)^2^.^1 ) / y^y'...
          , 'FontSize', 14)
   text(2.4, 50, 'constant y')
   xlabel('x')
   ylabel('G')

 % generate plot 2 in a subplot
   subplot(2,1,2)
   hold on
   x = [lowerLimitV(1), ...
       (lowerLimitV(1)+upperLimitV(1))/2, ...
       upperLimitV(1)];
   y = linspace(lowerLimitV(2), upperLimitV(2));
   G1 = (x(1) .* (7.14 - 0.26*y).^2.1 ) ./ (y).^y;
   G2 = (x(2) .* (7.14 - 0.26*y).^2.1 ) ./ (y).^y;
   G3 = (x(3) .* (7.14 - 0.26*y).^2.1 ) ./ (y).^y;
   plot(y, G1, '-r')
   plot(y, G2, '--g')
   plot(y, G3, '-.b')
   legend(['x = ', num2str(x(1))],...
          ['x = ', num2str(x(2))],...
          ['x = ', num2str(x(3))] )
```

```
hold off
text(2.4, 50, 'constant x')
xlabel('y')
ylabel('G')
```

Problem 6-A.17

Another function **G(x,y,z)** of three independent variables is defined as:

$$G(x, y, z) = \frac{ze^{-0.3/x}}{\ln(y)\sqrt{x}}$$

Write a function, **Prob6_A_17**, that takes no input variables but that creates a three-pane vertical subgraph. The three subgraphs will be a plot of **G(x,y,z)** subject to the following:

Pane A

$$0.1 < x < 4$$
$$y = 5, z = 3$$

Pane B

$$2 < y < 4$$
$$x = 0.8, z = 3$$

Pane C

$$0 < z < 4$$
$$x = 0.8, y = 5$$

The problem is not as difficult as the one just before because the ranges of the variables are fixed. This is more a problem simply of using subplot effectively. Students should not miss the point that the built-in MATLAB function for the natural logarithm is log.

An extension to this problem which could be challenging is to alter this problem spec to develop a more flexible function as was done in Problem A_16.16.

A function that meets this problem specification as written follows.

```
function Prob6_A_17
% plot G(x,y,z) = (z*exp(-0.3/x)) / (log(y)*sqrt(x))
%   as 2D plots,
%       PLOT 1: var x in (0.1,4), y=5, z=3
```

```
%       PLOT 2: var y in (2,4), x=0.8, z=3
%       PLOT 3: var z in (0,4), with x=0.8, y=5
% INPUT: none
% OUTPUT: none
% generate a graph
% sample call
%  Prob6_A_17

  figure

 % generate plot 1 in a subplot
   subplot(3,1,1)
   x = linspace(0.1,4);
   y = 5;
   z = 3;
   G = (z*exp(-0.3./x)) ./ (log(y)*sqrt(x));
   plot(x, G, '-r'), grid on
   title( 'G(x,y,z) = (z*exp(-0.3/x)) / (log(y)*sqrt(x))'...
          , 'FontSize', 14)
   xlabel('x')
   ylabel('G')
   disp('move cursor to graph to put label for constants')
   gtext('y = 5'), gtext('z = 3')
   disp('DONE')

 % generate plot 2 in a subplot
   subplot(3,1,2)
   x = 0.8;
   y = linspace(2,4);
   z = 3;
   G = (z*exp(-0.3/x)) ./ (log(y)*sqrt(x));
   plot(y, G, '-g'), grid on
   xlabel('y')
   ylabel('G')
   disp('move cursor to graph to put label for constants')
   gtext('x = 0.8'), gtext('z = 3')
   disp('DONE')

 % generate plot 3 in a subplot
   subplot(3,1,3)
   x = 0.8;
   y = 5;
   z = linspace(0,4);
   G = (z*exp(-0.3/x)) / (log(y)*sqrt(x));
   plot(z, G, 'm'), grid on
   xlabel('z')
   ylabel('G')
   disp('move cursor to graph to put label for constants')
   gtext('x = 0.8'), gtext('y = 5')
   disp('DONE')
```

Problem 6-A.18

The following sample data represents the number of drivers that were ticketed on a given freeway in Michigan for traveling over 75 mph and for traveling under 55 mph.

	Jan	*Feb*	*Mar*	*Apr*	*May*	*Jun*	*Jul*	*Aug*	*Sep*	*Oct*	*Nov*	*Dec*
Over 75 mph	38	29	43	51	67	84	79	95	73	55	46	19
Under 55 mph	12	11	9	3	4	2	0	1	3	7	11	14

Create two column vectors: **`over75mph`** and **`under55mph`** that each have twelve elements as given in the data table above.

Create a function **`Prob6_A_18(over75mph, under55mph)`**.

Your function should generate an overlay plot of high and low speeders versus months (use month numbers). Mark the data points for **`over75mph`** with triangles pointing up and **`under55mph`** with triangles pointing down. Use different colors and widths of line for connecting the data points. Label your graph appropriately and generate a legend.

This problem is relatively straightforward. Although the problem spec does not call for it - an extension would be to make the tick marks on the X-axis be labeled with month *names*. Code to do that is included in the function below. But note that to really understand that code, the general idea of graphical objects, setting and fetching properties for graphical objects, and cell vectors would be needed. These topics are not included directly in the text.

```
function Prob6_A_18(over75,under55)
% creates a data plot of speed related tickets
% INPUT: over75, 12 element vector, Jan-Dec over 75 tickets
%        under55, 12 element vector, Jan-Dec under 55 tickets
% OUTPUT: none
% generates graph
% sample call
%   gt75 = [38 29 43 51 67 84 79 95 73 55 46 19];
%   lt55 = [12 11 9 3 4 2 0 1 3 7 11 14];
%   Prob6_A_18(gt75, lt55)

  figure
  hold on
```

```
monthNum = 1:12;

plot(monthNum, over75, '--r^', 'LineWidth', 2)
plot(monthNum, under55, '-.bV', 'LineWidth', 1)
grid on

xlabel('month', 'FontSize', 12)
ylabel('number of tickets', 'FontSize', 12)
legend('tix for > 75', 'tix for < 55')
title('Number of speed-related tickets in 12 months', ...
      'FontSize', 16')

% following code puts an X-tick mark for each month,
% and changes the X-tick label from month number to month
% name
set(gca,'XTick',1:12)
set(gca, 'XTickLabel', ...
    {'jan', 'feb', 'mar', 'apr', 'may', 'jun', ...
     'jul', 'aug', 'sep', 'oct', 'nov', 'dec'})

hold off
```

Problem 6-A.19

Consider the following abstract function of temperature T (in degrees Centigrade) as a function of time (in minutes):

$$T(t) = A\ln(t) - Be^{Ct}$$

Write a function **Prob6_A_19(T1_constants,T2_constants)**, whose input variables are each vectors describing a concrete version of **T(t)**. Suppose the following two inputs:

```
T1_constants = [A1, B1, C1];
T2_constants = [A2, B2, C2];
```

One input **T1_constants=[A1,B1,C1]** describes one concrete version of **T**, and the other input **T2_constants = [A2, B2, C2]** describes a second concrete version of **T**.

Your function **Prob6_A_19** inputs parameters for two distinctly different versions of **T(t)**, **T1** and **T2**.

Your function should graphically determine the intersection point(s) for **T1** and **T2**.

Write a short paragraph suggesting what physical situation **T(t)** may capture.

Most students will find this problem challenging, largely because the problem is relatively open ended. Student will need to first understand what is being asked in this problem, then develop a solution in code, then (finally) find reasonable values for the two inputs (two parameterizations of the function).

One physical situation this problem could be used to describe would be the temperature at some point inside a physical object when a burst of energy impinges on the surface of the object. As the outside of the object is flash heated to some temperature, heat begins to flow inward in the object. The internal temperature starts to raise, and continues to rise. Eventually the temperature peaks then begins to fall off as the entire object cools.

The three parameters in the equation are empirically derived and are dependent on the shape of the object, the distance inside that the measurement points resides at, the amount of energy that impinges on the object, and the material properties of the object. Although to understand this in detail is far beyond the current scope, the qualitative sense of using this equation to model the situation described should be within the reach of freshman students.

The code below meets the problem specifications. The use of sprintf is not a must for students, but it is convienient to know, and it is touched on in the text.

```
function crossAt = Prob6_A_19(T1_constants,T2_constants)
% Creates an overlay plot for two instances of
%   T(t) = A*log(t) - B*exp(C*t)
% and returns the cross points for the two instances
% INPUT: T1_constants, three element vector
%                      [A1, B1, C1]
%        T2_constants, three element vector
%                      [A2, B2, C2]
% OUTPUT: crossAt, XY cross points for the two instances
% creates a graph
% sample call
%   const1 = [200 .01 .25];
%   const2 = [100 .01 .20];
%   crossAt = Prob6_A_19(const1, const2)

   figure

   t = linspace(1,45);
   T1 = T1_constants(1)*log(t);
```

```
T2 = -T1_constants(2)*exp(T1_constants(3)*t);
T = T1 + T2;
plot(t, T)

hold on
T1 = T2_constants(1)*log(t);
T2 = -T2_constants(2)*exp(T2_constants(3)*t);
T = T1 + T2;
plot(t, T, '-r')
grid on

title('T(t) = A*log(t) - B*exp(C*t)', 'FontSize', 14)
xlabel('time (min)')
ylabel('Temperature (deg Centigrade)')

str1 = sprintf('A = %5.2f, B = %5.2f, C = %5.2f', ...
               T1_constants(1), ...
               T1_constants(2), ...
               T1_constants(3));
str2 = sprintf('A = %5.2f, B = %5.2f, C = %5.2f', ...
               T2_constants(1), ...
               T2_constants(2), ...
               T2_constants(3));
legend(str1, str2, 'Location', 'best')

disp('Put cursor on cross point, click. RET when done.')
crossAt = ginput;
disp('done - cross points follow')
```

Problem 6-A.20

Consider the following function **`F(x,y)`**:

$$F(x,y) = \frac{x(7.14 - 0.26y)^{2.1}}{(y-x)^{y}}$$

Write a function to create a plot showing a family of **`F(x,y)`** curves defined as follows:

$$2 < y < 6$$

$$x = 0.0, x = 0.5, x = 1.0, x = 1.5$$

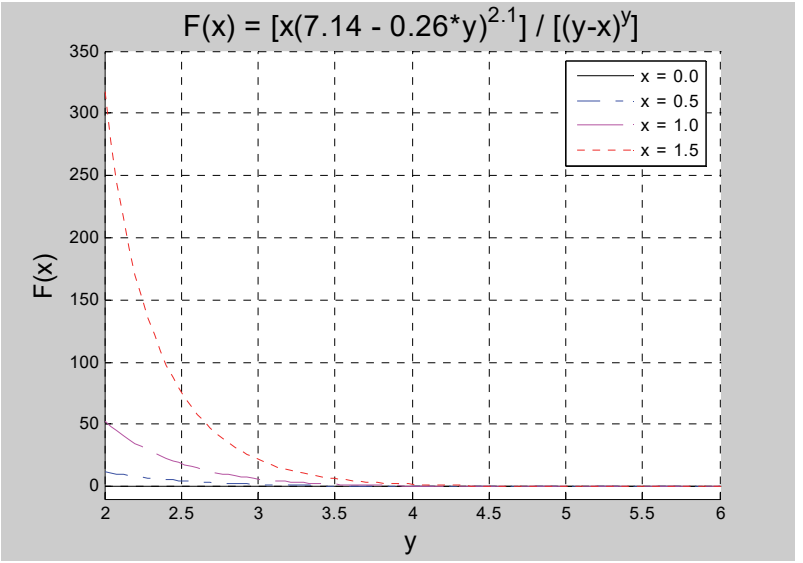
How many curves should you overlay?

Use a line width of 4 for each of your curves. Be sure to include a legend.

Save your figure as **Prob6_A_20.fig**.

Creating the code to meet the specification of this problem should be relatively straightforward for students. The following figure is an example result. (The figure below was produced with the lines being of width 1 for purposes of exposition in this manual. The code below meets the problem specs by setting line widths to 4.)

A reasonable way for students to get a "feel" for the function graphed is to try to decompose it and graph the individual parts of the decomposition on an overlay chart. The target function for this problem is susceptible to this approach: a decomposition to (a) the numerator and (b) the denominator of the function will help students to get a feel for this function.



The code in the script file below produces a graph meeting the specification for this problem.

```
% script file for Problem 6-A.20
y = linspace(2,6);

figure
hold on

x = 0.0;
```

```
F1 = x*(7.14 - 0.26*y).^2.1;
F2 = (y - x).^y;
F = F1 ./ F2;
plot(y,F, '-k', 'LineWidth', 4)

x = 0.5;
F1 = x*(7.14 - 0.26*y).^2.1;
F2 = (y - x).^y;
F = F1 ./ F2;
plot(y,F, '-.b', 'LineWidth', 4)

x = 1.0;
F1 = x*(7.14 - 0.26*y).^2.1;
F2 = (y - x).^y;
F = F1 ./ F2;
plot(y,F, '--m', 'LineWidth', 4)

x = 1.5;
F1 = x*(7.14 - 0.26*y).^2.1;
F2 = (y - x).^y;
F = F1 ./ F2;
plot(y,F, ':r', 'LineWidth', 4)

grid on
ylim([-10,350])
title('F(x) = [x(7.14 - 0.26*y)^2^.^1] / [(y-x)^y]', ...
      'FontSize', 16)
xlabel('y', 'FontSize', 14)
ylabel('F(x)', 'FontSize', 14)
legend('x = 0.0', 'x = 0.5', 'x = 1.0', 'x = 1.5')
```

Problem 6-A.21

Write a function, **Prob6_A_21**, to determine the intersection point graphically of the following curves:

$$F(x) = e^{-Bx}$$

$$G(x) = 1 - e^{-Bx}$$

Function **Prob6_A_21** should input the value of **B** and should use **ginput** to capture the coordinates of the intersection point. Finally, **Prob6_A_21** should return one vector containing the **x** and **y** coordinates of the intersection point.

By running **Prob6_A_21** with values of **B** of **[0.5,1.0,2.0]**, understand how the intersection point varies with value of **B**. Write a short, concise paragraph describing your observations.

To solve this problem, students will need to "experiment" to find appropriate lower and upper bound on x to make the chart most meaningful.

The code below produces a chart that goes beyond the problem spec in that the intersection point is labeled in the graph. The legend is necessary however, as is some means of labeling the chart with the value of B which produced it.

Symbolically, this problem has a solution too of course:

$$x_{intersect} = \frac{\ln(2)}{B}$$

A good exercise for students would be to ask them to "derive" the intersect value algebraically, then check their symbolic result by using the numerical/graphical solution they develop in MATLAB code - such as the function that is below. The function below includes computing the intersect values for the input B, and returns both numerical/graphical result and symbolic result for the intersection point.

Although not included in this text, MATLAB has a robust symbolic manipulation capability. The "symbolic toolbox" will be included in a second volume which is in progress. In MATLAB, the symbolic solution can be obtained as follows:

```
syms x B
solve('2*exp(-B*x) - 1')
```

The function below meets (and exceeds) the explicit requirements of this problem.

```
function [intPt, symbolic_intersect] = Prob6_A_21(B)
% graphically determine the intersection point of
%     F(x) = exp(-B*x);
%     G(x) = 1 - exp(-B*x);
% INPUT: b, scalar, free parameter of F and G
% OUTPUT: intPt, 2 element vector for the intersection point
%                [ X intersect val, Y interest val]
%         symbolic_intersect, 2 element vector for the
%                intersection point found from the symbolic
%                term for the x-intersect = ln(2)/B
% produces a graph
% sample call
% [num_int, sym_int] = Prob6_A_21(0.5)
```

```
figure, hold on
x = linspace(0,5);
F = exp(-B*x);
G = 1 - F;
plot(x, F, '-r')
plot(x, G, ':b')
title('Exp decay to 0   ---   Exp approach to 1', ...
      'FontSize', 14)
xlabel('x', 'FontSize', 10, 'FontWeight', 'bold')
ylabel('F(x)', 'FontSize', 10, 'FontWeight', 'bold')

disp('click on intersect point')
intPt = ginput(1);
intStr = ...
    sprintf('B = %2.1f\nintersect at (%2.1f, %2.1f)', ...
             B, intPt(1), intPt(2))
text(intPt(1), 0.2, intStr)
legend('F(x) = exp(-B*x)', ...
       'F(x) = 1 - exp(-B*x)', ...
       'Location', 'east')
hold off

x_intersect_sym = log(2)/B;
y_intersect_sym = exp(-B*x_intersect_sym);
symbolic_intersect = [x_intersect_sym, y_intersect_sym];
```

Problem 6-A.22

The following sample data represents the number of drivers that were ticketed on a given freeway in Michigan for traveling over 75 mph and for traveling under 55 mph.

	Jan	*Feb*	*Mar*	*Apr*	*May*	*Jun*	*Jul*	*Aug*	*Sep*	*Oct*	*Nov*	*Dec*
Over 75 mph	38	29	43	51	67	84	79	95	73	55	46	19
Under 55 mph	12	11	9	3	4	2	0	1	3	7	11	14

Write a function to create a 1 by 2 subplot. Create bar plots of high and low speeders. Arrange the widths of bars so no gap is in between. Name your function **`Prob6_A_22`**.

This problem starts out with the same data as Problem 6-A.18. But for Problem 6-A.25 the student is asked to make two bar charts. Bar plots are not covered in the text. Section 6-4 alerts the student to there being other types of charts that are available – many other types. Hence this problem is really aimed at student use of MATLAB HELP, and at exercising student use of subplots.

The function below uses the same general means as used in the solution for 6-A.18 to label the months on the x-axis.

There is on purposefully suboptimal requirement set in the problem. The two datasets are over the same time frame and aggregation frequency. Thus the total figure produced will be more illuminating if the subplots are stacked up/down instead of side by side. Students can usefully be asked to find this suboptimality. If the subplots are stacked up/down then an added benefit is that the title inserted will be centered as it should be.

```
function Prob6_A_22(over75,under55)
% creates two bar charts as subplots on a single figure
%   one subplot to show over 75 mph ticket data by month
%   the second to show under 55 mph data
% INPUT: over75, 12 element vector, Jan-Dec over 75 tickets
%        under55, 12 element vector, Jan-Dec under 55 tickets
% OUTPUT: none
% generates graph
% sample call
%   gt75 = [38 29 43 51 67 84 79 95 73 55 46 19];
%   lt55 = [12 11 9 3 4 2 0 1 3 7 11 14];
%   Prob6_A_22(gt75, lt55)

  figure

  subplot(1,2,1)
  bar(over75, 1)
  set(gca,'XTick',1:3:12)
  set(gca, 'XTickLabel', {'jan', 'apr','jul', 'oct'})
  ylabel('OVER 75 MPH', ...
         'FontWeight', 'bold')
  title('Num Tickets: OVER 75 MPH and UNDER 55 MPH', ...
        'FontSize', 14)


  subplot(1,2,2)
  bar(under55, 1)
  set(gca,'XTick',1:3:12)
  set(gca, 'XTickLabel', {'jan', 'apr','jul', 'oct'})
```

```
ylabel('UNDER 55 MPH', ...
       'FontWeight', 'bold')
```

Problem 6-A.23

Engineers often use the *small angle approximation* to simplify their calculations. An example application of small angle approximation is the calculation of angular displacement for small motions of a pendulum.

Here is one version of the small angle approximation:

$$\sin(x) = x$$

This proves to be a reasonable approximation provided x is given in radians.

Write a function, **`Prob6_A_23`**, to demonstrate graphically the validity of this approximation for angles from 0 to 40 degrees.

In a single figure, show the three separate plots:

a) An overlay plot of $y1 = \sin(x)$ and $y2 = x$ in the interval $[0, 40]$ degrees. Put a legend on your plot.

b) A plot of the absolute error, i.e., $y = abs(\sin(x) - x)$

c) A plot of the relative error, i.e., $y = abs\left(\frac{\sin(x) - x}{\sin(x)}\right)$

Write a short, concise paragraph discussing the results you see in the graph generated by your function.

This should be relatively straightforward as an exercise is both overlay plots and subplots. One point of interest that students should notice is the error that pops up in the command window: "Warning: divide by zero." This warning is generated in the computation of relative error for the angle equal to zero. The minor lesson here is the flexibility MATLAB provides to be able to handle this situation.

A function that meets the specification for this problem follows.

```
function Prob6_A_23
```

```
% show graphically the validity of the small angle
% approximation sin(x) = x by creating three subplots:
%  a) overlay plot of sin(x) and x
%  b) absolute error = abs(sin(x) -x)
%  c) relative error = abs( [sin(x)-x] / sin(x)  )
% INPUT: none
% OUTPUT: none
% generates graph
% sample call
%   Prob6_A_23

  figure

  ang = linspace(0,40);
  angRad = ang * (pi/180);  % convert to radians

  subplot(3,1,1), hold on
  plot(angRad, angRad, '-b')
  plot(angRad, sin(angRad), ':m')
  legend('approximation of sine', 'true sine', ...
         'location', 'northwest')
  title('Small Angle Approximation: sin(x) = x', ...
        'FontSize', 14)

  subplot(3,1,2)
  plot(angRad, abs(sin(angRad) - angRad), '-m')
  ylabel('absolute error', 'FontWeight', 'bold')

  subplot(3,1,3)
  plot(angRad, abs((sin(angRad) - angRad)./sin(angRad)),...
      '-m')
  ylabel('relative error', 'FontWeight', 'bold')
  xlabel('angle (radians)', 'FontWeight', 'bold')
```

Problem 6-A.24

Another version of the small angle approximation involves the cosine:

$$\cos(x) = 1 - \frac{x^2}{2}$$

Write a function, **`Prob6_A_24`**, to create analogous subplots to those required in Problem 6-A.23. This time, find the limit on the angle if the relative error must stay within one percent. In your relative error plot, draw a horizontal red dotted line at one percent error. Label the intersection of this line with the curve as the upper limit using **`text`** or **`gtext`**.

This problem is very similar to the last problem. However, freshman students are likely to have some difficulty in transferring what was learned in 6-A.23 to this problem. The reason is that the level of transfer for some will be to match syntactically, and they will miss the concepts that drive the transfer. The points of similarity are that the approximation for sine is x, while the approximation for cosine is 1-x^2/2.

The function below meets the problem specifications.

```
function Prob6_A_24
% show graphically the validity of the small angle
% approximation cos(x) =  1 - (x^2/2)
% call the approximation cos_app
% by creating three subplots:
%  a) overlay plot of cos(x) and cos_app
%  b) absolute error = abs(cos(x) - cos_app)
%  c) relative error = abs( [cos(x)-cos_app] / cos(x) )
% in (c) put in the line for relative error of 1%
% INPUT: none
% OUTPUT: none
% generates graph
% sample call
%   Prob6_A_24

  figure

  ang = linspace(0,40);
  angRad = ang * (pi/180);  % convert to radians
  cos_app = 1 - angRad.^2/2;

  subplot(3,1,1), hold on
  plot(angRad, cos_app, '-b')
  plot(angRad, cos(angRad), ':m')
  legend('approximation of cosine', 'true cosine', ...
         'location', 'southwest')
  title('Small Angle Approximation: cos(x) = 1 - x^2/2', ...
        'FontSize', 14)

  subplot(3,1,2)
  plot(angRad, abs(cos(angRad) - cos_app), '-m')
  ylabel('absolute error', 'FontWeight', 'bold')

  subplot(3,1,3)
  hold on
  plot(angRad, abs((cos(angRad) - cos_app)./cos(angRad)),...
      '-m')
  ylabel('relative error', 'FontWeight', 'bold')
  xlabel('angle (radians)', 'FontWeight', 'bold')
```

```
const01 = .01 * ones(1,length(angRad));
plot(angRad, const01, '-r', 'LineWidth', 2)
text(0.5, 0.012, 'cross over for 1%')
```

Problem 6-A.25

A numerical approach to plotting functions (using **plot**) may not always produce accurate results.

Create an overlay plot of $y = \tan(x)$ in the interval $[-\pi, \pi]$ using **ezplot** and **plot**.

What is the value of $\tan(x)$ when the angle is $\pi/2$ or $-\pi/2$? Which curve reflects the behavior of $\tan(x)$ more accurately?

Write a concise paragraph explaining your observations.

Most students will find this problem somewhat challenging. The crux reason for this disparity between the numerical plot and the symbolic plot is that for the numerical plot we assume a finite number of values for the independent variable, then compute corresponding values for the dependent variable. When a singularity in the functional relationship is present, then no matter how many values we choose to employ, it will not be enough close to the singularity. On the other hand, the symbolic plotting tool recognizes this problem and deals with it appropriately.

An exercise to help students see this is to instruct them to go to their use of linspace and increase the number of values generated for the independent variable – say to 1000.

The problem specification indicates the problem is to be solved using an overlay plot. In reality, this purposefully inserted suboptimal instruction should prod the student to try the problem with subplots instead.

The script file below produces both an overlay plot and a figure with subplots.

```
% script file for Problem 6-A.25

 % create an overlay plot meeting prob specs
   figure
   hold on
```

```
grid on

x = linspace(-pi,pi);
plot(x, tan(x), ':r')
ylim([-2*pi, 2*pi])
ezplot('tan(x)', [-pi, pi])
legend('tan(x) from plot function', ...
       'tan(x) from ezplot function')
set(gca,'XTick',-pi:pi/2:pi)
set(gca, 'XTickLabel', ...
         {'-pi', '-pi/2', '0', 'pi/2', 'pi'})

% create an subplot figure meeting prob specs
figure

subplot(2,1,1)
grid on
x = linspace(-pi,pi);
plot(x, tan(x), ':r')
title('tan(x) from plot function', 'FontSize', 14)
xlabel('x (radians)')
xlim([-pi, pi])
ylim([-2*pi, 2*pi])
set(gca,'XTick',-pi:pi/2:pi)
set(gca, 'XTickLabel', ...
         {'-pi', '-pi/2', '0', 'pi/2', 'pi'})

subplot(2,1,2)
grid on
ezplot('tan(x)', [-pi, pi])
title('tan(x) from ezplot function', 'FontSize', 14)
set(gca,'XTick',-pi:pi/2:pi)
set(gca, 'XTickLabel', ...
         {'-pi', '-pi/2', '0', 'pi/2', 'pi'})
```

Problem 6-A.26

Use **ezplot3** to make a 3D line plot of the following system of equations in the closed interval $[0,5\pi]$

$$x = \cos(t)$$

$$y = \sin(t)$$

$$z = e^{-t}$$

Write a function that takes the range of **t** as a vector input and produces this plot. Name the function **Prob6_A_26**.

```
This problem is very similar to the example problem solved in
Section 6-5.1.1.

function Prob6_A_26(t)
% This function inputs a range for the independent variable
% and plots x = cos(t), y = sin(t), z = e^-t
% INPUT: vector [tmin tmax]
% OUTPUTS: none

figure
ezplot3('cos(t)', 'sin(t)', 'exp(-t)', t);
```

Problem 6-A.27

Use the **meshgrid, griddata** and **mesh** functions of MATLAB to create a plot of the following dataset.

Table 6-1:

x	y	z
2.50	2.88	0.00
-0.38	-2.25	1.72
-2.62	-2.62	0.34
-2.13	2.13	-0.29
1.38	1.25	0.35
-2.38	1.38	-2.19
-2.13	2.88	3.25
2.63	3.00	-0.46
-0.38	-2.75	2.32
-1.88	-0.13	1.42

Set the marker color to green, and the marker size to 12. Set the font weight and size for all labels to bold and 14. Choose the ‘copper’ color map.

```
This problem is very similar to the example problem solved in
Section 6-5.2.

% Script file for Problem 6-A.27
% Plot the data given in the problem specs

clear

dataset = [2.50 2.88    0.00
-0.38   -2.25   1.72
-2.62   -2.62   0.34
-2.13   2.13    -0.29
1.38    1.25    0.35
-2.38   1.38    -2.19
-2.13   2.88    3.25
2.63    3.00    -0.46
-0.38   -2.75   2.32
-1.88   -0.13   1.42];

x = dataset(:,1); y = dataset(:,2); z = dataset(:,3);

rangeX = linspace(min(x), max(x), 20);
rangeY = linspace(min(y), max(y), 20);
[XI,YI] = meshgrid(rangeX ,rangeY );
ZI = griddata(x,y,z,XI,YI);

figure
mesh(XI,YI,ZI), hold on

plot3(x,y,z,'rp', 'MarkerFaceColor', 'g', 'MarkerSize', 12)
xlabel('x', 'FontWeight', 'bold')
ylabel('y', 'FontWeight', 'bold')
zlabel('z', 'FontWeight', 'bold')
title('Mesh Plot:Dataset of Three Variables', 'FontSize', 14)
colormap('copper')
```

Problem 6-A.28

Use **ezsurf** and **ezmesh** to make a 3D surface plot of the following function:

$$z(x, y) = 6(y^2 - x^2)^3 + (1 - y)^6$$

Write a function to solve this problem. Your function should take no inputs and produce two graphs, one for **ezsurf** and one for **ezmesh**. Name the function **Prob6_A_28**.

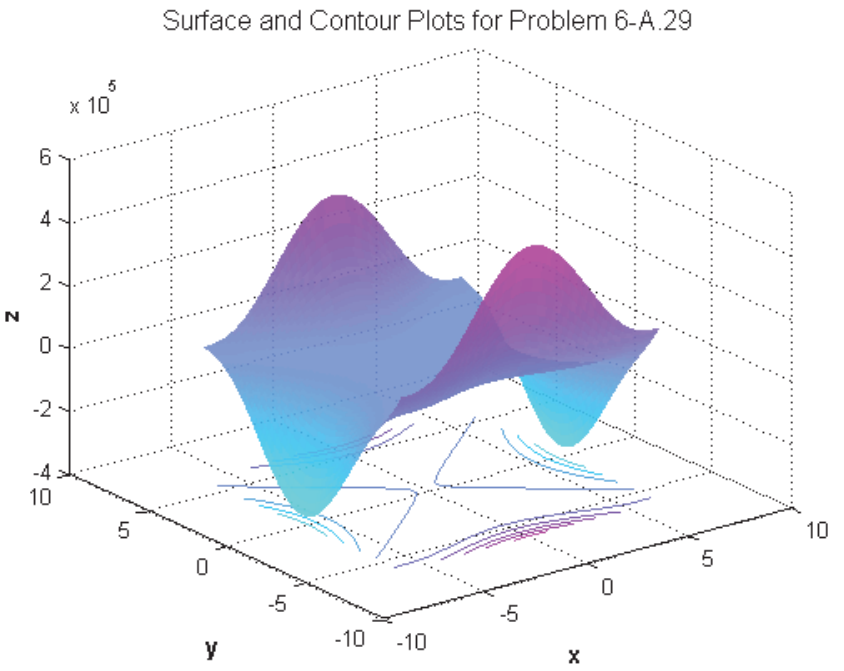
```
function Prob6_A_28
% makes a surface plot of 6*(y^2-x^2)^3+(1-y)^6
% in the range [-2pi, 2pi], [-2pi, 2pi]
% INPUTS: none
% OUTPUTS: none

figure
ezsurf('6*(y^2-x^2)^3+(1-y)^6')
title('Graph using ezsurf')
figure
ezmesh('6*(y^2-x^2)^3+(1-y)^6')
title('Graph using ezmesh')
```

Problem 6-A.29

Modify your solution for **Prob6_A_28**, and this time show both the surface and contour plots on the same graph. Set the transparency constant of the surface to 0.9. Set the shading to 'flat' and color map to 'cool'. Name your function **Prob6_A_29.**

Hint: Search for **surfc** in MATLAB help. Your graph should look similar to the one shown below.



This function primarily is an exercise in using MATLAB help. Students are encouraged to search for help in surfc, shading, colormap and alpha functions, and learn by themselves how to use these tools.

```
function Prob6_A_29
% makes an overlay plots of both the surface and
% contour plots of 6*(y^2-x^2)^3+(1-y)^6
% in the range [-2pi, 2pi], [-2pi, 2pi]
% INPUTS: none
% OUTPUTS: none

figure
x=linspace(-2*pi,2*pi,100);
y=linspace(-2*pi,2*pi,100);
[x,y]=meshgrid(x,y);
z=6*(y.^2-x.^2).^3+(1-y).^6;
surfc(x,y,z)

xlabel('x', 'FontWeight', 'bold', 'FontSize', 11)
ylabel('y', 'FontWeight', 'bold', 'FontSize', 11)
zlabel('z', 'FontWeight', 'bold', 'FontSize', 11)
title('Surface and Contour Plots for Problem 6-A.29', ...
    'FontSize', 12)
shading flat
```

```
colormap('cool')
alpha('.9')
```

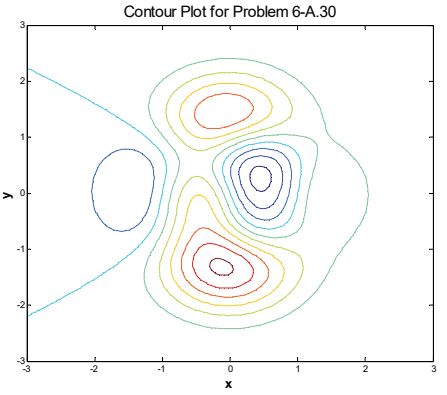
Problem 6-A.30

Given **z** for the independent variables **x** and **y**:

$$z(x,y) = 5\left(x^3 + y^4 - x - \frac{y}{3}\right)e^{(-x^2-y^2)}$$

Make a contour plot of **z**.

For this problem, write a function **Prob6_A_30** that takes two vectors as input: the interval for **x** and the interval for **y**. Your function should output a contour graph for the given range of **x** and **y**. Try your function with varying inputs to find the most meaningful range. Your result should resemble the graph given on the right.




```
function Prob6_A_30(xint,yint)
% generates a contour plot for the function
% whose specs are given in the problem
% INPUTS: a vector, the range for x
%         a vector, the range for y
% OUTPUTS: none
% Sample call:
% Prob6_A_30([-3 3],[-3 3])

x=linspace(xint(1),xint(2));
y=linspace(yint(1),yint(2));
[x,y]=meshgrid(x,y);
z=5*(x.^3+y.^4-x-y/3).*exp(-x.^2-y.^2);
contour(x,y,z);
xlabel('x','FontWeight', 'bold', 'FontSize', 14)
ylabel('y','FontWeight', 'bold', 'FontSize', 14)
```

```
title('Contour Plot for Problem 6-A.30', 'FontSize', 16)
```

Problem 6-A.31

A helix in the 3D coordinates is defined as:

$$x = \cos(t)$$
$$y = \sin(t)$$
$$z = t$$

Write a function that takes no inputs and plots this helix in the range $[0,8\pi]$. Use **plot3** in your solution. Put a grid on your plot.

Name your function for this problem **Prob6_A_31**.

```
function Prob6_A_31
% creates a plot of a helix defined by:
% x = cos(t), y = sin(t), z = t
% INPUTS: none
% OUTPUTS: none

t=linspace(0,8*pi);
x=cos(t); y=sin(t); z=t;

figure
plot3(x,y,z), grid on
xlabel('x')
ylabel('y')
zlabel('z')
title('Helix for Problem 6-A.31')
```

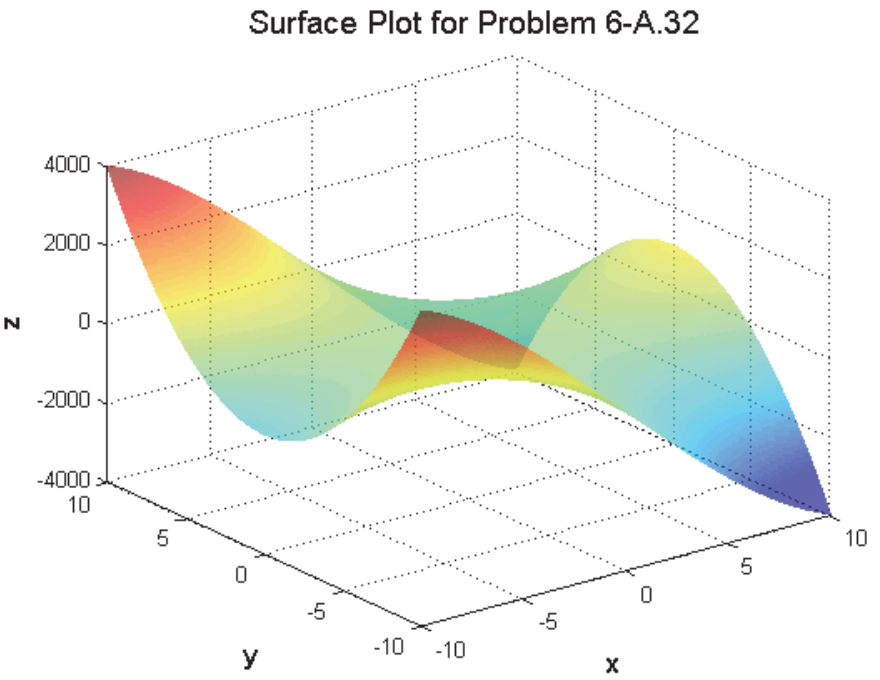
Problem 6-A.32

Plot the following function using **surf**.

$$z(x, y) = x^3 - 5xy^2$$

Adjust the axes for **x** and **y** and make sure you show the characteristics of the surface. Write a function **Prob6_A_32** that takes the ranges for **x** and **y** as vectors and produces the plot.

When finished, your graph should look like the one below.




```
function Prob6_A_32(xint,yint)
% generates a surface plot of z = x^3 - 5xy^2
% for a given range of x and y
% INPUTS: vector for range of x
%         vector for range of y
% OUTPUTS: none
% Sample call:
% Prob6_A_32([-10 10],[-10 10])

x=linspace(xint(1),xint(2));
y=linspace(yint(1),yint(2));
[x,y]=meshgrid(x,y);
z=x.^3-5*x.*y.^2;

figure
surf(x,y,z)
shading flat, alpha(.6)
```

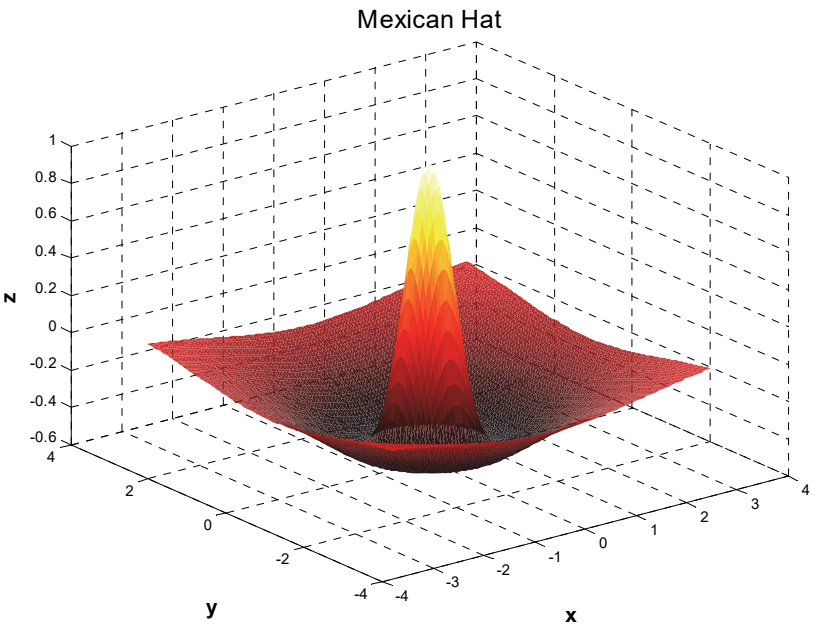
```
xlabel('x','FontWeight', 'bold', 'FontSize', 12)
ylabel('y','FontWeight', 'bold', 'FontSize', 12)
zlabel('z','FontWeight', 'bold', 'FontSize', 12)
title('Surface Plot for Problem 6-A.32','FontSize', 14)
```

Problem 6-A.33

Plot a Mexican Hat Surface defined by the following function:

$$z(x, y) = \left(e^{-(x^2 + y^2)/0.3} - 0.4e^{-(x^2 + y^2)/7} \right) / 0.6$$

Write a function called **`Prob6_A_33`** that takes no inputs and produces the plot. A Mexican Hat surface looks like the following:




```
function Prob6_A_33
% generates a surface plot a Mexican Hat surface
% INPUTS: none
```

```
% OUTPUTS: none

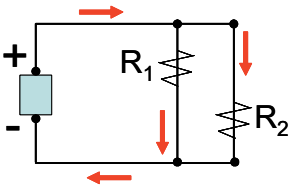
x=linspace(-pi,pi);
y=linspace(-pi,pi);
[x,y]=meshgrid(x,y);
z=(exp(-(x.^2+y.^2)/0.3)-0.4*exp(-(x.^2+y.^2)/7))/0.6;

figure
surf(x,y,z)
shading flat, colormap hot
xlabel('x','FontWeight', 'bold', 'FontSize', 14)
ylabel('y','FontWeight', 'bold', 'FontSize', 14)
zlabel('z','FontWeight', 'bold', 'FontSize', 14)
title('Mexican Hat','FontSize', 16)
```

Set B: Problem Solving with MATLAB 2-D Plotting

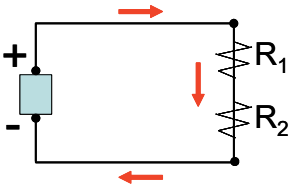
Problem 6-B.1

A parallel circuit containing two resistors is shown to the upper right. The resistors R_1 and R_2 are in parallel because the current that flows through the circuit (depicted by the arrows) is split, some flows through the R_1 line and some goes through the R_2 line.



resistors in parallel

Parallel circuits are contrasted with series circuits. In a series circuit like the one shown to the lower right there are the same two resistors. In this case though all current flows through each of the resistors.



resistors in series

Suppose you have built a parallel circuit of **n** identical electrical resistors, each of **r** ohms. The effective resistance in the circuit **R** is given by the following equation:

$$R(r,n) = \frac{r}{n}$$

Plot equivalent resistance versus number of resistors for $R(r,n)|_{r=100}$ in the interval for n $[1, 20]$. The symbol $R(r,n)|_{r=100}$ means the function **R** with **r** fixed at 100 ohms.

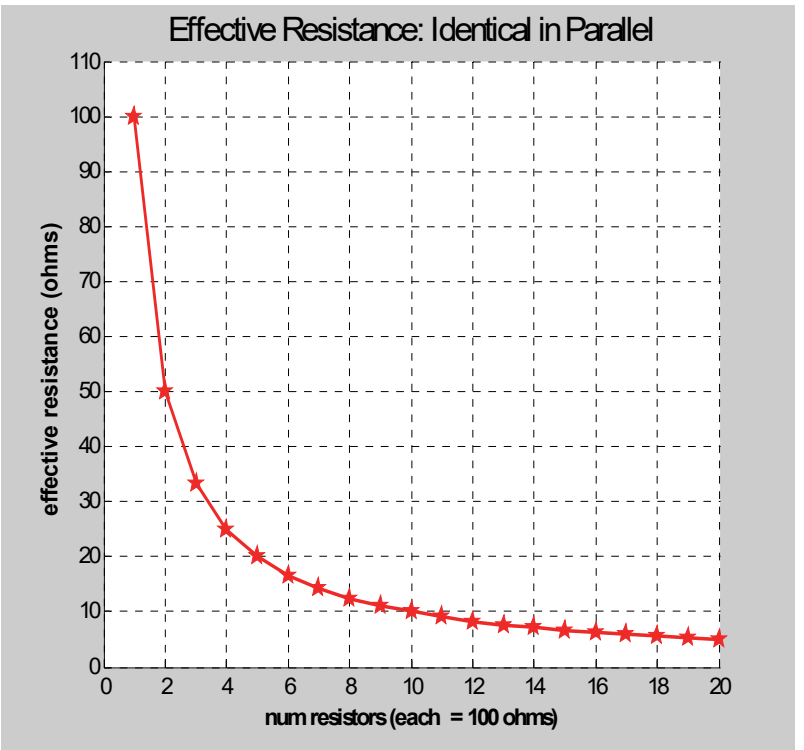
Use an appropriate type of line in your plot (a phrase like '1.5 resistors' is meaningless). Fully label your graph.

Save your plot as **Prob6_B_1.fig**.

Students will find this problem straight forward provided they patiently read the problem specification.

In the text, the case is made strongly that when plotting a functional relation, data markers should *not* be used. The solution below over rides that rule because of the part of the problem specification emphasizing the non-divisible nature of resistors – only integer values on the x-axis are physically meaningful.

Students should produce a chart similar to that below.

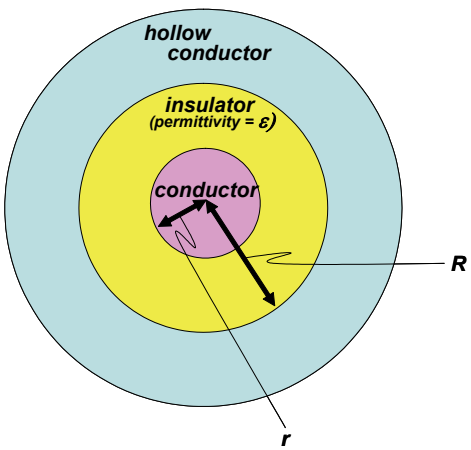


```
function Prob6_B_1(r, nBounds)
% create a plot of the effective resistance for n identical
% resistances, each of r ohmns. The effective resistance is
% given by
%   R(r,n) = r / n
%   for nLower < n < nUpper
% INPUT: r, scalar, the value of each resistance
%        nBounds, two element vector
%                 [lowerBoundOnN, upperBoundOnN]
% OUTPUT: none
% generates a graph
% sample call
%   Prob6_B_1(100, [1,20])

  figure, hold on
  nVector = nBounds(1):nBounds(2);
  R = r ./ nVector;
  plot(nVector, R, 'rp', 'MarkerSize', 10, ...
                         'MarkerFaceColor', 'red')
  plot(nVector, R, ':r', 'LineWidth', 2)
  ylim([0, max(R) + 10])
  grid on
  title('Effective Resistance: Identical in Parallel', ...
        'FontSize', 14)
  labelStr = sprintf('num resistors (each  = %3.0f ohms)', r);
  xlabel(labelStr, 'FontWeight', 'bold')
  ylabel('effective resistance (ohms)', 'FontWeight', 'bold')
```

Problem 6-B.2

Coaxial cable is constructed by placing a core of conducting material inside an insulating material, then placing that inside a hollow cylindrical conductor. A cross-section for a coaxial cable is sketched to the right, where ***R*** is the outer radius of the insulating ring, ***r*** is the inner radius of insulating ring, and ε is the permittivity of the insulator, a physical constant dependent on the



type of insulator. The permittivity is a measure of how well the material insulates.

The capacitance per unit length ***C*** (farads/m) of a coaxial cable can be computed using the following relation:

$$C = \frac{2\pi\varepsilon}{\ln\left(\frac{R}{r}\right)}$$

Remember: the mathematical function **ln** is the natural logarithm.

Write a function, **Prob6_B_2_core**, that inputs ε, ***R***, and a vector holding the lower and upper limit on values of ***r***. Your function should examine the effect on ***C*** of changes in ***r*** for given values for ε and ***R*** by creating an appropriate plot.

Test your function by calling it with the following:

```
epsilon = 2.0364 x 10^-11;          % farads/m
R = 4;                              % mm
r_range =  [0.1, 3.9];              % mm
Prob6_B_2(epsilon, R,  r_range)
```

Write another function, **Prob6_B_2_top**, to examine the effect of both **R** and **r** on the unit capacitance of a coaxial cable. Do this by supplying **Prob6_B_2_top** test values to use for **R**; four test values will be supplied for **R**. **Prob6_B_2_top** will set up an overlay plot, one plot for each value of **R** will be put into the chart. **Prob6_B_2_top** should call **Prob6_B_2_core** to put on each one of the **R**-value plots. Test **Prob6_B_2_top** by calling it with the following:

```
epsilon = 2.0364 x 10^-11;                % farads/m
R_testVal = [4.0, 4.15, 4.30, 4.50];      % mm
r_range =  [0.1, 3.9];                    % mm
Prob6_B_2_top(epsilon,R_testVal,r_range)
```

If you call your function with values such that ***r>R*** for some instances, what will the value of the unit capacitance be for those cases (greater or less than 0)? What is wrong with this question? Write your answers in a concise paragraph.

This is a relatively complex problem from a number of perspectives. First, the physical situation must be understood, and to do that, students need study the sketch that comes with the

problem. The sketch shows a cross section. A key way to judge if students are understanding the physical situation is to ask if a unit conversion is necessary since the permittivity has units of Farads/meter while the linear units on the cross section are millimeters. If students do not understand that there is no units conversion necessary then they are not understand the physical set up – nor for that matter the nature of the equation for capacitance.

Second, students are asked to develop a two-function solution to the problem where a "core function" plots data for a single given value of R, and a "top level function" carries out the task of plotting for four test values of R. This should be conceptually not too difficult for students at this stage in their MATLAB studies, but one point will be a tripping point in many cases: a top level function which calls another function expecting a plot to be added to an existing graphic must supply the called function what line spec the plot should use.

The two functions below use a number of points that go beyond the stated problem specification, but which are accessible to students through MATLAB HELP.

```
% the top level function -----------------------------
function Prob6_B_2_top(epsilon, R_testVal, r_range)
% plots the capacitance of a coaxial insulator versus the
% radius of the wire being shielded, given the permittivity
% of the insulator (epsilon), four test values for the radius
% of the radial cross section of the insulator (R_testVal),
% and the range of values of the radius of the wire being
% shielded (rBounds)
% CALLS: Prob6_B_2_core
% INPUTS: epsilon, scalar, permittivity of insulator
%                          (farads/meter)
%         R_testVal, 4 element vector,
%                    radius of cross section from center of
%                    shielded wire to outer edge of insulator
%                          (millimeters)
%         r_range, vector, upper and lower limits on the
%                  range or radius of wire insultated
%                           (millimeters)
% OUTPUT: none
% produces graph, all labeling done in this function rather
%                 than in Prob6_B_2_core
% sample call
%    epsilon = 2.0364*10^-11;              % farads/m
%    R_testVal = [4.0, 4.15, 4.30, 4.50];  % mm
%    r_range =  [0.1, 3.9];                % mm
%    Prob6_B_2_top(epsilon, R_testVal, r_range)

  figure, hold on, grid on
  Prob6_B_2_core(epsilon, R_testVal(1), r_range, '-k')
  Prob6_B_2_core(epsilon, R_testVal(2), r_range, ':b')
  Prob6_B_2_core(epsilon, R_testVal(3), r_range, '-.m')
```

```
  Prob6_B_2_core(epsilon, R_testVal(4), r_range, '--g')

  title(['Unit Coax Capacitance (\epsilon = ', ...
          sprintf('%11.5g)', epsilon)], ...
          'FontSize', 14)
  xlabel('radius of conductor wire (mm)', ...
         'FontWeight', 'bold')
  ylabel('unit capacitance (farads/meter)', ...
         'FontWeight', 'bold')
  disp('place annotation in graph')
  gtextStr = sprintf('%s\n%s',...
                     'situation is conductor of radius r,',...
                     'with coax insulator of radial sector R');
  gtext(gtextStr, 'FontWeight', 'bold')
  legend(sprintf('R = %3.2f (mm)', R_testVal(1)), ...
         sprintf('R = %3.2f (mm)', R_testVal(2)), ...
         sprintf('R = %3.2f (mm)', R_testVal(3)), ...
         sprintf('R = %3.2f (mm)', R_testVal(4)), ...
         'Location', 'NorthWest')


% the called function -----------------------------
function Prob6_B_2_core(epsilon, R, rBounds, plotSpec)
% plots the capacitance of a coaxial insulator versus the
% radius of the wire being shielded, given the permittivity
% of the insulator (epsilon), the radius of the radial
% cross section of the insulator (R), and the range of
% values of the radius of the wire being shielded (rBounds)
% INPUTS: epsilon, scalar, permittivity of insulator
%                          (farads/meter)
%         R, scalar, radius of cross section from center of
%                    shielded wire to outer edge of insulator
%                          (millimeters)
%         rBounds, vector, upper and lower limits on the
%                  range or radius of wire insultated
%                           (millimeters)
%         plotSpec, string, line specification for plot
% OUTPUT: none
% produces graph
% sample call
%    epsilon = 2.0364*10^-11;            % farads/m
%    R = 4;                              % mm
%    r_range =  [0.1, 3.9];              % mm
%    Prob6_B_2_core(epsilon, R, r_range)

  r = linspace(rBounds(1), rBounds(2));
  C1 = 2*pi*epsilon;
  C2 = log(R ./ r);
  C = C1 ./ C2;
  plot(r, C, plotSpec)
```

Problem 6-B.3

Implicit polynomials can be used for object recognition in images, for defect detection in quality control problems, and for interpretation of magnetic resonance images (MRIs). The following contour data are extracted from an image of a butterfly.

x	0.0	-4.0	-14.0	-23.0	-23.0	-19.0	-5.0	0.0	6.0	17.0	24.0	21.0	6.0
y	-2.0	-12.0	-20.0	-11.0	11.0	19.0	14.0	2.9	13.0	20.0	0.0	-18.0	-13.0

A pattern recognition specialist derives the following implicit polynomial that represents the above data:

$$f(x, y) = 0.000032044x^4 + 0.000016821y^4 - 0.0185x^2$$

Do an overlay plot of the data and the function $f(x,y)=0$. Show the data points with blue pentagrams. How accurate is the implicit polynomial model in representing the butterfly data?

Label your plot appropriately.

This problem is not difficult, although some students will need to refer to the section in the text on ezplot: implicit functions. In addition, some students will need encouragement to experiment to find the X and Y limits for the call to ezplot.

The script file below meets the problem specification - although the marker color has been changed to red to make it stand out more. Setting the specification for blue data markers was purposefully inserted to encourage students to think about the finished product (the graph in this case) and ask about the specification – and changing it.

```
% script file for Problem 6-B.3
figure
ezplot('0.000032044*x^4 + 0.000016821*y^4 - 0.0185*x^2', ...
        [-25,25, -25, 25])
```

```
x = [0.0 -4.0 -14.0 -23.0 -23.0 -19.0 -5.0 ...
     0.0 6.0 17.0 24.0 21.0 6.0];
y = [-2.0 -12.0 -20.0 -11.0 11.0 19.0 14.0 ...
     2.9 13.0 20.0 0.0 -18.0 -13.0];
hold on,
plot(x,y, 'rp', ...
       'MarkerFaceColor', 'r', ...
       'MarkerSize', 10)

legend('implicit polynomial fit', ...
       'actual data points', ...
       'Location', 'north')
```

Problem 6-B.4

MATLAB functions **tic** and **toc** are used for timing the execution of operations. The function **display** prints a string to the Matlab Command Window. The function **input** prompts the user to supply a value. Look up these functions in MATLAB help.

Write a MATLAB function named **Prob6_B_4** that performs the following:

Part A Displays a random integer number to the user. Utilizes **round(100*rand)** to generate the random number, and look up **round** and **rand** so you understand how they are being used.

Part B Starts a stopwatch.

Part C Asks the user to input *three times* the number displayed (from Part A).

Part D Stops the stopwatch when the user enters an answer.

Part E Displays the number of seconds the user took to enter the answer.

Part F Checks if the user is correct. Displays 1 if the answer is right, 0 otherwise.

```
This problem is a departure; there is no plotting required here.
This is a problem totally aimed at exercising student use of
MATLAB HELP. The code below meets the problem specification.
```

```
function Prob6_B_4
% Asks the user to multiply 3 times a random number between
% 0 and 100. Records the time for user response, and
% displays the time to respond and whether or not the answer
% input was correct.
% INPUT: none
% OUTPUT: none
% sample call
%   Prob6_B_4

    myRand = round(100*rand);
    tic
        inputStr = ...
             sprintf('\nwhat is three times %2.0f? >>', myRand);
        userAns = input(inputStr);
    toc
    timeRequired = toc;
    userAnswerCorrect = (userAns == 3*myRand);
    responseTimeStr = ...
        sprintf('\nThe user took %3.1f seconds to answer',...
                 timeRequired);
    display(responseTimeStr);
    display('and the respose was (1=correct, 0=incorrect)');
    userAnswerCorrect
```

Problem 6-B.5

The height **h** (m) and horizontal distance **x** (m) traveled by a ball thrown at an angle α (degrees) with an initial speed $\mathbf{v_0}$ (m/s) are given by the following relation:

$$x(t) = v_0 \cos(\alpha) t$$

$$h(t) = v_0 \sin(\alpha) t - \frac{1}{2} g t^2$$

g is the acceleration due to gravity (9.81 m/s/s).

The ball will hit the ground at time $\mathbf{t_{hit}}$ (sec):

$$t_{hit} = 2\frac{v_0}{g}\sin(\alpha)$$

Part A Create a function, **Prob6_B_5a**, that inputs the initial speed and angle of the ball's flight. Your function should plot height

versus distance traveled by the object in the interval $[0, t_{hit}]$. Your function should return **t_{hit}**. Test your function by calling it with an initial speed $v_0 = 50$ m/s, and an initial angle $\alpha = 65^o$.

Part B Create a second function, **Prob6_B_5b**, whose functionality is like **Prob6_B_5a** except the plots show green if the height is less than or equal to 40 meters, cyan if the height is greater than 40 meters and less than or equal to 80 meters, and blue if the height is greater than 80 meters.

To annotate your plots in both Part A and Part B, look up in MATLAB HELP and use the built-in function **texlabel** to create text in your plot for the two equations above.

The basic problem solving for this problem is to (a) find t_{hit} and then use it as the upper limit on a vector for time (0, t_{hit}). Using the time vector, produce a corresponding height vector using the relationship given. The rest of the problem specification has to do with labeling of the resultant plots. Note that the inbuilt function texlabel is introduced.

Most students will have difficulty with Part B. Most will partition the data into three segments using find, then plot the three data subsets. The difficulty is that following this procedure creates to extraneous horizontal lines: one at y=40 m, and one at y=80 m. The difficulty is that the data must be partitioned into five data subsets, as shown in the code for Part B below.

```
%   CODE FOR PART A ============================================

function tHit = Prob6_B_5a(initSpeed, firingAngle)
% For a projectile shot with initial speed initialSpeed and
% firing angle firingAngle, compute and return the time of
% flight of the projectile assuming the earth's curvature
% may be neglected as well as air resistance. In addition,
% create a plot of projectile height versus flight time.
% INPUT: initialSpeed, scalar, in m/s
%        firingAngle, scalar, radians
% OUTPUT: tHit, scalar, seconds
% create plot
% using internal constant g=9.81 m/s/s
% sample call
%   timeToHit = Prob6_B_5b(70, 45)
```

```
  figure
  g = 9.81;                % m/s/s
  firingAngle_rad = firingAngle*pi/180;   % radians
  tHit = 2*(initSpeed/g)*sin(firingAngle_rad);  % s
  t = linspace(0, tHit);  % s

  height = initSpeed * sin(firingAngle_rad) * t  ...
            - 0.5*g*t.^2;  % m

  plot(t,height)
  grid on
  title('Height vs Flight Time for Projectile', 'FontSize', 14)
  xlabel('flight time (sec)', 'FontWeight', 'bold')
  ylabel('height (m)', 'FontWeight', 'bold')

  equStr = texlabel('t_hit = 2(v_0/g)*sin(alpha)');
  disp('place equation for tHit')
  gtext(equStr, 'FontSize', 14)

  equStr = texlabel('h = v_0*sin(alpha)*t - (1/2)*g*t^2');
  disp('place equation for height')
  gtext(equStr, 'FontSize', 14)

  str = sprintf(['initial speed (v_0) = %4.0f m/s\n', ...
                 'firing angle (alpha) = %3.0f deg'], ...
                initSpeed, firingAngle);
  str = texlabel(str);
  disp('place parameters')
  gtext(str, 'FontSize', 14, 'Color', 'blue')



%   CODE FOR PART B ===========================================

function tHit = Prob6_B_5b(initSpeed, firingAngle)
% For a projectile shot with initial speed initialSpeed and
% firing angle firingAngle, compute and return the time of
% flight of the projectile assuming the earth's curvature
% may be neglected as well as air resistance. In addition,
% create a plot of projectile height versus flight time.
% INPUT: initialSpeed, scalar, in m/s
%        firingAngle, scalar, radians
% OUTPUT: tHit, scalar, seconds
% create plot: if height < 40m, then green
%              if 40m < height < 80m, then cyan
%              if 80 < height, then blue
% using internal constant g=9.81 m/s/s
% sample call
%   timeToHit = Prob6_B_5b(70, 45)

  figure, hold on
  g = 9.81;                % m/s/s
  firingAngle_rad = firingAngle*pi/180;   % radians
```

```
tHit = 2*(initSpeed/g)*sin(firingAngle_rad);  % s
t = linspace(0, tHit);  % s

height = initSpeed * sin(firingAngle_rad) * t  ...
          - 0.5*g*t.^2;  % m

heightShort_ndx = find( (height < 40) & (t < tHit/2) );
plot(t(heightShort_ndx), height(heightShort_ndx), '-m', ...
     'LineWidth', 3)

heightMid_ndx = find((40 <= height  &  height <= 80) & ...
                     (t < tHit/2) );
plot(t(heightMid_ndx), height(heightMid_ndx), ':k', ...
     'LineWidth', 3)

heightTall_ndx = find(80 < height);
plot(t(heightTall_ndx), height(heightTall_ndx), '--b', ...
     'LineWidth', 3)

heightMid_ndx = find((40 <= height  &  height <= 80) & ...
                     (t > tHit/2) );
plot(t(heightMid_ndx), height(heightMid_ndx), ':k', ...
     'LineWidth', 3)

heightShort_ndx = find( (height < 40) & (t > tHit/2) );
plot(t(heightShort_ndx), height(heightShort_ndx), '-m', ...
     'LineWidth', 3)

title('Height vs Flight Time for Projectile', 'FontSize', 14)
xlabel('flight time (sec)', 'FontWeight', 'bold')
ylabel('height (m)', 'FontWeight', 'bold')
legend('height < 40m', '40m <= height <= 80m', '80m < height')

equStr = texlabel('t_hit = 2(v_0/g)*sin(alpha)');
disp('place equation for tHit')
gtext(equStr, 'FontSize', 14)

equStr = texlabel('h(t) = v_0*sin(alpha)*t - (1/2)*g*t^2');
disp('place equation for height')
gtext(equStr, 'FontSize', 14)

str = sprintf(['initial speed (v_0) = %4.0f m/s\n', ...
               'firing angle (alpha) = %3.0f deg'], ...
               initSpeed, firingAngle);
str = texlabel(str);
disp('place parameters')
gtext(str, 'FontSize', 14, 'Color', 'blue')
```

Problem 6-B.6

The height **h** (m) of the water in a leaking tank as a function of the time **t** (sec) at which the leak started can be modeled with the following relation:

$$h(t) = h_0 e^{-Bt}$$

Parameter **B** depends on the geometry of the tank and the area of the opening, and the constant $\mathbf{h_0}$ is the initial (**t**=0) height of the water. **B** can be a function of **h**, but for this problem, we will consider only constant values of **B**.

Explore the effect of **B** on **h(t)** by following the directions below.

Write a function, **Prob6_B_6**, that creates an overlay family of plots of **h(t)** for varying values of **B** … **B=[0.1,0.2,0.4,0.8]**. Write your function in a general fashion so that it takes the following arguments:

```
Prob6_B_6(h0, bVector)
```

bVector is a vector holding four values of **B**.

Describe in a concise paragraph your observation of the effect of differing constant values of **B** on **h(t)**.

B depends on the area of the hole producing the leak and on the geometry of the tank. The dependency on the hole area is easy to see: The larger the hole, the faster the leak. But see if you can determine why **B** would depend on the solid geometry shape of the tank.

The MATLAB mechanics for this problem should not present an obstacle for students. However, understanding the link between the physical situation and the mathematical model is left open ended by ending the problem with the question as to why B should depend on the shape of the tank.

The following code meets the problem specifications.

```
function Prob6_B_6(initHeight, B_values)
% Models the decrease in liquid height in a tank when there
% is a leak. The modeling equation is
%    h(t) = initialHeight * exp(-B*t)
% where B is a constant.
% INPUTS: initialHeight, scalar, the starting liquid height
%         B_values, 4 element vector of values for B
%                   (4 is assumed - problem doesn't spec
%                   this)
```

```
% OUTPUT: none
% generates family of overlay plots, one plot for each B
% value
% sample call
%   B = [0.1,0.2,0.4,0.8];
%   h0 = 100;   % m
%   Prob6_B_6(h0, B)

  t = linspace(0,30);
  figure, hold on, grid on
  stanLineWidth = 2;

  h1 = initHeight * exp(-B_values(1)*t);
  plot(t,h1, '-k', 'LineWidth', stanLineWidth)

  h2 = initHeight * exp(-B_values(2)*t);
  plot(t,h2, ':b', 'LineWidth', stanLineWidth)

  h3 = initHeight * exp(-B_values(3)*t);
  plot(t,h3, '--m', 'LineWidth', stanLineWidth)

  h4 = initHeight * exp(-B_values(4)*t);
  plot(t,h4, '-.r', 'LineWidth', stanLineWidth)


titleStr = sprintf('Leaking Tank: Init Height=%4.1f', ...
                     initHeight);
  title(titleStr, 'FontSize', 14)
  disp('place equation for h(t)')
  gtext(texlabel('h(t) = h_0*e^(-B*t)'), ...
        'FontSize', 14, 'FontWeight', 'bold');

  xlabel('t (time)', 'FontWeight', 'bold');
  ylabel('h (height of liquid in tank', 'FontWeight', 'bold');

  legend(sprintf('B = %2.1f', B_values(1)), ...
         sprintf('B = %2.1f', B_values(2)), ...
         sprintf('B = %2.1f', B_values(3)), ...
         sprintf('B = %2.1f', B_values(4)))
```

Problem 6-B.7

The management team of the Edsel automotive company is planning to expand one of its plants by adding a new assembly line for sport utility vehicles (SUVs). The cost of setting up the new SUV assembly line is estimated to be $7 million. The cost of manufacturing (raw materials, labor, etc.) an SUV is $36,000 and the company is planning to sell each SUV for $38,500.

Industrial engineers of the company say that the new assembly line will be capable of manufacturing nine SUV units a day.

How many days will it take for the company to be at the break-even point?

Create a function, **Prob6_B_7**, to solve the problem graphically. Plot a cost (fixed cost plus manufacturing cost) and a revenue curve. Label the break-even point using **gtext**.

This problem (and the next) will seem daunting to students initially. However, the solution is really just an exercise in accounting. If students treat this systematically before trying to write MATLAB code, the solution will be straightforward.

The one point about implementation that students may need to be "nudged" on is the use of "ones" in developing a vector for the fixed cost.

A solution follows.

```
function daysToBreakEven = ...
          Prob6_B_7(fixedCost, perUnitVariableCost, ...
                    salePrice, prodPerDay)
% determine graphically the break even point for a new
% manufacturing line with known (a) fixed cost value for the
% new line, (b) per unit variable costs for manufacturing,
% and (c) the expected unit sales price
% INPUT: fixedCost, scalar, fixed costs
%        perUnitVariableCost, scalar, unit cost of
%                             manufacture, all variable costs
%        salePrice, scalar, expected sales price
%        prodPerDay, scalar, num of SUVs made per day
% OUTPUT: daysToBreakEven, scalar, num of days to break even
% produces a graph to find break even point
% sample call
%   fixedCost = 7 * 10^6;  % dollars
%   varCost = 36000;       % dollars/SUV
%   salePrice = 38500;     % dollars/SUV
%   production = 9;        % SUVs per day
%   Prob6_B_7(fixedCost, varCost, salePrice, production)

  numUnits = linspace(0, 5000);
  totalVariableCost = numUnits * perUnitVariableCost;
  fixedCostVector = fixedCost * ones(1, length(numUnits));
  totalManufacturingCost = totalVariableCost + fixedCostVector;
  grossSales =  numUnits * salePrice;
  profit = grossSales - totalManufacturingCost;
```

```
figure, hold on, grid on
plot(numUnits, totalManufacturingCost, '--r')
plot(numUnits, grossSales, '-b')
legend('total manufacturing costs', 'total revenue',...
       'location', 'northwest')

disp('click on the break even point')
breakEvenNumber = ginput(1);
daysToBreakEven = breakEvenNumber(1)/prodPerDay;
breakEvenStr = ...
    sprintf('break even num units = %4.0f\n in %4.1f days',...
            breakEvenNumber(1), daysToBreakEven);
disp('annotate - click to place')
gtext(breakEvenStr, 'FontSize', 12, 'FontWeight', 'bold')
title('Break-even analysis for New SUV Line', 'FontSize', 14)
xlabel('number of SUVs produced (at nine per day)', ...
       'FontWeight', 'bold')
ylabel('dollars', 'FontWeight', 'bold')
```

Problem 6-B.8

Solve the previous problem assuming that the unit price for an SUV drops as the quantity manufactured increases. The fixed and manufacturing costs remain the same, but the production quantity **Q** and the selling price **P** are related by the following equation:

$$Q(P) = 9500 - 0.10P$$

Write a function, **Prob6_B_8**, that solves the problem. In addition to revenue and cost curves, plot as well the profit curve and label the point of maximum profit. Your function should return the number of SUVs your new line should create to produce maximum profit..

What do you suppose is the underlying reason for the relation between **Q** and **P**?

The chief difference between this problem and the one previous is the functional relation between quantity of SUVs made and the selling price. While in the previous problem this relation was constant, in this problem, the relation is not constant. This version of Q(P) here is more realistic that the constant relation of the previous problem because as more SUVs are made, the market will be begin to saturate and the price of the SUVs will have to go down. At instructor discretion, this problem presents

```
an opportunity to discuss engineering markets from an economics
viewpoint.

A solution follows.


function daysToBreakEven = ...
          Prob6_B_8(fixedCost, perUnitVariableCost, prodPerDay)
% determine graphically the break even point for a new
% manufacturing line with known (a) fixed cost value for the
% new line, and (b) per unit variable costs for
% manufacturing. Assume also that the production quantity Q
% is a function of selling price P: Q(P) = 9500 - 0.10P.
% INPUT: fixedCost, scalar, fixed costs
%        perUnitVariableCost, scalar, unit cost of
%                             manufacture, all variable costs
%        prodPerDay, scalar, num of SUVs made per day
% OUTPUT: daysToBreakEven, scalar, num of days to break even
% produces a graph to find break even point
% sample call
%   fixedCost = 7 * 10^6;  % dollars
%   varCost = 36000;       % dollars/SUV
%   production = 9;        % SUVs per day
%   maxProfitDays = Prob6_B_8(fixedCost, varCost, production)

  numUnits = linspace(0, 6000);
  salePrice = (9500 - numUnits) / 0.10;
            % THE LINE ABOVE IS THE CRUX DIFFERENCE FROM
            % PROBLEM B.7

  totalVariableCost = numUnits * perUnitVariableCost;
  fixedCostVector = fixedCost * ones(1, length(numUnits));
  totalManufacturingCost = totalVariableCost + fixedCostVector;
  grossSales =  numUnits .* salePrice;
  profit = grossSales - totalManufacturingCost;

  figure, hold on, grid on
  plot(numUnits, totalManufacturingCost, '--r')
  plot(numUnits, grossSales, '-.b')
  plot(numUnits, profit, '-g', 'LineWidth', 3)
  legend('total manufacturing costs', ...
         'total revenue',...
         'total profit', ...
         'location', 'southwest')

  disp('click on the max profit point')
  maxProfitNumber = ginput(1);
  daysMaxProfit = maxProfitNumber(1)/prodPerDay;
  maxProfitStr = ...
   sprintf('max profit num units = %4.0f\n in %4.1f days',...
            maxProfitNumber(1), daysMaxProfit);
  disp('annotate - click to place')
  gtext(maxProfitStr, 'FontSize', 12, 'FontWeight', 'bold')
  title('Profit analysis for New SUV Line', 'FontSize', 14)
```

```
xlabel('number of SUVs produced (at nine per day)', ...
       'FontWeight', 'bold')
ylabel('dollars', 'FontWeight', 'bold')
```

Problem 6-B.9

You are the manager of the GSXR motorcycle group for Black Hawk Motorcycle Company. Under your direction, a new model, the 1000, has been designed with 24 prototypes built. The following data represent the average speeds of the GSXR1000 prototypes around the Willow Springs race track:

Motorcycle	1	2	3	4	5	6	7	8	9	10	11	12
Speed (mph)	120	83	134	76	94	135	126	104	88	67	94	101
Motorcycle	13	14	15	16	17	18	19	20	21	22	23	24
Speed (mph)	134	98	93	113	168	92	134	149	83	99	106	170

Suppose as manager of the GSXR group, you have been charged with producing the 1000 model to have a specified speed capability on courses like Willow Springs.

Write a function, **`Prob6_B_9`**, that inputs your desired average speed at Willow Spring (in mph) and a vector containing the average speeds of the 24 prototypes. This function will produce a plot in which the speeds greater than or equal to the spec speed are plotted with blue squares, and others are plotted as red triangles.

This problem is relatively straightforward. The computational core needed to solve the problem lies in performing the relational tests as indicated.

Code follows.

```
function Prob6_B_9(desiredSpeed, allSpeeds)
% Plot speeds of motorcycles differentiating between those
% above a desired speed and those below that desired speed
```

```
% INPUT: desiredSpeed, scalar, the desired speed (mph)
%        allSpeeds, vector, average speed for each
%                           motorcycle tested (mph)
% OUTPUT: none
% generates a graph, all above desired speed: blue squares
% all below desired speed: red triangles
% sample call
%     desiredSpeed = 100;   % mph
%     allSpeeds = [120 83 134 76 94 135 126 ...
%                  104 88 67 94 101 134 98 93 ...
%                  113 168 92 134 149 83 99 106 170];  % mph
%     Prob6_B_9(desiredSpeed, allSpeeds)

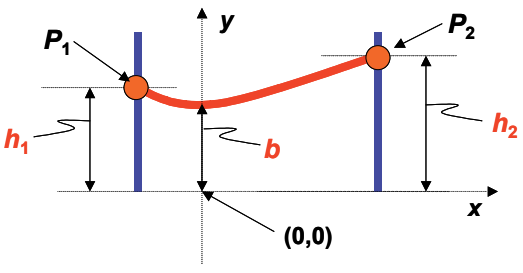
   figure, hold on, grid on

   speedsUnder_ndx = find(allSpeeds < desiredSpeed);
   speedsOver_ndx = find(allSpeeds >= desiredSpeed);

   plot(speedsUnder_ndx, allSpeeds(speedsUnder_ndx), ...
        'r^', 'MarkerSize', 10, ...
        'MarkerFaceColor', 'red')
   plot(speedsOver_ndx, allSpeeds(speedsOver_ndx), ...
        'bs', 'MarkerSize', 10, ...
        'MarkerFaceColor', 'blue')
   title('Average Speeds: Motorcycle Trials', 'FontSize', 14)
   xlabel('motorcycle identifier', 'FontWeight', 'bold')
   ylabel('average speed (mph)', 'FontWeight', 'bold')
   legendStr1 = sprintf('motorcycles < %3.0f mph average',...
                        desiredSpeed);
   legendStr2 = sprintf('motorcycles >= %3.0f mph average',...
                        desiredSpeed);
   legend(legendStr1, legendStr2, ...
          'location', 'north')
```

Problem 6-B.10

A catenary is a uniform cable, fixed at its two ends, which supports its own weight. Examine the sketch to the right. A catenary cable (shown in red) is attached at points $\mathbf{P_1}$ and $\mathbf{P_2}$. The attachment points are at a height $\mathbf{h_1}$ and $\mathbf{h_2}$ along an arbitrary *y*-axis. The *x*-*y* coordinates are arbitrary except that the *y*-axis must align with the down direction and the *x*-axis must have its zero point at an *x*-value corre-



sponding to the lowest point of the catenary cable. The lowest point of the catenary cable is at a *y*-value of **b**.

In the coordinate system described, the path of the catenary cable can be described as the following:

$$y(x) = b\cosh\left(\frac{x}{b}\right)$$

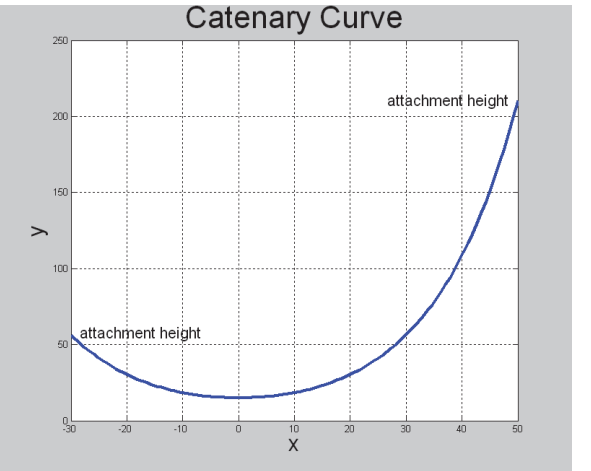
Write a function, **`Prob6_B_10`**, that inputs (a) the *x*-coordinate of $\mathbf{P_1}$, (b) the *x*-coordinate of $\mathbf{P_2}$, and (c) the value of **b**. Your function should return the heights of the two attachment points: the *y*-coordinate of $\mathbf{P_1}$ and the *y*-coordinate of $\mathbf{P_2}$. In addition, your function should create a plot of the catenary curve that the cable follows. Label the height of the two endpoints; use **`text`** for that labeling.

Test your function with the following:

```
P1_x = -30;   % m
P2_x = 50;    % m
b = 15;       % m
[P1_y, P2_y] = Prob6_B_10(-30, 50, 15)
```

The graph you generate should look similar to the one at the right.

Intuitively, when the two *x*-coordinate attachments points are equal in magnitude, what should be true about the y-coordinates of the two attachment points? Use your function to test your intuition.



This problem is straightfoward as long as the freshman learner gets a grasp of the physical situation.

Code follows.

```
function [P1_y, P2_y] = Prob6_B_10(P1_x, P2_x, b)
% Find the attachment points of a catenary curve with it two
% attachment points in the y-coordinate,  for a catenary
% defined by x-coordinate attachment points and a b that is
% the y-coordinate of the lowest part of the curve. The
% relation for y is ... y(x) = b*cosh(x/b)
% INPUT: P1_x, scalar, x-coordinate of one of the
%                      attachment points
%        P2_x, scalar, x-coordinate of the other attachment
%                      point
%        b, scalar, y-coordinate of lowest point in curve
% OUTPUT: P1_y, scalar, y-cooridnate of the first attachment
%                      point
%         P2_y, scalar, y-coordinate of the other attachment
%                      point
% creates a plot of the resultant catenary curve
% sample call
%     P1_x = -30;  % m
%     P2_x = 50;   % m
%     b = 15;      % m
%     [P1_y, P2_y] = Prob6_B_10(P1_x, P2_x, b)

      figure, hold on
      P1_y = b*cosh(P1_x/b);
      P2_y = b*cosh(P2_x/b);
      txtPt1Str = sprintf('height pt 1 = %3.1f meters',...
                          P1_y);
      text(P1_x+2,P1_y, txtPt1Str, ...
           'FontWeight', 'bold', ...
           'color', 'red')
      txtPt2Str = sprintf('height pt 2 = %3.1f meters',...
                          P2_y);
      text(P2_x-32,P2_y, txtPt2Str, ...
           'FontWeight', 'bold',...
           'color', 'red')

      x = linspace(P1_x, P2_x);
      y = b * cosh(x/b);

      plot(x,y, '-b', 'LineWidth', 3)
      grid on
      title('Catenary Curve', 'FontSize', 16)
      xlabel('x (meters)', 'FontWeight', 'bold')
      ylabel('F(x)  (meters)', 'FontWeight', 'bold')
```

```
disp('place equation for catenary curve')
gtext(texlabel(...
   sprintf('F(x) = b*cosh(x/b)\nb = %3.0f meters', b)), ...
    'FontWeight', 'bold', 'FontSize', 14)
```

Problem 6-B.11

Run the MATLAB demo **graf3d** and experiment with various plot types, shadings and color maps. Make sure you try each plot type using the drop down menu.

Now run the teapot demo (**teapotdemo**) and observe the capabilities of MATLAB in producing graphics. Also practice with the options on the menu, such as the 'Orbit Camera', 'Orbit Scene Light' and 'Toggle Scene Light'.

```
To run the demos, students need to type:
>> graf3d
and
>> teapotdemo

These are highly illustrative demos in various 3D plot types and
computer graphics capabilities of MATLAB.
```

Problem 6-B.12

Robots are used in various tasks that are difficult, unpleasant, or dangerous for humans. One such robot is built for the process of removal of rust on bridges, which produces chemicals dangerous for humans.

The rust-removing robot is programmed to follow a certain path on the trusses of the bridge. This path is defined as:

$$x = u$$

$$y = \sin(u)$$

$$z = \cos(u)$$

when $0 \le t < 8\pi$ seconds, and

$$x = \sin(u - \pi)$$

$$y = u - 8\pi$$

$$z = \cos(u)$$

when $8\pi \le t < 16\pi$ seconds.

Write a MATLAB function that takes no inputs and plots the path of the rust-removing robot for $0 \le t < 16\pi$ seconds. Use overlay plots in your function to show the entire motion on a a single graph. Put a legend to distinguish between the two intervals of time.

```
function Prob6_B_12
% generates the plot of the path for a rust-removing robot
% INPUTS: none
% OUTPUTS: none

u=linspace(0,8*pi);
x=u;
y=sin(u);
z=cos(u);

figure
plot3(x,y,z); hold

u=linspace(8*pi,16*pi);
x=sin(u-pi);
y=u-8*pi;
z=cos(u);
plot3(x,y,z,'r'); grid

xlabel('x','FontWeight', 'bold', 'FontSize', 14)
ylabel('y','FontWeight', 'bold', 'FontSize', 14)
zlabel('z','FontWeight', 'bold', 'FontSize', 14)
title('Path of the Rust-Removing Robot','FontSize', 16)

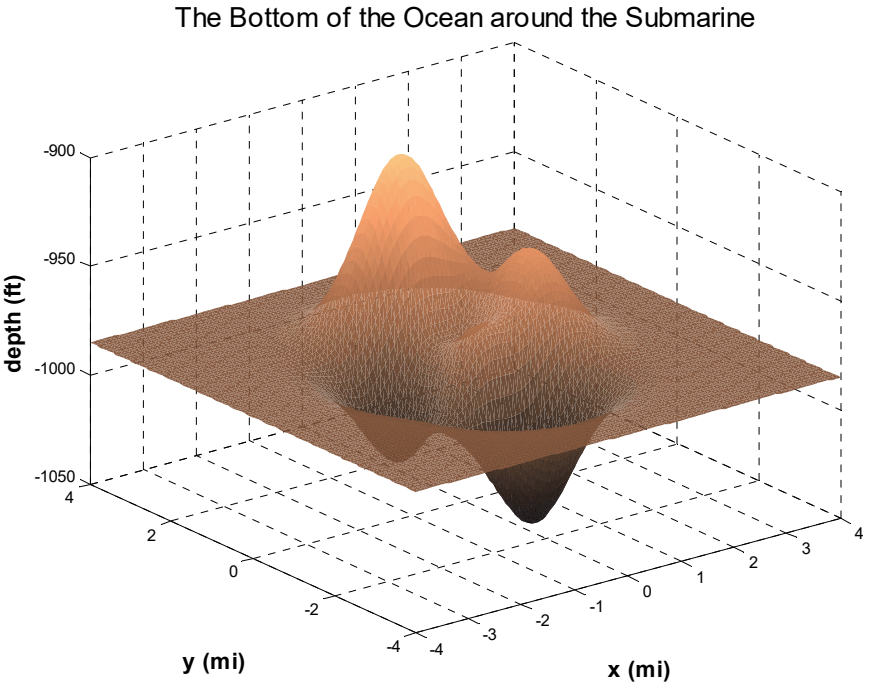
legend('Path from 0 to 8pi sec','Path from 8pi to 16pi sec', ...
     'Location', 'NorthEast')
```

Problem 6-B.13

A submarine crashed somewhere in the Pacific Ocean and a very secret government agency calls for your help. Geologists modeled the bottom of the ocean in the 8 x 8 mi area where the submarine is believed to have crashed. You are given this highly classified model:

$$z = 30e^{(-x^2-y^2-2y-1)} - 100(-x^3-y^5)e^{(-x^2-y^2)} - 985$$

where **x** and **y** are the coordinates in miles on the surface of the ocean, and **z** is the depth (feet) of the ocean at the coordinates. The bottom of the ocean looks like this in 3D coordinates:



Radio communication with the submarine revealed that the submarine is exactly at the deepest point in this area (GPRS is not working). Your mission is to find out where exactly and at what depth the submarine lies in this area.

Write a function called **`Prob6_B_13`** that inputs the range vectors for **x** and **y** (both [-4,4] in this case) and produces a 3D surface plot. Visually determine the location of the submarine.

```
This problem is actually an optimization (or minimization) prob-
lem that is disguised under fun. The point of the problem is to
show that visual approximations can be done to find a solution
to some seemingly difficult problems.

A visual investigation reveals that the submarine is in the
vicinity of (0,-1.8 mi, -1048 ft) coordinates. The actual
coordinates are (0, -1.6632 mi, -1045.7 ft). Finding an exact
solution with MATLAB's numeric optimization tools is also
included below.

function Prob6_B_13(xint,yint)
% plots the ocean surface wrt given model
% INPUTS: vector range for x
%         vector range for y
% OUTPUTS: none
% Sample call:
% Prob6_B_13([-4,4],[-4,4])

x=linspace(xint(1),xint(2));
y=linspace(yint(1),yint(2));
[x,y]=meshgrid(x,y);
z=30*exp(-x.^2-y.^2-2*y-1)-100*(-x.^3-y.^5).*...
    exp(-x.^2-y.^2)-985;

surf(x,y,z), shading flat

xlabel('x (mi)','FontWeight', 'bold', 'FontSize', 14)
ylabel('y (mi)','FontWeight', 'bold', 'FontSize', 14)
zlabel('depth (ft)','FontWeight', 'bold', 'FontSize', 14)
title('The Bottom of the Ocean around the Submarine',
'FontSize', 16)
```

```
% The numerical solution to Problem 6-B.13

clear

submarine = @(x)30*exp(-x(1).^2-x(2).^2-2*x(2)-1)-...
    100*(-x(1).^3-x(2).^5).*exp(-x(1).^2-x(2).^2)-985;

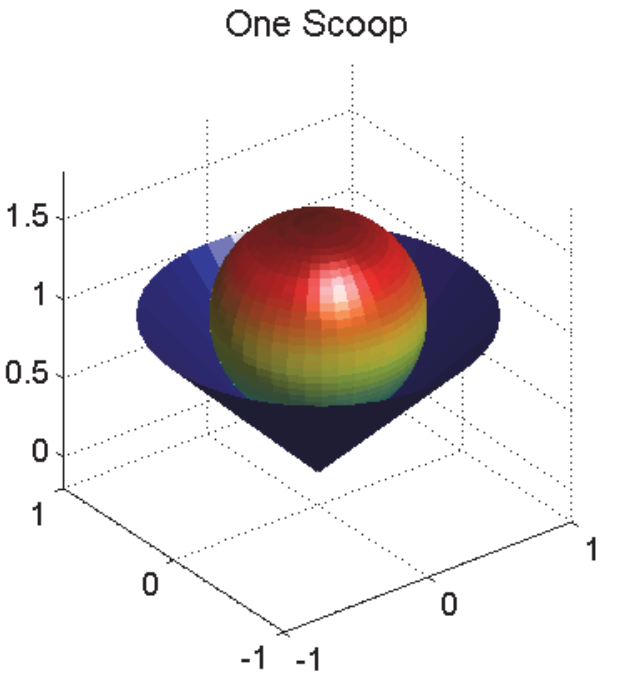
x = fminsearch(submarine,[2,-4])

% Note that according to the initialization point,
% fminsearch can be stuck at a local minimum.
```

Problem 6-B.14

Use MATLAB's 3D graphing tools to plot a sphere in a cone. Your figure should look like the sample shown. To make this plot, open the editor and start your code with a **`close all`** command. Now experiment with the **`cylinder`** command to produce a cone. Next, switch the **`hold`** on your figure, and try different configurations of **`sphere`** or **`ellipsoid`** commands to place the ball into the cone. When you are done, switch the shading to flat. See the Help document for **`shading`** to do that.

When done, turn on the camera toolbar with the **`cameratoolbar`** command. Now you have more options to apply on your figure. Experiment with 'Orbit Camera' and 'Orbit Scene Light' switches. Click on Insert/Light to add a new light. Try different configurations for the place, type, and the color of the light.




```
% Script file for Problem 6-B.14
% Sphere in a cone

clear
```

```
figure

[x,y,z]=cylinder([0,1],40);
surf(x,y,z), hold

ellipsoid(0,0,1,.6,.6,.6,40);

shading flat, colormap default
set(gca,'FontSize',14)
title('One Scoop', 'FontSize', 16)
cameratoolbar
```

Problem 6-B.15

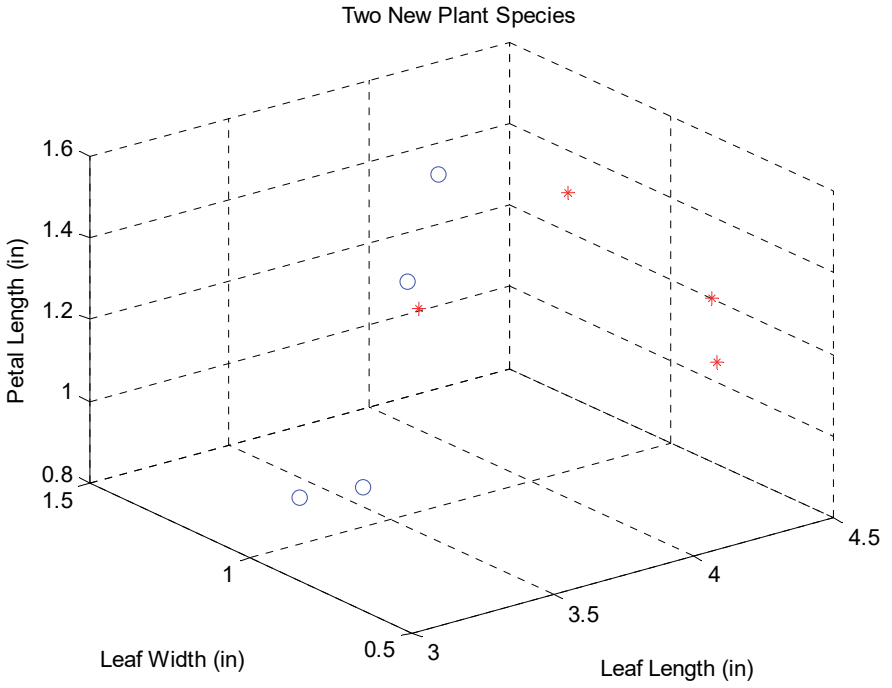
A plant scientist goes out to expedition in a remote island and discovers two new plant species. She takes measurements on the floral leaf length, floral leaf width, and petal length of the flowers of the plants. Now she needs your help to visualize the data and understand the nature of these new species.

She gives you the following sample to be analyzed:

Plant Type	Leaf Length (in)	Leaf Width (in)	Petal Length (in)
Species 1	4.2	0.6	1.2
Species 2	3.4	1.2	0.8
Species 2	3.9	1.2	1.5
Species 1	3.9	0.8	1.6
Species 2	3.9	1.3	1.2
Species 1	3.6	1.0	1.3
Species 1	4.3	0.7	1.3
Species 2	3.4	1.0	0.9

Write a script file to plot the leaf length, leaf width and petal length for the new plant species 1 and 2. Use different colored markers for data points for the two species of plants. Rotate the axis in 3D to visualize the data from varying angles.

Make sure your graph is appropriately labeled. Your graph should look like the one below when completed.




```
% Script file for Problem 6-B.15
% Two new plant species

clear
data=[1 4.20.61.2;
    2 3.4 1.2 0.8;
    2 3.9 1.2 1.5;
    1 3.9 0.8 1.6;
    2 3.9 1.3 1.2;
    1 3.6 1.0 1.3;
    1 4.3 0.7 1.3;
    2 3.4 1.0 0.9];

firstSp = find(data(:,1)==1);
secondSp = find(data(:,1)==2);

figure
plot3(data(firstSp,2),data(firstSp,3),data(firstSp,4),'r*');

hold on, grid on

plot3(data(secondSp,2),data(secondSp,3),data(secondSp,4),'bo');
```

```
xlabel('Leaf Length (in)','FontWeight', 'bold', 'FontSize', 12)
ylabel('Leaf Width (in)','FontWeight', 'bold', 'FontSize', 12)
zlabel('Petal Length (in)','FontWeight', 'bold', 'FontSize', 12)
title('Two New Plant Species', 'FontSize', 14)
```

Problem 6-B.16

MATLAB offers you effective ways to discover functions that represent the behavior of experimental data. One such tool is the Fit Editor.

Suppose you collected the following data on the stopping distance of a car with respect to its speed:

Table 6-2: Stopping distances for a vehicle

Speed (mi/h)	Stopping Distance (ft)
10.0	26.9
20.0	66.5
30.0	123.9
40.0	200.0
50.0	273.2
60.0	372.5
70.0	471.9

Write a MATLAB function **`Prob6_B_16`** that takes no inputs and outputs the 7x1 vector of stopping distances. Your function should also plot the data using markers only (do not connect the data points). In the function, fully label your plot and put a grid on the figure. Run your function.

Once you have the figure, we can start experimenting with the Fit Editor. Click on Tools / Basic Fitting on your Figure window. This will bring up the Basic Fitting tool.

On the Basic Fitting window, find the ‘Show Equations’ checkbox and put a check in it. This will show the equation of any discovered function on your figure. Leave significant digits as 2.

Now select ‘Plot Residuals’. Select the Bar Plot box, and switch it to ‘Line Plot’. Residuals are the differences between your discovered function and the actual data points. The goal of function discovery is not only to minimize the

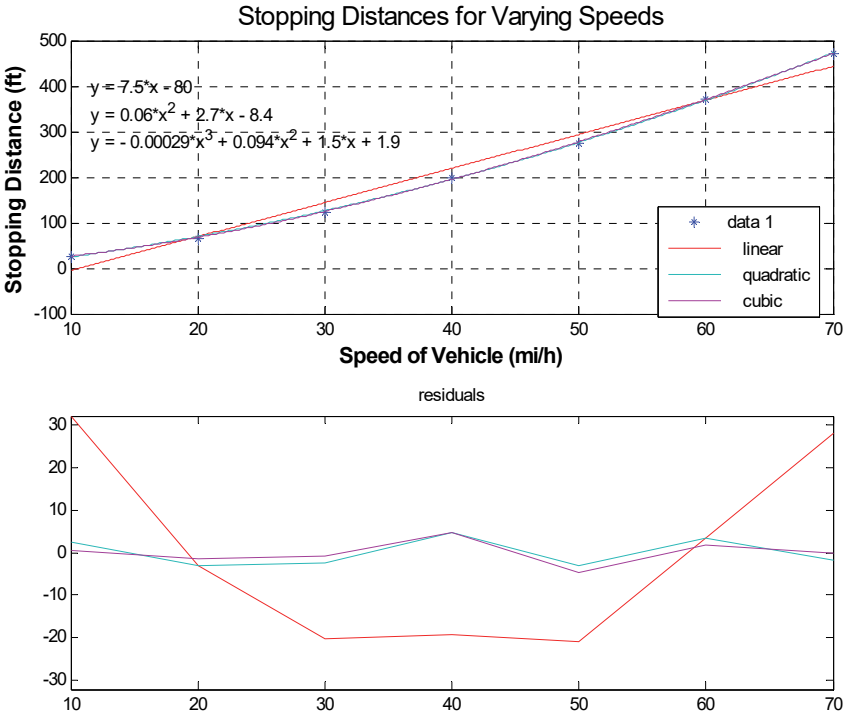
residuals, but also to keep the residual distribution ‘stable’, i.e. not having a trend to decrease or increase.

We are ready to discover our first function that ‘hopefully’ represents the dataset. The simpler the function (the model for the data) the better for us, so start with a linear function. Remember, linear functions are the functions of type **`y=ax+b`**. Look at the residuals. Do they seem to be stable?

Let us now try a quadratic polynomial. These are second degree polynomials. Select the quadratic polynomial and observe the residuals. Is the behavior of the residuals better now?

Also try a cubic polynomial. You will observe that there is not much change in the trend of the residuals. But how about the complexity of the model we are using? Now we have a third-degree polynomial, which does not substantially outperform a second-degree polynomial. Which model should we choose?

At this point your figure should look like the following:



```
This problem is meant to be an introduction to function discov-
ery. The problem statement briefly introduces the concepts of
function discovery, different types of functions, residuals, and
Occam's Razor principle. The students are asked to think which
function is the best fit for this data set.

function stopDist = Prob6_B_16()
% plots and returns the stopping distances for vehicles
% INPUTS: none
% OUTPUT: column vector of stopping distances

dataSet = [10.026.9
    20.066.5
    30.0123.9
    40.0200.0
    50.0273.2
    60.0372.5
    70.0471.9];

figure
plot(dataSet(:,1),dataSet(:,2),'*')
grid on

xlabel('Speed of Vehicle (mi/h)','FontWeight', 'bold', ...
     'FontSize', 12)
ylabel('Stopping Distance (ft)','FontWeight', 'bold', ...
     'FontSize', 12)
title('Stopping Distances for Varying Speeds', 'FontSize', 14)

stopDist = dataSet(:,2);
```

Problem 6-B.17

Reconsider the expression we found in **Problem 6-B.16** for the stopping distance of a vehicle. The best approximating function was:

$$y = 0.06x^2 + 2.7x - 8.4$$

In this equation, **x** is the speed in mi/h and **y** is the stopping distance in feet. The actual physical relationship for stopping distance of a speeding vehicle is

$$y = \frac{v^2}{2a}$$

where **a** is the deceleration of the vehicle. So, where are the other terms coming from? The constant term, -8.4 is very small compared to the stopping distances we measured so can be ignored as the measurement error, but the **2.7x** term can be as high as 189 ft when the speed **x** is 70 m/h. Can you think of a reason we may have such a 'delaying' effect on the deceleration of the vehicle?

If you said 'human reaction time', you were right. Now let us use our findings for scientific analysis. Consider the scenario, you are driving home after a late night project meeting. You are so tired that your reaction constant (which we found as 2.7 for normal people) doubled. Dropping the constant term -8.4 in our model, what would be your stopping distance if you suddenly saw an obstacle at 10 mph? What about 70 mph?

Use the model we found and the new reaction constant to re-tabulate the stopping distances for a tired driver. To solve this problem, write a MATLAB function **Prob6_B_17** that takes no inputs and outputs the 7x1 vector of stopping distances.

Table 6-3: Stopping distances for a tired driver

Speed (mi/h)	Stopping Distance (ft)
10.0	
20.0	
30.0	
40.0	
50.0	
60.0	
70.0	

```
function stopDist = Prob6_B_17()
% plots and returns the stopping distances for vehicles
% using the 'tired driver' model.
% INPUTS: none
% OUTPUT: column vector of stopping distances

newReactionConst = 2*2.7; % doubled reaction time

speeds = [10:10:70]';
stopDist = 0.06*speeds.^2 + newReactionConst*speeds;
```

Table 6-4: Stopping distances for a tired driver

Speed (mi/h)	Stopping Distance (ft)
10.0	60
20.0	132
30.0	216
40.0	312
50.0	420
60.0	540
70.0	672

Problem 6-B.18

Another very useful plotting tool in MATLAB is the Figure Palette. Before we start using this tool, run the functions you wrote for problems 6-B.16 and 6-B.17 to retrieve the stopping distances for alert and tired drivers in vectors named **`stopAlert`** and **`stopTired`**.

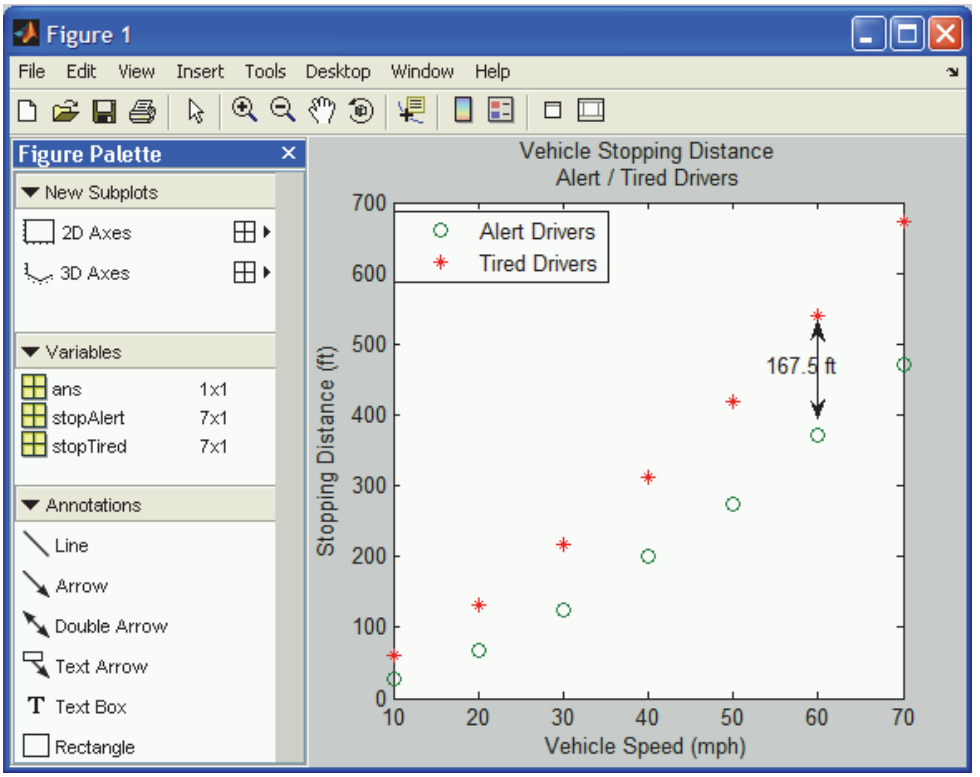
Now start Figure Palette by typing

```
>> figurepalette
```

This will bring up a fully interactive graphing tool. Create a new subplot by clicking on '2D Axes'. Now right click on the axes and select 'Add Data'. Choose the 'Plot Type' as scatter. Specify the X Data Source as [10:10:70], and Y Data Source as **`stopAlert`**. Redo this operation to add the **`stopTired`** data in an overlay plot.

Right click again on the axes and click on Properties. Put labels on the x and y axes and add a title to your figure. Use the Annotations pane to emphasize the difference between alert and tired drivers. Explore the tools to find how to put a legend on the figure.

When you are done, your figure should look as below:



Problem 6-B.19

The World Wildlife Association needs to know if there is a possibility of Anatolian Van Cats, the only domestic cat species that love water and even swim, becoming extinct. They made recordings of the population of this species in Anatolia every decade.

Table 6-5:

Year	1810	1820	1830	1840	1850	1860	1870	1880	1890	1900
Population	22000	20000	19000	17000	15000	14000	12000	12000	11000	9000
Year	1910	1920	1930	1940	1950	1960	1970	1980	1990	2000
Population	9000	8000	7000	7500	6000	6500	5000	4500	4500	4400

Plot the data using MATLAB and find the best function fit using the Fit Editor. Using the function you found, predict the populations in year 1800, 1845 and 1910. Are your results realistic?

If you are getting way off results, increase the significant digits shown on the Fit Editor.

```
% Script file for Problem 6-B.19
% Population of Anatolian Van Cats vs. decades

clear
van = [1810 1820 1830 1840 1850 1860 1870 1880 1890 1900 ...
    1910 1920 1930 1940 1950 1960 1970 1980 1990 2000;
    22000 20000 19000 17000 15000 14000 12000 12000 11000 9000
...
    9000 8000 7000 7500 6000 6500 5000 4500 4500 4400];

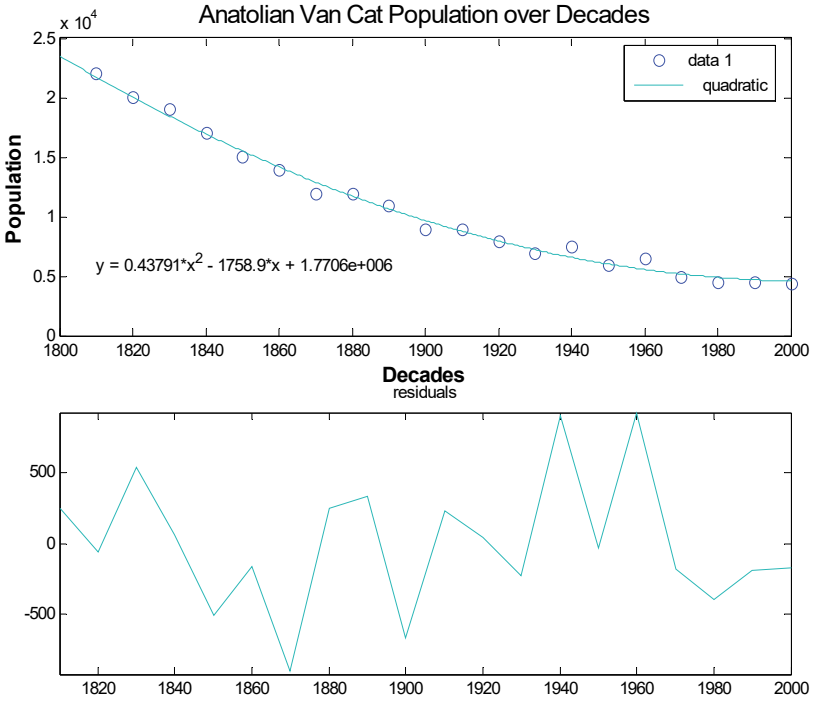
figure

plot(van(1,:),van(2,:),'o');

xlabel('Decades','FontWeight', 'bold', 'FontSize', 12)
ylabel('Population','FontWeight', 'bold', 'FontSize', 12)
title('Anatolian Van Cat Population over Decades', 'FontSize',
14)

hold
ezplot('0.43791*x^2 - 1758.9*x + 1.7706e+006',[1800,2000])

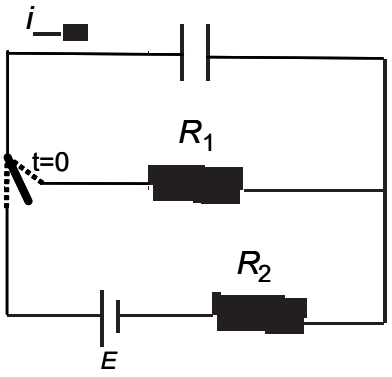
x = 1800;
pop1800 = 0.43791*x^2 - 1758.9*x + 1.7706e+006
x = 1845;
pop1845 = 0.43791*x^2 - 1758.9*x + 1.7706e+006
x = 2010;
pop2110 = 0.43791*x^2 - 1758.9*x + 1.7706e+006
x = 2050;
pop2050 = 0.43791*x^2 - 1758.9*x + 1.7706e+006
```




```
We find the populations in years 1800, 1845, 2110, 2050
approximately as 23400, 16100, 44100, 5200. The interpolated
1845 population is realistic, and possibly so are the
extrapolated 1800 and 2010 populations. On the other hand, 2050
prediction seems to be far from the reality. This is a good
chance to discuss the nature and risk of extrapolation analysis
with students.
```

Problem 6-B.20

An electric circuit has a resistor and a capacitor. The capacitor is charged to 0.2 Volts, and when time starts the switch is thrown to the position shown.



The electric charge at the capacitor starts to dissipate in time. The voltage at the capacitor is measured at fixed time intervals and the following data is obtained:

Table 6-6:

time (msec)	10	20	30	40	50	60	70	80	90	100
Voltage (V)	1.988	1.247	0.763	0.413	0.254	0.142	0.083	0.057	0.042	0.039

Plot the data using data markers and put a grid on your figure. Fully label the axes and put a title to your plot. Find a function that represents the behavior of the voltage drop.

Sidebar... If you are wondering how to discover a representation for the data with an exponential function, consider the following. Exponential functions are in the form $y = be^{mx}$. Taking the logarithm of both sides, we find $\log(y) = \log(b) + mx$. This implies, if we plot x vs. the logarithm of the data, we will obtain a linear behavior. Thus, we can discover a linear function and take the exponent of the constant term to discover the parameters of the exponential function.

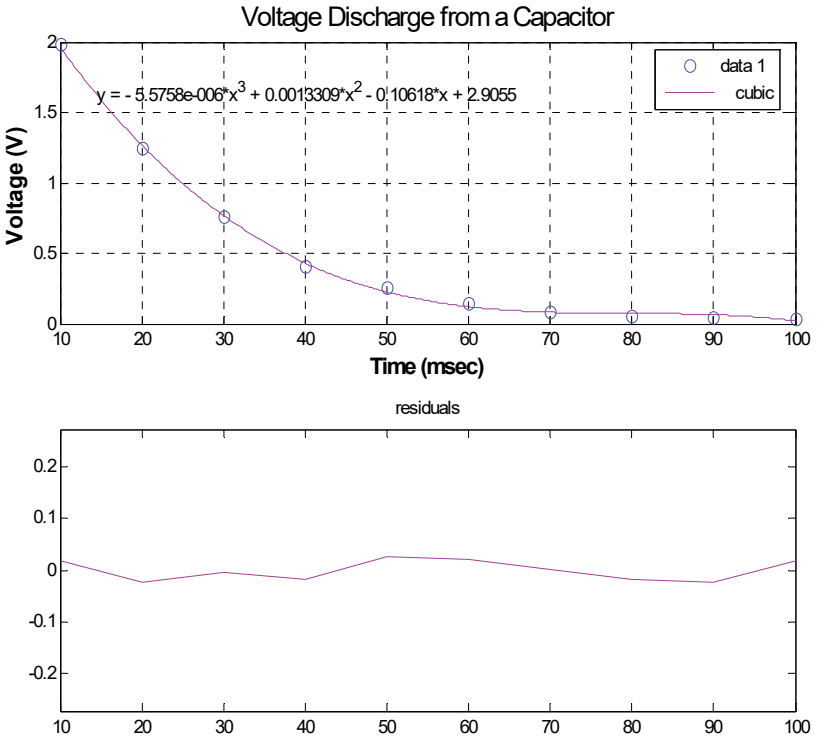
```
% Script file for Problem 6-B.20
% Voltage drop across a capacitor

clear
data = [10:10:100;
    1.988 1.247 0.763 0.413 0.254 0.1420.083 0.057 0.042 0.039];

figure
```

```
plot(data(1,:),data(2,:),'o');
grid on

xlabel('Time (msec)','FontWeight', 'bold', 'FontSize', 12)
ylabel('Voltage (V)','FontWeight', 'bold', 'FontSize', 12)
title('Voltage Discharge from a Capacitor', 'FontSize', 14)
```



SOLUTIONS FOR CHAPTER 7
ARRAYS

If students learn the lesson that most of what they need to know about arrays is a generalization of what they know about vectors, the problems of this chapter will seem much more straightforward to them. The exceptions lie mainly in the matrix operations: matrix multiply and matrix left divide in particular.

Many freshman students in particular will need to take care to (re-)learn the basics of matrix multiplication.

Set A: Nuts and Bolts Problems for Arrays

Problem 7-A.1

Write a function, **`Prob7_A_1`**, of no inputs, and one array output. Internally, your function should construct the following array using the square bracket operator and should output the array.

$$\begin{bmatrix} 1 & 4 & 3 & 7 & 2 \\ 6 & 8 & 9 & 5 & 1 \end{bmatrix}$$

```
function anArray = Prob7_A_1
% creates and returns the array:
% [1 4 3 7 2
%  6 8 9 5 1]
% INPUTS: None
% OUTPUT: An array

anArray = [1 4 3 7 2; 6 8 9 5 1];
```

Problem 7-A.2

Write a function, **Prob1**, of no inputs and one array output. Internally, your function should construct the following array using the square bracket operator and should output the array.

$$\begin{bmatrix} 8 & 6 \\ 7 & 8 \\ 9 & 1 \\ 6 & 3 \\ 5 & 1 \end{bmatrix}$$

```
function anArray = Prob1  % creates and returns the array:
                          % [8 6

%  7 8
%  9 1
%  6 3
%  5 1]
% INPUTS: None
% OUTPUT: An array

anArray = [8 6; 7 8; 9 1; 6 3; 5 1];

% or...
% anArray = [8 7 9 6 5; 6 8 1 3 1]';
% Note the Transpose operator
```

Problem 7-A.3

Write a function, **Prob2**, of no inputs and two array outputs. Internally, your function should construct the following two arrays using the square bracket operator, the colon operator (or alternative to the colon operator, the **linspace** function), and the transpose operator. The two arrays constructed should be your function's outputs.

$$arrayA = \begin{bmatrix} 1 & 11 & 21 & 31 & 41 & 51 & 61 & 71 \\ 80 & 75 & 70 & 65 & 60 & 55 & 50 & 45 \\ 3 & 6 & 9 & 12 & 15 & 18 & 21 & 24 \\ 99 & 89 & 79 & 69 & 59 & 49 & 39 & 29 \end{bmatrix}, \quad arrayB = \begin{bmatrix} 1 & 80 & 3 & 99 \\ 11 & 75 & 6 & 89 \\ 21 & 70 & 9 & 79 \\ 31 & 65 & 12 & 69 \\ 41 & 60 & 15 & 59 \\ 51 & 55 & 18 & 49 \\ 61 & 50 & 21 & 39 \\ 71 & 45 & 24 & 29 \end{bmatrix}$$

Hint: Do you notice any patterns in the rows of **`arrayA`**? For example, the first row of **`arrayA`** could be created by **`1:10:71`** or **`linspace(1,71,8)`**. Once you figure out how to create each row, you can concatenate them.

```
function [arrayA, arrayB] = Prob2
% creates the two arrays given in the problem definition
% INPUTS: None
% OUTPUTS: arrayA and arrayB given in Problem 2

arrayA = ...
        [1:10:71; linspace(80,45,8); 3:3:24; linspace(99,29,8)];
arrayB = arrayA';
```

Problem 7-A.4

Write a function, **`Prob7_A_4`**, of no inputs and one array output. Internally, your function should construct the following array and should output it.

$$\begin{bmatrix} 7 & 6 & 44 & -9 & 61 \\ 14 & 4 & 55 & -5 & 36 \\ 21 & 2 & 66 & -1 & 11 \\ 28 & 0 & 77 & 3 & -14 \\ 35 & -2 & 88 & 7 & -39 \\ 42 & -4 & 99 & 11 & -64 \end{bmatrix}$$

```
function anArray = Prob7_A_4
% creates the array given in the problem definition
```

```
% INPUTS: None
% OUTPUT: the array given in Problem 7-A.4

anArray = [7:7:42; 6:-2:-4; 44:11:99; -9:4:11; 61:-25:-64]';
```

Problem 7-A.5

Write a function, **Prob3**, of no inputs and one array output. Internally, your function should construct the following array and should output it.

$$\begin{bmatrix} 0 & 0 & 0 \\ 8 & 8 & 8 \\ -32 & -32 & -32 \end{bmatrix}$$

Hint: To create the second row, we could use **8*ones(1,3)**.

```
function anArray = Prob3
 % creates the array:
% [0    0   0
%  8    8   8
% -32 -32 -32]
% INPUTS: None
% OUTPUT: An array

anArray = [zeros(1,3); 8*ones(1,3); -32*ones(1,3)];
```

Problem 7-A.6

Write a function, **Prob7_A_6**, of no inputs and one array output. Internally, your function should construct the following array and should output it.

$$\begin{bmatrix} 4 & 4 & 4 & 4 & 0 & 0 & 0 & 0 \\ 6 & 4 & 2 & 0 & 41 & 51 & 61 & 71 \\ -5 & -10 & -15 & 16 & 18 & 20 & 22 & 24 \\ 7 & 14 & 21 & 28 & 21 & 14 & 7 & 0 \end{bmatrix}$$

```
We could approach this problem as we did the previous ones, i.e.
create the entire array in a single line. However, as the prob-
lems get more complicated, it is safer to approach the problem
in a systematic manner. In this case, we should first observe
that this array consists of 4 regions, which could be created
separately and concatenated using the comma and semicolon oper-
ators.

function anArray = Prob7_A_6
% creates the array given in the problem definition
% INPUTS: None
% OUTPUT: the array given in Problem 7-A.6

tempArrayUL = [4*ones(1,4); 6:-2:0];     % The Upper Left region
tempArrayUR = [zeros(1,4); 41:10:71];    % The Upper Right region
tempArrayLR = [-5:-5:-15; 7:7:21];       % The Lower Right region
tempArrayLL = [16:2:24; 28:-7:0];        % The Lower Left region

anArray = [tempArrayUL, tempArrayUR; tempArrayLR, tempArrayLL];
```

Problem 4

Write a function, **Prob4**, that takes one array input and produces one array output. Internally, your function should construct the single output array with elements taken from the input array, specifically from the second and third rows and from the first three columns of the input array. Assume that the input array has dimensions n x m such that n>3, and m>3.

What you are doing is pulling out a number of specified elements for construction of the output from the input.

```
function selectRegion = Prob4(anArray)
% returns an array that consists of the 2nd and 3rd rows
% in the first 3 columns of the input array
% INPUT: An n by m array where n and m >=3
% OUTPUT: A 2 by 3 array

selectRegion = anArray(2:3,1:3);
```

Problem 7-A.8

Write a function, **Prob5**, that takes one array input and produces one array output. Internally, your function should construct the single output array with elements taken from the input array, specifically from the third and fourth rows and from the last five columns of the input array. Assume that the input array has dimensions n x m such that n>4, and m>5. In your indexing solution, use **end**.

```
function selectRegion = Prob5(anArray)
% returns an array that consists of the 3rd and 4th rows
% in the last 5 columns of the input array
% INPUT: An n by m array where n >= 4 and m >= 5
% OUTPUT: A 2 by 5 array

selectRegion = anArray(3:4,end-4:end);
```

Problem 7-A.9

Write a function, **Prob7_A_9**, that takes one array input and produces one array output. Internally, your function should set the last four elements in the first row of the input to 2 and return the modified array. Assume that the input array has dimensions n x m such that m>3.

```
function anArray = Prob7_A_9(anArray)
% sets the last four elements in the first row of the input
% array to 2
% INPUT: An n by m array where m >= 4
% OUTPUT: The modified n by m array

anArray(1,end-3:end)=2;
```

Problem 7-A.10

Write a function, **Prob6**, that takes one array input and produces one array output. Internally, your function should set the four corners of the input array to 99 and return the modified array.

For example, if the input array has dimensions 2 x 3, then the modified array would be such that elements (1,1), (2,1), (1,3), and (2,3) would all have the value 99.

Starters may have difficulty following the second solution proposed here. In the text, we preferred to stick to one type of indexing operation, i.e. row-column pairs. However, it is also possible to do indexing into arrays with scalar addresses. This addressing scheme starts with the first column and counts every cell in sequence from top to bottom, left to right. Practice with the solution provided below to get a better grasp of this kind of indexing.

```
function anArray = Prob6(anArray)
% sets the corners of the input array to 99
% INPUT: An n by m array
% OUTPUT: The modified n by m array

anArray(1,1)=99;
anArray(1,end)=99;
anArray(end,1)=99;
anArray(end,end)=99;

% or...
% [n,m] = size(anArray);
% anArray([1,n,n*(m-1)+1,n*m])=99;
```

Problem 7-A.11

Write a function, **Prob7_A_11**, that takes one array input and produces one vector output. Internally, your function should pull out the last row of the input array and return it as the vector output.

Test your function with the following inputs:

[5] (note that the last row of scalar **5** is the scalar **5**)

```
[1 2 3 4 5]

[1 2 3 4 5]'

[1 2 3 4 5; 5 4 3 2 1]
```

```
function lastRow = Prob7_A_11(anArray)
% returns the last row of the input array as vector
% INPUT: An n by m array
% OUTPUT: A 1 by m vector

lastRow = anArray(end,:);
```

Problem 7-A.12

Write a function, **Prob7**, that takes one array input and produces one array output. Internally, your function should reverse the order of elements in the second column of the input and output the resultant modified input array. Assume that the input array has dimensions n x m such that m>1.

Hint: A vector **V** can be reversed by **V=V(end:-1:1)**.

```
function anArray = Prob7(anArray)
% reverses the order of elements in the 2nd column
% of the input array
% INPUT: An n by m array
% OUTPUT: An n by m array

anArray(:,2) = anArray(end:-1:1,2);
```

Problem 7-A.13

For the two matrices below, find **A*B** by hand. Verify your answer using MATLAB.

$$A = \begin{bmatrix} 1 & 2 & 0 \\ 2 & 1 & 3 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 1 \\ 0 & 1 \\ 1 & 4 \end{bmatrix}$$

(Do the operation by hand. If you do not understand the concept of matrix multiplication, you are not going to understand when to apply it later in complex word problems.)

First we observe the dimensions of arrays A and B. A is 2x3 and B is 3x2, so the matrix multiplication C = A*B is legal and the product C is 2x2.
To calculate the first row first column element in C, we multiply the first row in A with the first column in B and add the products:
C(1,1) = 1*2+2*0+0*1 = 2

To calculate the first row second column element in C, we multiply the first row in A with the second column in B and add the products:
C(1,2) = 1*1+2*1+0*4 = 3

To calculate the second row first column element in C, we multiply the second row in A with the first column in B and add the products:
C(2,1) = 2*2+1*0+3*1 = 7

To calculate the second row second column element in C, we multiply the second row in A with the second column in B and add the products:
C(2,1) = 2*1+1*1+3*4 = 15

So we find C as:

$$C = \begin{bmatrix} 2 & 3 \\ 7 & 15 \end{bmatrix}$$

Problem 7-A.14

Fill out the table to the right with a Yes or a No in the column marked Legal? indicating whether a matrix multiplication is legal for arrays of the stated dimensions.

In the column marked C Dimensions, fill in the dimensions of the array resulting from the matrix multiplication if the operation is legal.

Is **C=A*B** legal? If the operation is legal, then what are the dimesions of **C**?

# Rows in A	# Cols in A	# Rows in B	# Cols in B	Legal?	C Dimensions
2	**3**	**3**	**3**	Yes	2x3
3	**3**	**2**	**2**	No	
2	**3**	**2**	**3**	No	
2	**3**	**3**	**1**	Yes	2x1
10	**12**	**12**	**2**	Yes	10x2

Problem 7-A.15

For the matrices **A** and **B** below, does **A*B** give the same result as **B*A**?

$$A = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 2 & 0 \\ 1 & 3 \end{bmatrix}$$

Work out your answer by hand first and then verify your result using MATLAB.

No, they do not give the same results.

A*B evaluates to $\begin{bmatrix} 5 & 9 \\ 8 & 12 \end{bmatrix}$ **whereas B*A is** $\begin{bmatrix} 2 & 6 \\ 7 & 15 \end{bmatrix}$.

```
The point of this exercise is to show that matrix multiplication
is not commutative.
```

Problem 7-A.16

Write a function, **Prob7_A_16**, that takes no input and returns a vector solving the following system of equations using matrix operations.

$$\begin{aligned} 2x + 5y + z &= 12 \\ x + 2y - 3z &= 5 \\ x - y + z &= -1 \end{aligned}$$

```
function solnVector = Prob7_A_16
% Solves the system of equations:
% 2x+5y+z=12
% x+2y-3z=5
% x-y+z=-1
% INPUTS: None
% OUTPUT: A column vector of [x; y; z]

A = [2 5 1;1 2 -3; 1 -1 1]; B = [12; 5; -1];
solnVector = A\B;
```

Problem 7-A.17

Write a function, **Prob7_A_17**, that takes no input and returns a vector solving the following system of equations using matrix operations.

$$\begin{aligned} x + y - z &= 6 \\ 2x - y + 2z &= 3 \\ 3x + 2y + 5z &= 35 \end{aligned}$$

```
function solnVector = Prob7_A_17
% Solves the system of equations:
% x+y-z=6
% 2x-y+2z=3
% 3x+2y+5z=35
% INPUTS: None
% OUTPUT: A column vector of [x; y; z]

A = [1 1 -1; 2 -1 2; 3 2 5]; B = [6; 3; 35];
solnVector = A\B;
```

Problem 7-A.18

Write a function, **Prob7_A_18**, that solves a set of linear equations. Inputs to your function should be (a) a vector of the right-hand side values for system of linear equations and (b) an array of coefficients for the set of equations. The output should be a column vector with the solution for the unknowns in the system.

```
function solnVector = Prob7_A_18(V, coeff)
% solves linear system of equations
% INPUTS: A vector of the right hand-side values V
%         An array of system coefficients coeff
% such that:
%         coeff*X = V
% OUTPUT: The solution in a column vector

solnVector = coeff\V;
```

Problem 7-A.19

Write a function, **Prob7_A_19**, that inputs one array, and produces the following output:

(a) An array containing the *locations* of elements in the input that are less than zero

(b) A corresponding column vector containing the *values* of elements in the input that are less than zero

Use **find** in your solution.

For example, if the input array is:

```
>> A = [3 -5 9; -22 8 7; -4 10 -11]
A =
     3    -5     9
   -22     8     7
    -4    10   -11
```

A call to **Prob7_A_19(A)** should produce the following:

```
>> [locations, values] = Prob7_A_19(A)
locations =
     2     1
     3     1
     1     2
     3     3
values =
   -22
    -4
    -5
   -11
```

So, each row of **locations** is an identification of a cell in the input that is less than zero, and the corresponding row in **values** contains the value in the input at that location.

```
function [locations, values] = Prob7_A_19(anArray)
% finds the indices and values of the elements in an array
% that are less than zero
% INPUT: An array
% OUTPUTS: An nx2 array of indices
%          An nx1 vector of values

[locationRow locationCol] = find(anArray<0);
locations = [locationRow locationCol];
values = anArray(find(anArray<0));
```

Problem 8

Write a function, **Prob8**, that takes two inputs: an array and a scalar. Your function should return two logical values: (a) a logical value holding **TRUE** if all elements of the input array are greater than the scalar input and holding **FALSE** if otherwise, and (b) a similar logical value but testing if any of the elements of the input array is greater than the scalar input.

For example, using the same input array as in the last problem:

```
>> A = [3 -5 9; -22 8 7; -4 10 -11];
```

A call to **Prob8** should produce the following:

```
>> [allG, anyG] = Prob7_A_20(A, 2)
allG =
     0
anyG =
     1
```

```
The solution of this exercise requires the following pieces of
information:
1. a relational test between an array and a scalar is performed
cell by cell,
2. a logical function operates on each column of an array and
produces a row vector,
3. a logical function operates on the entire row of a row
vector.

function [allGreater, anyGreater] = Prob8(anArray, aScalar)
% tests (a) if all elements in the input array are greater than
% the input scalar and (b) if any element in the input array is
% greater than the input scalar
% INPUTS: An array, a scalar
% OUTPUTS: Logical T/F if all elements are greater
%          Logical T/F if any element is greater

allGreater = all(all(anArray > aScalar));
anyGreater = any(any(anArray > aScalar));
```

Problem 7-A.21

Write a function, **Prob7_A_21**, that inputs two arrays of the same dimensionality and outputs **TRUE** if every element of the first input array is greater than the corresponding element in the second input array.

```
function allGreater = Prob7_A_21(array1, array2)
% tests if all elements in the first input array are greater
% than the corresponding elements in the second input array
% INPUTS: Two arrays of the same dimensions
% OUTPUT: Logical T/F if all elements are greater

allGreater = all(all(array1>array2));
```

Problem 7-A.22

Write a function, **Prob9**, that inputs two arrays of the same dimensionality and outputs **TRUE** if every element of the first input array is equal to the corresponding element in the second input array.

```
function allEqual = Prob9(array1, array2)
% tests if all corresponding elements in the two input array
% are equal
% INPUTS: Two arrays of the same dimensions
% OUTPUT: Logical T/F if all elements are equal

allEqual = all(all(array1==array2));
```

Set B: Problem Solving with MATLAB Arrays

Problem 7-B.1

Write a function, **Prob10**, that takes a row vector of temperatures in degrees Fahrenheit and outputs a two-column array. The first column should consist of the input values in degrees Fahrenheit. The second column should contain the corresponding temperatures in degrees Centigrade. Use Google if you do not remember the conversion relationship.

```
function convTable = Prob10(F)
% creates a conversion table from the given vector of
% Fahrenheit degrees to Centigrade degrees
% INPUT: A 1xn row vector of F temperatures
% OUTPUT: An nx2 array which consists of [F C]

convTable = [F' (F'-32)/1.8];
```

Problem 7-B.2

Your professor requests help with calculation of final grades and grade analysis. *(Dream on!)* The Prof wants a function, **Prob7_B_2**, that will input a table of student numbers and student grades and output the final grade for each student, class grade average, and the student number of the top scoring student

Student #	MT	HW	P	F
273	90	75	91	100
677	65	72	85	87
496	80	82	100	74
641	100	98	95	65
911	92	77	90	81
216	95	83	90	80

A sample input to your function is provided to the right. In this table, the columns contain the student numbers, grades for the midterm, homework, term project and

the final exam. Your professor indicates that each student's weighted final grade should be calculated using the following equation:

$$\text{finalGrade} = 0.3MT + 0.2HW + 0.2P + 0.3F$$

```
function [finalGrades, classAvg, tops] = Prob7_B_2(aTable)
% finds the final grades, class average, and the student number
% of the top scoring student given individual examination scores
% INPUT: An nx5 array of student numbers, midterm, homework,
% term project and final examination grades, arranged in columns
% OUTPUTS: An nx2 array of student numbers and final grades
%          A scalar class average
%          A scalar student number of the top scoring student

finalGrades = 0.3*aTable(:,2)+0.2*aTable(:,3)+...
              0.2*aTable(:,4)+0.3*aTable(:,5);
stuNumbers = aTable(:,1);
classAvg = mean(finalGrades);
tops = stuNumbers(find(finalGrades==max(finalGrades)));
finalGrades = [aTable(:,1) finalGrades];
```

Problem 7-B.3

Write a function, **Prob11**, that inputs an NxM array, finds row and column sums, and concatenates the row and column sums as the last column and row, respectively. The (N+1, M+1) location should contain the sum of all data in the input array.

Your function should output the updated array. Here is an example input:

$$\begin{bmatrix} 2 & 1 & 5 \\ 4 & 6 & 2 \end{bmatrix}$$

Your output array should be as follows:

$$\begin{bmatrix} 2 & 1 & 5 & 8 \\ 4 & 6 & 2 & 12 \\ 6 & 7 & 7 & 20 \end{bmatrix}$$

```
function anArray = Prob11(anArray)
% appends the input array with column sums, row sums,
% and overall sum
% INPUT: An nxm array
% OUTPUT: An (n+1, m+1) array arranged such that:
%         [input_array      sum_of_rows;
%          sum_of_columns   sum_of_all_numbers]

anArray = [anArray sum(anArray')'; ...
           sum(anArray) sum(sum(anArray))];
```

Problem 7-B.4

The National Widget Company sells four types of widgets. Data in the following sample table shows the selling price and number of items sold for each widget in 2002, 2003, and 2004.

Product ID	Selling Price ($)	# Items Sold in 2002	# Items Sold in 2003	# Items Sold in 2004
Widget 1	75	162	157	141
Widget 2	36	421	456	504
Widget 3	78	64	66	75
Widget 4	94	49	56	60

Write a function, **`Prob12`**, that inputs (a) a column vector containing the selling price for each type of widget and (b) an array containing the yearly sales data for each type of widget. For example, for the data in the above table, here are the two inputs:

$$\text{sellingPrice} = \begin{bmatrix} 75 \\ 36 \\ 78 \\ 94 \end{bmatrix}, \quad \text{salesData} = \begin{bmatrix} 162 & 157 & 141 \\ 421 & 456 & 504 \\ 64 & 66 & 75 \\ 49 & 56 & 60 \end{bmatrix}$$

Your function should output (a) a row vector containing the total revenue for each year of data in **`salesData`**, and (b) a column vector containing the

total revenue over all years for each widget. For example, for the data in the above table, the two outputs from **Prob12** should be as follows:

$$\text{totalYearlyRevenue} = \begin{bmatrix} 36904 & 38603 & 40209 \end{bmatrix}$$

$$\text{totalAllYearsEachWidgetType} = \begin{bmatrix} 34500 \\ 49716 \\ 15990 \\ 15510 \end{bmatrix}$$

Hint: You can compute the **totalYearlyRevenue** output using a matrix operation. But the most straightforward way to compute the **totalAllYearsEachWidgetType** output is not by using a matrix multiplication.

```
The goal in this exercise is to achieve a better understanding
of cell-by-cell and matrix operations, and to distinguish
between the situations where they are applied.

function [yearly, widget] = Prob12(salesPrices, yearlySales) %
yearly revenue and product based revenue calculator
% INPUTS: A column vector of sales prices of each product
%         An array of sales data, products in rows, and years in
% columns
% OUTPUTS: A row vector of total revenues for each year
%          A column vector of total revenues for each product

yearly = salesPrices'*yearlySales;
widget = [salesPrices salesPrices salesPrices].*yearlySales;
widget = sum(widget,2);
```

Problem 7-B.5

Suppose you have a catalogue of automobiles following the form of the table below.

Your goal is to find cars (IDs of cars) that meet the criteria of having a stated color, being of a specified type, and costing less than a specified amount.

Write a MATLAB function, **Prob13**, that will input the following:

(a) The numerical data in the format of table shown above

ID #	Color (0=Black, 1=Blue, 2=Red)	Type (0=Sedan, 1=Coupe, 2=Convertible)	Price ($ x 10^3)
1002	0	1	24
2176	1	2	18
3201	2	1	36
4204	1	1	26
4110	2	2	35
5611	1	2	23
7556	0	0	29
8732	1	0	31
9666	0	1	28

(b) Desired color of car (as a number: see the table above)

(c) Desired type of a car (as a number: see the table above)

(d) Desired upper limit for the price of the car (in dollars)

Your function should return a vector of car IDs that meet the specified criteria.

```
function IDs = Prob13(table, color, type, price)
% finds and returns the cars that meet the specified criteria
% INPUTS: An array of car IDs, colors, types and prices
%         A scalar color (0=Black, 1=Blue, 2=Red)
%         A scalar type (0=Sedan, 1=Coupe, 2=Convertible)
%         A scalar upper limit for the price (thousand dollars)
% OUTPUT: A column vector of vehicle IDs


indices = find(table(:,2)==color & ...
          table(:,3)==type & table(:,4)<price);
IDs = table(indices,1);
```

Problem 7-B.6

Write a function, **Prob7_B_6**, that inputs two arrays, **A** and **B**, of the same dimensionality. Your function should update the two arrays, and then output the (possibly) changed arrays **A** and **B**. The rules for updates are the following:

a) If an element in **A** is smaller than the corresponding element in **B**, set element in **A** to zero.

b) If element in **A** is greater than the corresponding element in **B**, set element in **B** to zero.

c) If element in **A** is equal to the corresponding element in **B**, set element in **A** to zero and set the element in **B** to zero.

```
function [A,B] = Prob7_B_6(A,B)
% Updates the two input arrays according to the following rules:
% a) If an element in A is smaller than the corresponding
% element in B, set element in A to zero.
% b) If element in A is greater than the corresponding element
% in B, set element in B to zero.
% c) If element in A is equal to the corresponding element in B,
% set element in A to zero and set the element in B to zero.
% INPUTS: Two arrays of the same dimensions
% OUTPUTS: Two modified arrays of the same dimensions

% Let's take copies of A and B first
AOrig = A; BOrig = B;

% Now we can update A and B using the original inputs
A(find(AOrig<BOrig))=0;
B(find(AOrig>BOrig))=0;
A(find(AOrig==BOrig))=0;
B(find(AOrig==BOrig))=0;
```

Problem 7-B.7

Gillian, an agricultural engineer, is developing a new meal ration for emus. Gillian's goal is to create a mixture of corn, soybean meal, and cottonseed that will meet the nutritional needs of emus. The new ration will be sold in bags containing **proteinKG** kg of protein, **fatKG** kg of fat, and **fiberKG** kg of

fiber. (Gillian wants to experiment with the values for each nutrient, so she is thinking of them as variables.)

The table below gives the nutrient content of each type of available raw material.

	Corn	Soybean Meal	Cottonseed
% protein	24%	42%	19%
% fat	41%	20%	21%
% fiber	31%	19%	11%

Write a function, **`Prob14`**, that will assist Gillian. Your function should input the value set for **`proteinKG`**, for **`fatKG`**, and for **`fiberKG`**. Your function should output (a) a vector containing the amount of corn to put in each bag, the amount of soybean meal to put in each bag, the amount of cottonseed to put in each bag, and (b) the total amount of feed in a single bag. The data in the table above should be represented internally in your function in an array.

Test your function with inputs from **`Prob14(21,25,16)`**. The total amount of feed you should compute for one bag of emu ration with these inputs should be 96 kg.

Once you complete your function for Gillian and have tested it, try the inputs of **`Prob14(21,31,16)`**. How can you tell from the result that this input set is not realizable as a bag of emu feed using the raw materials based on the results of your function?

```
function [mixture,weight] = Prob14(proteinKg,fatKg, fiberKg)
% Finds the amount of corn, soybean and cottonseed to obtain
% desired nutritional values
% INPUTS: Desired nutritional kilograms of protein, fat, fiber
% OUTPUTS: The required mixture in a column vector
%          Total weight of the feed

nutritionTable = [0.24 0.42 0.19; 0.41 0.2 0.21; ...
                  0.31 0.19 0.11];
desiredKg = [proteinKg; fatKg; fiberKg];
mixture = nutritionTable\desiredKg;
```

```
weight = sum(mixture);
```

Problem 7-B.8

Steel is often produced by blending carbon with other steel alloys that contain nickel, manganese, and chromium. The percentage of each of these metals in a blend affects the properties of the resultant steel. For instance, stainless steel is defined as a ferrous alloy (an alloy containing iron) with a minimum of 10.5 percent chromium content.

Suppose you are working for a steel blending company. Your company produces steel by combining three different stock alloys that each contain percentages of nickel, manganese, chromium, and carbon. The percentages for each of the three stock alloys your company uses, along with the cost per ton you have invested in each, are shown in the table below.

	Chromium (%)	Nickel (%)	Manganese (%)	Carbon (%)	Cost per Ton ($)
Alloy 1	25	7	5	2	115
Alloy 2	6	5	6	4	92
Alloy 3	11	6	7	5	129

You receive an order for 70 tons of stainless steel that is composed of 40% Alloy 1, 30% Alloy 2, and 30% Alloy 3. Does this alloy composition produce stainless steel? What are the overall chromium, nickel, manganese, and carbon percentages in the blend you will produce? What is the total cost of this production?

Instead of doing this problem from scratch each time you get a new order, your boss has asked you to write a function, **`Prob7_B_8`**, that inputs (a) the total number of tons in an order, (b) a vector containing the percentages of chromium, nickel, manganese and carbon specified in the order, and (c) a vector containing the cost you have in each ton for Alloy 1, Alloy 2, and Alloy 3. Your function should represent internally the percentages of chromium, nickel, and manganese for all of your stock alloys in an array.

Your function should output the following:

- A vector containing the computed number of tons of each of your three stock alloys to use to meet the order

- The cost you will have in raw materials to meet the order
- An answer to the question: Does the blend qualify as stainless steel? (return a logical value)

Point to think about: Why was it a good idea to utilize the cost of your base alloys as inputs in your function, while on the other hand the percentage content makeup of each stock material was represented internally in your function?

```
function [alloysTons, overallCost, isStainless] = ...
               Prob7_B_8(totOrder,percentAlloys,alloyCosts)
% This function calculates a mixture of all alloys to produce
% steel
% INPUTS: Total order (tons)
%         Percent requirement of Alloys in the mixture in a row
% vector
%         Alloy costs per ton in a row vector
% OUTPUTS: Tons of Alloy 1, Alloy 2, Alloy 3 in the mixture (a
% row vector)
%          Overall cost of the production
%          True if the steel is stainless
% Sample line to call the function:
% [alloysTons, overallCost, isStainless] = ...
% prob7_B_8(50,[40 30 30],[100 70 90])


  alloyContent = [.25 .07 .05 .02; .06 .05 .06 .04; ...
                  .11 .06 .07 .05];
  percentAlloys = percentAlloys*0.01; % convert percentage
  alloysTons = percentAlloys*totOrder;
  overallCost = sum(alloysTons.*alloyCosts);

% Finding the ingredients in the alloy mixture:
  ingredientPercentages = percentAlloys*alloyContent;
  isStainless = ingredientPercentages(1)>=.105;

% Why did we make the costs input but not the alloy contents?
% Because costs might change according to market conditions
% but the alloy content is less prone to change since it
% involves change in the manufacturing process.
```

Problem 7-B.9

A furniture manufacturer makes dining tables, desks, coffee tables, and end tables. The material and labor requirement for each type of production is given in the table below:

	Dining Table	Desk	Coffee Table	End Table
Wood (sq ft)	145	130	95	55
Labor (hrs)	12	9	7	5
Selling Price ($)	750	520	340	177

The current cost for wood is $2.25 per sq. ft.; the current cost for labor per hour is $8.50.

Your supervisor has asked you to write a function, **`Prob7_B_9`**, to process new orders. The orders will come in as quantities of dining tables, desks, coffee tables, and end tables. Your function should deal with these four numbers as a single input vector and as its first input argument. The second and third inputs to your function should be the current cost for wood and the current cost for labor.

Your function should compute and output (a) the total profit that you company will make on the order, (b) the total amount of wood used in the order, and (c) the total number of hours required to complete the order.

```
function [profit,sqFtWood,hrLabor] = ...
           Prob7_B_9(productOrders,woodCost,laborCost)
% This function makes a profit and resource analysis
% INPUTS: The order for dining tables, desks, coffee tables,
%         and end tables (a row vector)
%         Cost of wood (per sq ft)
%         Cost of labor (per hour)
% OUTPUTS: Profit made from this order ($)
%          Amount of wood used (sq ft)
%          Hours of labor spent
% Sample line to call the function:
% [profit,sqFtWood,hrLabor]=prob7_B_9([5 7 3 1],2.25,8.5)

resourceAndPrice = [145 130 95 55;12 9 7 5; 750 520 340 177];
woodLaborPrice = productOrders*resourceAndPrice';
totalCost = [woodCost laborCost]*woodLaborPrice(1:2)';
```

```
profit = woodLaborPrice(3)-totalCost;
sqFtWood=woodLaborPrice(1);
hrLabor=woodLaborPrice(2);
```

Problem 7-B.10

Your friend, Katherine, a stock broker, asks for your help in analyzing stock prices. She tells you that she keeps daily records for a number of stocks. Her strategy is to buy stocks whenever the stock price is lower than its average over the last three days and to sell a given stock if its price is higher than the three-day average.[1]

Katherine sends you a sample record of stock prices as shown in the table below. The first column represents the stock identifier and the remaining columns are stock prices in successive days.

Stock Number	Price ($) April 21st	Price ($) April 22nd	Price ($) April 23rd	Price ($) April 24th
001	14.00	15.25	13.50	15.50
002	21.00	18.50	19.25	19.00
003	7.00	7.75	7.25	7.50
004	32.50	31.25	30.00	30.25

Write a function, **`Prob7_B_10`**, that inputs the stock prices in the table as an array. Your function should compute and output two vectors:

- The identifiers for stocks that should be bought on April 25
- The identifiers for stocks that should be sold on April 25

Each day Katherine wants to play the market with her strategy, she is going to have to retype the array with current stock prices. This would get old, fast. What would you like to be able to do to prevent retyping? How could you implement your idea by extending your function? (For one possible option, see **`save`** and **`load`** in MATLAB Help.)

1. The first thing you should tell Katherine is that she is likely to lose her shirt with this strategy!

```
function [buy, sell] = Prob7_B_10(stockPrices)
% running average analysis for stock exchange
% INPUT: An array of stock identifiers and daily stock prices
%        arranged in columns
% OUTPUTS: A column vector of identifiers of stocks to buy
%          A column vector of identifiers of stocks to sell

stockIdentifiers = stockPrices(:,1);

averages = mean(stockPrices(:,end-2:end),2);

buyIndices = find(stockPrices(:,end)<averages);
sellIndices = find(stockPrices(:,end)>averages);

buy = stockIdentifiers(buyIndices);
sell = stockIdentifiers(sellIndices);
```

SOLUTIONS FOR CHAPTER 8 MORE FLEXIBILITY: INTRODUCTION TO CONDITIONAL AND ITERATIVE PROGRAMMING

Problems in this chapter are in general the most difficult in this text, especially for some of the problems in Set B. This chapter should be the culmination of a number of threads played out in this textbook, and the problems of Chapter 8 should help the students bring these threads together.

The most important thread is that students must become independent problem solvers. Particularly in Set B, there are a number of suggestions made to the instructor to encourage students to extend beyond the problem spec that is given.

A second thread is that in MATLAB (as is true in any solid computational environment) there are almost always multiple ways of solving problems. When using MATLAB, this is particularly true given the availability of loop constructs and vector operations.

A third thread is that using graphical plotting methods can be helpful for understanding a situation and that this understanding can open a path to full problem solution. Some of the problems below have this characteristic.

Finally, a fourth thread is that using MATLAB HELP effectively is essential for MATLAB users. No university course and no single textbook can possibly "teach" all the capabilities of MATLAB.

Set A: Nuts and Bolts Problems for Programming Constructs

Problem 8-A.1

Write a MATLAB function, **`Prob8_A_1`**, that inputs a a scalar **`x`** and returns a scalar **`y`** according to the following rules. Use Form 4 of **IF-THEN-ELSE** in your function.

$$\left.\begin{array}{l} y = -x - \pi \\ y = \sin(x) \\ y = -x + \pi \end{array}\right\} \; for \; \left\{\begin{array}{c} x \le -\pi \\ -\pi < x \le \pi \\ \pi < x \end{array}\right.$$

On the surface, this problem is easy. A function that meets the problem spec is as follows:

```
function y = Prob8_A_1(x)
% if x<=-pi, outputs -x-pi; if -pi<x<=pi, outputs sin(x);
% if pi<x, outputs -x+pi
% INPUT: x a scalar
% OUTPUT: y a scalar

  if x <= -pi
      y = -x-pi;
  elseif (-pi < x) & (x <= pi)
      y = sin(x);
  elseif pi < x
      y = -x+pi;
  else
      error('uncovered input')
  end
```

Note that the IF and ELSEIF clauses are used to cover all "possibilities" that are straightforward to see. In reality, in this case, all of these "possibilities" taken together are exhaustive. But it is good practice to write a IF construct to employ an ELSE clause that will catch unforeseen contingencies - and produce an error if "hit."

There are two additional steps that instructors may want to consider. These two steps are not explicitly written into the problem. The two provide an example of the type of extension

that instructors might want to make in the problem specs for this chapter.

First, to emphasize an earlier lesson about solution testing, you might tell students to test their solution function for this problem by producing a graph of f(x) = Prob8_A_1(x) for -2π < x < 2π. A script using a FOR loop to do this testing is as follows:

```
testX = linspace(-2*pi, 2*pi);
testY = [];
for oneX = testX
    testY = [testY, Prob8_A_1(oneX)];
end
figure
plot(testX, testY, 'bo')
grid on
title('Prob8-A-1 results')
xlabel('x')
ylabel('y per problem spec')
```

Second, a good additional exercise would be to require your students to build their version of Prob8_A_1 so that it will work on a vector of input values. A version of Prob8_A_1 meeting this additional requirement by using a FOR loop is shown below:

```
function [allX, allY] = Prob8_A_1_modified(allX)
% if x<=-pi, outputs -x-pi; if -pi<x<=pi, outputs sin(x);
% if pi<x, outputs -x+pi
% INPUT: x a vector
% OUTPUT: y a vector of corresponding values
% sample call and test:
%     allX = linspace(-2*pi, 2*pi);
%     [aX, aY] = Prob8_A_1_modified(allX);
%     figure
%     plot(aX, aY, 'o'), title('Test Prob 8-A-1 modified')
%     grid on
%     xlabel('x'), ylabel('y per problem spec')

  allY = [];
  for x = allX
      if x <= -pi
          y = -x-pi;
      elseif (-pi < x) & (x <= pi)
          y = sin(x);
      elseif pi < x
          y = -x+pi;
      else
          error('uncovered input')
      end
      allY = [allY, y];
```

```
end
```

Problem 8-A.2

Create a function, **`Prob8_A_2`**, that returns the course grade for a given numerical score for the term. Numerical scores are on a 100-point scale, and final course grades are on a 4-point scale. Your function must input a numerical score and return a vector containing the numerical score for a student in first position and the corresponding course grade in the second position. Here is the grading scale for the course:

- 90 and above: 4.0
- Between 80 and 89: 3.0
- Between 70 and 79: 2.0
- Between 60 and 69: 1.0
- 59 and below: 0.0

For example, if the input score is **`84`**, your function must return the term grade **`3`**. Again, use Form 4.

```
This problem is essentially like the preceding problem in form.
The solution below is a vectorized function that goes beyond the
actual problem spec.

The problem spec is intentionally not complete. Note that the
problem spec does not take into account a score of, for example,
89.2. It would be worthwhile to push your students on this point
to reinforce the idea that it is the solver's responsibility
(i.e., the student's) to clarify any points found lacking in a
problem specification.

function courseGrade_VECTOR = Prob8_A_2(numericalScore_VECTOR)
% converts a vector of numerical scores to a vector or
% corresponding course grades using the following:
% score < 60          yields 0.0,
% 60 <= score < 70    yields 1.0
% 70 <= score < 80    yields 2.0
% 80 <= score < 90    yields 3.0
% 90 <= score         yields 4.0
% INPUTS: numericalScore_VECTOR: vector of term numerical
%                                scores
```

```
% OUTPUTS: courseGrade_VECTOR: vector of corresponding term
%                              scores
% sample call:
%    allNumScores = [55 95 85 65 75 0 100];
%    courseGrade_VECTOR = Prob8_A_2(allNumScores)

  courseGrade_VECTOR = [];
  for one_numScore = numericalScore_VECTOR
      if one_numScore < 60
          oneCourseGrade = 0.0;
      elseif (60 <= one_numScore) & (one_numScore < 70)
          oneCourseGrade = 1.0;
      elseif (70 <= one_numScore) & (one_numScore < 80)
          oneCourseGrade = 2.0;
      elseif (80 <= one_numScore) & (one_numScore < 90)
          oneCourseGrade = 3.0;
      elseif (90 <= one_numScore)
          oneCourseGrade = 4.0;
      else
          error('bad numerical score')
      end
      courseGrade_VECTOR = [courseGrade_VECTOR, oneCourseGrade];
  end
```

Problem 8-A.3

Write a function, **Prob8_A_3**, using two scalar inputs **x** and **y** and the following code:

```
if x > 3
   if y > x
      disp('x is greater than 3 and y is greater than x')
   end
end
```

This code is two nested Form 1 versions of **IF-THEN-ELSE**.

Modify your function, and save it as **Prob8_A_3b**, so you have one Form 1 conditional.

Test your revised code with the following pairs of **x** and **y**:

x	1	1	4	4	4
y	2	4	3	4	5

```
This question is meant to exercise students' ability to use log-
ical functions and apply logical functions in IF constructs.
Code meeting the problem spec is below:

function Prob8_A_3b(x,y)
% displays a message if x>3 and y>x
% INPUTS: x, a scalar
%         y, a scalar
% OUTPUTS: none
% sample call:
%    Prob8_A_3(5,7)

  if (x > 3) & (y > x)
      disp('x is greater than 3 and y is greater than x')
  end
```

Problem 8-A.4

Are the following Form 1 and Form 2 **IF-THEN-ELSE** statements functionally equivalent? That is, do they generate the same result? Why?

Code 1	Code 2

`if x > y`
`  z = 1;`
`else`
`  z = 0;`
`end`
	`z = 0;`
	`if x > y`
	`  z = 1;`
	`end`

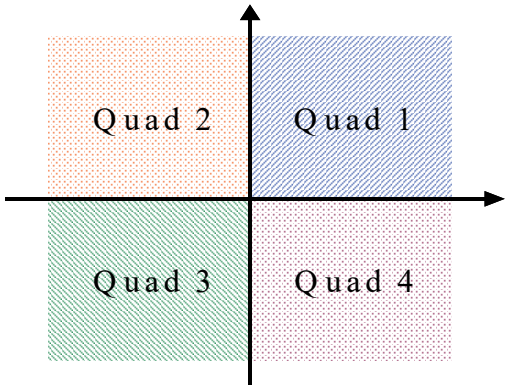
```
Code 1 and Code 2 are functionally equivalent. The point of this
problem is to show students two common alternative forms for
setting a variable dependent on a test.

The topic of which is these is "best practice" is left open by
this problem. In general Code 1 is preferred because it captures
in one code "block" the two options for setting the value of z.
```

```
It is left to instructor option whether to include a discussion
of modularity in talking about this problem.
```

Problem 8-A.5

Write a MATLAB function Form 3 conditional as function, **Prob8_A_5**, that inputs an angle in degrees and returns one three-element vector containing the quadrant of the angle (consider angles between 0 and 360 degrees), and the sine and cosine of the angle. See the following drawing for quadrants.



Assume that 0° is in quadrant 1, 90° is in quadrant 2, 180° is in quadrant 3, and 270° is in quadrant 4.

Use Form 4 of **IF-THEN-ELSE** for this problem.

```
A Form 4 IF is requested in this problem specification. However,
specifying a Form 4 IF construct does not pin down how many
clauses there should be in the code. In the code below, again, a
solution is given that allows error trapping.

function [quad, sinAng, cosAng] = Prob8_A_5(angDeg)
% inputs an angle in degrees and ouputs (a) the quadrant in
% which the resides, (b) the sine of the angle, and (c) the
% cosine of the angle
% INPUTS: angDeg, a scalar angle in degrees
%                          with 0 <= angDeg <= 360
% OUTPUTS: quad, a scalar, the quadrant number of the angle
```

```
%                          0 <= angDeg <  90  => quad 1
%                         90 <= angDeg < 180  => quad 2
%                        180 <= angDeg < 270  => quad 3
%                          0 <= angDeg < 360  => quad 4
% sample call:
%     [q, sinA, cosA] = Prob8_A_5(32)

      if (0 <= angDeg) & (angDeg < 90)
         quad = 1;
      elseif (90 <= angDeg) & (angDeg < 180)
         quad = 2;
      elseif (180 <= angDeg) & (angDeg < 270)
         quad = 3;
      elseif (270 <= angDeg) & (angDeg < 360)
         quad = 4;
      else
         error('input angle out of range')
      end

      angRad = angDeg * (pi/180);
      sinAng = sin(angRad);
      cosAng = cos(angRad);
```

Problem 8-A.6

Do the following tasks using loop structures:

1. Find the sum of the first 50 integers.
2. Find the product of the first 10 even integers.

```
Both of these problems should be straightforward. Both can be
done without a loop as well, using instead a vector operation.
It will help your students maintain continuity if you ask them
to produce a vector operation solution too. The code below shows
the equivalent vector operations as comments. Note that the
variables should *not* be named sum and prod because these are
names of MATLAB build-in functions.

% script for Prob8_A_6
clc, clear

% TASK 1: find sum of first 50 integers using a loop
  my_sum = 0;
  for i = 1:50
     my_sum = my_sum + i;
  end
```

```
  my_sum

  % solution with vector operation
  %    my_sum = sum(1:50)


% TASK 2: find product of first 10 integers using a loop
  my_prod = 1;
  for i = 2:2:10
     my_prod = my_prod * i;
  end
  my_prod

  % solution with vector operation
  %    my_prod = prod(1:10)
```

Problem 8-A.7

Consider the Form 4 **IF-THEN-ELSE** statement shown below and answer the following questions. To get your answers, step through the code the way MATLAB does when the code is executed. Keep track of the variables and how their values change each time through the loop. It is easiest to do this if you write down the variables and their values in a simple table. After you have completed the paper and pencil exercise, perform this activity using MATLAB, and verify your answers.

```
x=140;
for i = 1:10
   if x < 1
      break
   elseif x>50
      x=x/5;
   elseif x>60
      x=x/7;
   else
      x=x/10;
   end
end
disp(x)
```

The built-in function **break** is serving a special purpose in the code above. Look up **break** in MATLAB Help and be sure you understand how it is being used here.

1. What will MATLAB display?
2. How many times will this loop be executed?

The break statement executed inside a loop forces program control to "jump" to the outside of loop. In a sense, it enables a way that the functionality of a standard WHILE loop can be achieved by using a FOR loop.

To understand the code in this problem, students must get into the mindset of "pretending to be the computer." Loops are typically hard for students to understand if they have not dealt with them before. Even though each line of code may in isolation be understood, still some students have difficulty imagining the code repeating again and again. A large contributing factor is that these students do not systematically step through the code – again… pretending to be the computer.

Inserting the code lines as shown below will help students understand the end result better as well.

```
% script for code of Prob 8-A.7

x=140;
for i = 1:10
   i
   if x < 1
      break
   elseif x>50
      x=x/5;
   elseif x>60
      x=x/7;
   else
      x=x/10;
   end

   [i x]
end
disp(x)
```

Problem 8-A.8

Write a function, **Prob8_A_8**, that (a) returns the value of **N** such that the series sum for $\sum_{k=1}^{\infty} \frac{1}{k^2}$ is precise to within 0.001 and (b) returns the value of the series to that precision.

Since this infinite series is monotonically decreasing, you can determine when to stop (and to stop using **break**) by comparing the sum after **M** terms have been added, to the sum after **M-1** terms have been added:

sum_M-sum_{M-1} < 0.001, then N = M.

This problem is common when dealing with computing an infinite series. You cannot set up your computational loop to do an infinite number of iterations through the loop because that would require infinite time. The solution is to determine a degree of precision that you will be content with, then stop the loop and return the answer when that level of precision is reached.

This problem is meant as an exercise in FOR loop setup. The important point for students to learn is that before a LOOP is set up, it is sometimes (usually) necessary to define one or more variables to maintain a "memory" of the results or settings in earlier iterations.

A function meeting the specs for this problem is below.

```
function [stopAt_N, seriesSum] = Prob8_A_8
% Determines when to stop the series 1/k... such that the
% precision of the sum of the series is accurate to within
% 0.001. The value of the series sum to precision 0.001 is
% also returned.
% INPUTS: none
% OUTPUTS: stopAt_N, scalar, the number of the series
%                    element such that the series sum attains
%                    precision of 0.001.
%          seriesSum, scalar, sum of the series to stopAt_N
%                     terms
% sample call:
%      [stopAt_N, seriesSum] = Prob8_A_8

   seriesSum = 0;
   oldTerm = 1;
   for i = 2:inf
      newTerm = 1/(i^2);
```

```
   seriesSum = seriesSum + newTerm;
   if (oldTerm - newTerm) <= 0.001
      stopAt_N = i;
      break
   end
   oldTerm = newTerm;
end
```

Problem 8-A.9

Consider the following code:

```
a = [];
x = 0;
for i = 1:4
    y = 2*x^2+3;
    if y < 10
        z = y;
    end
    x = x + 1;
    a = [a; [x,y,z] ];
end
```

This code implements a **FOR LOOP** that updates the values of **x**, **y**, and **z**. First, implement the code on a piece of paper and fill in the following table.

Pass	Value of x	Value of y	Value of z
First	?	?	?
Second	?	?	?
Third	?	?	?
Fourth	?	?	?

Next, implement the code in a function file, **Prob8_A_9**, run it, and verify your answers.**A** is used to record values of **x**, **y**, and **z** at the end of each pass.

This sort of table is important for students in that it is a tangible embodiment of being "systematic" when trying to understand code. A completed version of the table is below.

Pass	Value of x	Value of y	Value of z
First	1	3	3
Second	2	5	5
Third	3	11	5
Fourth	4	21	5

Problem 8-A.10

Write a MATLAB function, **`Prob8_A_10`**, that inputs two vectors and computes and returns a vector that contains their cell-by-cell sum. Assume that inputs are two same-length vectors, and use a **FOR LOOP** over columns to compute the cell-by-cell sum.

```
function vSum = Prob8_A_10(v1, v2)
% computes the cell-by-cell sum of two vectors using a loop
% INPUT: v1, row vector of length M
%        v2, row vector of length M
% OUTPUT: vSum, col vector of corresponding sums
% example call:
%    vSum = Prob8_A_10([3, 1, 5], [2,10,3])

   vSum = [];
   for oneRow = [v1; v2]
      vSum = [vSum; sum(oneRow)];
   end
```

Problem 8-A.11

Write a MATLAB function, **`Prob8_A_11`**, that inputs two arrays and computes and returns an array that contains their cell-by-cell sum. Assume that inputs are two same-sized arrays, and use two nested **FOR LOOP** constructs to compute the cell-by-cell sum.

```
This exercise is very important. Students should take away from
this problem the ability to "step through" the elements of an
array. One key point for students: that because an array is two
dimensional, two loops are required.
function aSum = Prob8_A_11(A1, A2)
% computes the cell-by-cell sum of two arrays
% using two nested loop
% INPUT: A1, array of size N x M
%        A2, array of size N x M
% OUTPUT: aSum, array of size N x M
%               of corresponding sums
% example call:
%    A = [1 2 3; 4 5 6];
%    B = [10 20 30; 40 50 60];
%    mySum = Prob8_A_11(A, B)

   [numRows, numCols] = size(A1);
   for rCt = 1:numRows
      for cCt = 1:numCols
         aSum(rCt, cCt) = A1(rCt, cCt) + A2(rCt, cCt);
      end
   end
```

Problem 8-A.12

Consider this sequence:

$$7k^3 \quad k = 1, 2, 3, \ldots, 10$$

Your task is to compute the sum of the first ten elements of the series.

Write a function, **`Prob8_A_12a`**, that solves the problem using vector operations.

Write a second function, **`Prob8_A_12b`**, that solves the problem using a **FOR LOOP**.

Be sure that your two functions are returning the same answer.

```
For this problem students create a vector solution as a solution
based on using a loop. Students need to be conversant with both
```

```
methods in order to be able to choose the most applicable method
for a given problem.

Solutions following both paths are shown below. Notice that the
solution using a FOR loop (Prob8_A_12b) goes beyond the problem
specification and allows an integer input to indicate how many
elements of the series to use in a sum. The extension is made
here largely to accommodate the next problem.

function sumFirstTen = Prob8_A_12a
% compute the sum of elements 7k^3, for k = 1,2,..., 10. Use
vector operations
% INPUTS: none
% OUTPUTS: sumFirstTen, scalar, the indicated sum
% sample call:
%        mySum = Prob8_A_12a

    kVals = 1:10;
    sumFirstTen = sum(7*kVals.^3);


function sumFirstN = Prob8_A_12b(N)
% compute the sum of elements 7k^3, for k = 1,2,..., N.
% Use a FOR loop.
% INPUTS: N, positive integer, sum extends to Nth element
% OUTPUTS: sumFirstN, scalar, the indicated sum
% sample call:
%        mySum = Prob8_A_12b(10)

    sumFirstN = 0;
    for k = 1:N
        sumFirstN = sumFirstN + 7*k^3;
    end
```

Problem 8-A.13

For the sequence given in problem A.12, write a function, **Prob8_A_13**, that inputs a positive integer **N** and calculates and returns a vector containing the first **N** series sums as a vector output. (Use the series in Problem A.12.) Use **Prob8_A_12b** (the second function you wrote for Problem A.12) in your new function, **Prob8_A_13**.

```
This problem cannot be solved as stated. The problem is that
Prob8_A_12b does is not described as having a input. Thus to
```

```
solve this problem as specified, an extension to Prob8_A_12b
must be accomplished.

There are two distinct lessons for students here. First,
specifications should be well understood. Modifications to
specifications may be necessary. Second, once Prob8_A_12b is
correctly extended, this problem becomes easy; the second lesson
focuses on problem decomposition.

function seriesSumsV = Prob8_A_13(N)
% Computes the first N series sums for the series with
% elements 7k^3. For example, for N = 3, the output will
% have three elements [7, 7+7*2^3, 7+7*2^3+7*3^3], or
% [7, 63, 252].
% INPUTS: N, integer
% OUTPUTS: seriesSumsV, vector with N elements, each a
%                       series sum as described above
% sample call:
%    sumsVector = Prob8_A_13(3)
% uses function:
%    Prob8_A_12b

    seriesSumsV = [];
    for i = 1:N
        seriesSumsV = [seriesSumsV, Prob8_A_12b(i)];
    end
```

Problem 8-A.14

Create a function, **`Prob8_A_14`**, that inputs a vector and outputs the maximum value in the input vector. Use a **FOR LOOP** instead of the built-in function **`max`**.

```
This is a straightforward problem.

function currentMax = Prob8_A_14(valueVector)
% find the max value in a vector using a loop - not the
% standard build-in function max.
% INPUTS: valueVector, vector of values
% OUTPUTS: theMax, scalar, the max value in valueVector
% sample call:
%    aMaxVal = Prob8_A_14([4 2 0 99 31]);

   currentMax = -inf;
   for oneVal = valueVector;
       if oneVal > currentMax
```

```
            currentMax = oneVal;
        end
    end
```

Problem 8-A.15

A student writes the following code as part of a function to calculate the growth of $10,000 in a bank account over the next 25 years. He assumes a constant interest rate of 6%:

```
initial_money=10000 % dollars
for year = 1:25
   % new balance is found by applying the interest
   money=(initial_money*1.06);
   % increment year by one
   year=year+1;
end
```

Put this code into a function that has no input and that returns **money** and call it **Prob8_A_15**.

This code has two runtime errors. Find and fix the problems and test your revised function in MATLAB.

A correct form of the code is shown below. Both runtime errors have to do with removing the variable initial_money and replacing it with the variable money. The line "year = year + 1" placed as it was in the original code above had no effect - it was a NOOP. Loop variables (year in the example) can be reset in a loop, but (a) it will have no effect on the next iteration and (b) it is bad programming form largely from an "understandability" dimension.

```
function money = Prob8_A_15
% calculate growth in a bank account:
%    deposits $10,000 at the start
%    leaves the money in the account for 25 yrs
%    gets a return of 6% every year

   money = 10000; % dollars
   for year = 1:25
       % new balance is found by applying the interest
       money=(money*1.06);
```

```
end
```

Problem 8-A.16

George opens a new bank account with $35,000. He is planning to deposit $5,000 every year for the next 14 years. The interest rate on his account is 4% and is constant. George would like to create a plot showing the growth of his money. He would like enough flexibility to see what would happen with differing values for (a) initial deposit, (b) annual deposits, (c) interest rates, and (d) length of time he will maintain the account.

Create a function, **`Prob8_A_16`**. The function should input the four variables that George wants to vary, create a plot of the amount of money in George's account versus time, and output a vector where each element is the starting yearly balance for the corresponding year.

```
This problem is conceptually not difficult. There is one detail
that many students tend to trip over: setting a starting point
of the savings account at year 0. Put another way, the value of
the savings account at the end of year 1 (or the start of year
2) should be one year after the initial deposit. In the solution
shown below, this detail is handled by building a vector,
yrBalance, of yearly balance values such that yrBalance(1) is
the initial deposit, yrBalance(2) is the value after 1 year, …,
and yrBalance(N+1) is the balance in the account after N years.

function yrBalance = Prob8_A_16(initDeposit, annualDeposit, ...
                                interestRate, time)
% Compute and plot the yearly beginning balance for a bank
% account.
% INPUTS: initDeposit, scalar, the amount initially
%                              deposited (dollars)
%         annualDeposit, scalar, the amount deposited at the
%                              start of each year after the first
%                              (dollars)
%         interestRate, scalar, constant interest rate
%                                   (decimal)
%         time, scalar, the number of years the account will
%                         be active
% OUTPUTS: yrBalance, vector with <time+1> elements showing
%                        the yearly starting balance
%                        first element is the initial deposit
% sample call:
%     bal = Prob8_A_16(10000, 1000, 0.05, 30)
% also create a plot of yrBalance VS time
```

```
yrBalance(1) = initDeposit;
for yrNum = 2:time+1
    yrBalance(yrNum) = ...
        yrBalance(yrNum-1)* (1 + interestRate)  + ...
        annualDeposit;
end

figure
plot(0:time, yrBalance, 'LineWidth', 2)
grid on
title('Bank Account Balance vs. Time', 'FontSize', 14)
xlabel('number of years', 'FontWeight', 'bold')
ylabel('account balance (dollars)', 'FontWeight', 'bold')
annotationStr = sprintf(['init deposit = $%6.0f\n',...
                         'yearly deposit = $%6.0f\n',...
                        'interest rate = %2.1f percent\n',...
                         'time = %2.0f years'], ...
                        initDeposit, annualDeposit, ...
                        interestRate*100, time);
disp('PLACE AN ANNOTATION STRING')
gtext(annotationStr, 'FontWeight', 'bold')
```

.

Set B: Problem Solving Using Programming Constructs

Problem 8-B.1

The *Freshest Foods Market* is announcing a new promotion. Their theme is:

> "Cutting prices as you buy! 10% off every additional item you buy after your purchase totals $20, and 20% off after $50!"

When she reaches the counter Julie arranges her groceries as follows:

Item	Gum	Orange	Cereal	Cheese	Coffee	Tomato	Pizza	Sushi	Shrimp
Unit Price ($)	**0.45**	**2.95**	**1.99**	**4.99**	**4.95**	**3.99**	**7.99**	**16.95**	**11.99**
Number of items	**1**	**1**	**2**	**1**	**1**	**2**	**2**	**1**	**2**

How much will Julie pay? Build a function, **Prob8_B_1**, to answer the question. Input should be an array representing data as in the table above. Output should be the total bill Julie will pay, as a scalar.

Use a **FOR LOOP** in your function. Consider how you would solve this problem with vector operations using the built-in function **cumsum** as part of your solution.

Functions for both a FOR solution and a CUMSUM (vector) solution are shown below.

Note the structure of the IF construct embedded in the FOR solution. Some freshman learners will not distinguish in this type of problem any necessity to put one or the other clause first in the construct. As an exercise to emphasize this point, the instructor might write the IF as the following ...

```
if totalCost > 20
    discount = 0.10;
elseif totalCost > 50
    discount = 0.20;
end
```

... and ask students to step through the code for cases of totalCost = 21, and totalCost = 51, and indicate the value of discount that results.

There is a more general lesson from this problem as well. Note that the logic for the vector solution (shown below as Prob8_B_1b) is somewhat more difficult to follow (and to write) than the solution using a FOR construct (shown below as Prob8_B_1). A very useful discussion can be carried on focusing on the question "What makes this problem easier to do using a FOR as opposed to vector operations?" The answer lies in the nature of vector operations. It is very easy with vector operations to operate on an entire vector or array at once. But in this problem, the problem spec forces an identification of the element in the array at which a certain condition is met. This characteristic, although not full proof, is a good rule of thumb to indicate that a FOR solution will be easier to devise than a vector based solution.

```
function totalCost = Prob8_B_1(costAndNum_table)
% Compute the total cost of a grocery basket full of items
% when the following sale terms apply: after $20, there is a
% 10% discount; after $50, there is a 20% discount. Solve
% using a LOOP.
% INPUT: costAndNum_table, array, table in which each column
%                           is ... <unitPrice; numItemsBought>
```

```
%                           Note that the order is
%                           significant; sale is
%                           accomplished from low numbered
%                           columns to high numbered
%                           columns.
% OUTPUT: totalCost, scalar, the total cost of the
%                           groceries.
% sample call:
%  dataTable = ...
%       [0.45,2.95,1.99,4.99,4.95,3.99,7.99,16.95,11.99; ...
%           1,   1,   2,   1,   1,   2,   2,     1,   2];
%  totCost = Prob8_B_1(dataTable)
count = 1;
    totalCost = 0;
    discount = 0.0;
    for oneItem = costAndNum_table
        oneItemPrice = oneItem(1);
        oneItemNum = oneItem(2);
        costThisItemNoDiscount = oneItemPrice * oneItemNum;
        costThisItem = costThisItemNoDiscount * (1 - discount);
        totalCost = totalCost + costThisItem;
        if totalCost > 50
            discount = 0.20;
        elseif totalCost > 20
            discount = 0.10;
        end
    end


function costAll = Prob8_B_1b(costAndNum_A)
% Compute the total cost of a grocery basket full of items
% when the following sale terms apply: after $20, there is a
% 10% discount; after $50, there is a 20% discount. Solve
% using vector operations.
% INPUT: costAndNum_A, array, table in which each column
%                      is ... <unitPrice; numItemsBought>
%                      Note that the order is significant;
%                      sale is accomplished from low
%                      numbered columns to high numbered
%                      columns.
% OUTPUT: totalCost, scalar, the total cost of the
%                      groceries.
% sample call:
%  dataTable = ...
%       [0.45,2.95,1.99,4.99,4.95,3.99,7.99,16.95,11.99; ...
%           1,   1,   2,   1,   1,   2,   2,     1,   2];
%  totCost = Prob8_B_1b(dataTable)

   origCost = costAndNum_A(1,:) .* costAndNum_A(2,:);

   cost = origCost;
   totalCost = cumsum(cost);
   costAll = totalCost(end);
```

```
cost_G20_ndx = find(totalCost > 20);
if length(cost_G20_ndx) <2
    return
end
cost_G20_ndx(1) = [];
cost(cost_G20_ndx) = cost(cost_G20_ndx)*0.9;
totalCost = cumsum(cost);
costAll = totalCost(end);

cost_G50_ndx = find(totalCost > 50);
if length(cost_G50_ndx) <2
    return
end
cost_G50_ndx(1) = [];
cost(cost_G50_ndx) = origCost(cost_G50_ndx)*0.8;
totalCost = cumsum(cost);
costAll = totalCost(end);
```

Problem 8-B.2

David deposits $1,000 at Bank A and Mark deposits $1,000 at Bank B. Bank A offers a 5% annual interest rate whereas Bank B offers 8.5%.

Write a MATLAB function, **`Prob8_B_2`**, that inputs David's initial deposit, Mark's initial deposit, the interest rate (as a percentage) offered by David's bank, and the interest rate (as a percentage) offered by Mark's bank.

Your function should return the number of years for Mark's savings to become at least twice as much as David's.

Assume that the annual interest payments are deposited to the accounts at the end of each year, and that David and Mark do not deposit any additional money after their initial deposit.

```
This exercise is relatively straightforward. Students may need
to be prompted for the use of inf and return embedded in an IF.

function yearNum = ...
     Prob8_B_2(davidDep, markDep, davidIntRate, markIntRate)
% Compute the number of years for one bank account to be at
% least twice another. Only an initial deposit - no annual
% deposit. Simple interest earned at year end.
% INPUT: davidDep, scalar, amount of deposit of David
%        markDep, scalar, amount of deposit of Mark
%        davidIntRate, scalar, rate David earns (as percent)
```

```
%        markIntRate, scalar, rate Mark earns (as percent)
%                             davidIntRate < markIntRate.
% OUTPUT: numYrs, scalar, number of years at which David's
%                         account balance is at least twice
%                         that of Mark's.
% sample call:
%    yrsToDouble = Prob8_B_2(1000, 1000, 5, 8.5)

    davidIntRate = davidIntRate / 100;
    markIntRate = markIntRate / 100;
    davidSum = davidDep;
    markSum = markDep;
    for yearNum = 1:inf
        davidSum = davidSum * (1 + davidIntRate);
        markSum = markSum * (1 + markIntRate);
        if markSum > 2 * davidSum
            return
        end
    end
```

Problem 8-B.3

This following code line will generate a vector of 100 random numbers in the range [-0.5, 0.5]. (Look up **rand** in MATLAB Help.)

```
My_array = rand([1 100])-0.5;
```

Build a function, **Prob8_B_3**, that inputs a vector of 100 random numbers between -0.5 to 0.5. Your function should scan the input vector from beginning to end and count the number of times the sign changes moving from one element to the next. Consider zero to be neither positive nor negative. Hence, there are three number ranges to consider: less than zero, zero, and greater than zero.

You might find it useful to remind yourself of the answers to the following two questions: When is the product of two numbers negative? When is the product zero?

If students work the problem without taking the "hint" it will be a bit longer than if they do take the "hint." There are three states that elements of the input can be in: negative, positive, or zero. There are transitions possible from

- **negative to zero,**
- **negative to positive,**
- **zero to negative,**

```
    zero to positive,
    positive to zero, and
    positive to negative.

Thus to implement this directly would require a IF construct
with a total of six clauses. Taking the "hint" given in the
problem however makes the solution a good deal more compact, as
shown in the function below. But note in the solution given the
role that the second line of the ELSEIF clause plays: to block
the possibility of there being two successive element which are
both zero.

function numSignChanges = Prob8_B_3(rand100)
% Count the number of sign changes in the input vector.
% Elements in the input can be negative, zero, or positive.
% INPUT: rand100, vector, a vector of numbers
% OUTPUT: numSignChanges, scalar, the number of times the
%                                 sign changes from element
%                                 to the next in the input
%                                 vector.
% sample call:
%      randNums = rand([1 100])-0.5;
%      numSignChanges = Prob8_B_3(rand100)

   numSignChanges = 0;
   previousNumber = rand100(1);
   for thisNumber = rand100(2:end)
       if previousNumber*thisNumber < 0
           numSignChanges = numSignChanges + 1;
       elseif (previousNumber*thisNumber == 0) & ...
              ((previousNumber ~=0) | (thisNumber ~=0))
           numSignChanges = numSignChanges + 1;
       end
       previousNumber = thisNumber;
   end
```

Problem 8-B.4

Write a game function, **`Prob8_B_4`**, that randomly chooses an integer between 0 and 100 and asks the user to guess the number. The program should prompt "Guess the number!" until the user enters the correct number or a number outside of the range. If the guess is correct, the program displays "Good guess!" and quits. If not, the program prompts "Guess higher" or "Guess lower" according to the number guessed.

When the program terminates, the program displays the number of attempts made by the user.

Use the built-in function **input** to get input from the player. (See MATLAB Help.)

An interesting question to pose to students after they have a working solution to this problem is … what is the best strategy for a user to play the "game?" A discussion of search can following, ending with rough ideas of binary search.

If the user guess correctly the first time, or if the user terminates the first time through the loop, the message displayed by the function below is not really grammatical. An exercise extending the problem would be to correct that situation.

```
function Prob8_B_4
% An interactive game. Chooses a random number between 0 and
% 100 and asks the user to guess it.
% INPUT: none
% OUTPUT: none
% sample call:
%    Prob8_B_4

   theAnswer = round(rand*100);
   disp('------------')
   guessStr = sprintf([' GUESS AN INTEGER BETWEEN [0 100]\n',...
                      '  (input -99 to terminate)         >> ']);
   guess = input(guessStr);
   numberTries = 1;
   for i = 1:inf
       if theAnswer == guess
           disp([' CORRECT! (after ', ...
                 num2str(numberTries), ' attempts)']);
           disp('------------')
           return
       elseif (guess < 0) | (guess > 100)
           disp([' TERMINATING! (after ', ...
                 num2str(numberTries), ' attempts)']);
           disp('------------')
           return
       elseif theAnswer > guess
           guess = input(' GUESS HIGHER >> ');
           numberTries = numberTries + 1;
       else
           guess = input(' GUESS LOWER >> ');
           numberTries = numberTries + 1;
       end
   end
```

Problem 8-B.5

The Fibonacci numbers are defined by the sequence such that each value is the sum of the previous two values. The first two numbers in the Fibonacci sequence are given below:

$$F_1 = 1$$
$$F_2 = 1$$

The formula for calculating the successive numbers in the sequence is the following:

$$F_i = F_{i-1} + F_{i-2}$$

i is an integer greater than 2.

Write a function, **Prob8_B_5**, that takes one argument—a positive integer **N**—and outputs a vector of Fibonacci values so the last number in the sequence is the greatest Fibonacci number that is less than **N**.

An interesting extension to this problem is to ask students how many elements their answer vector has when the input is 23,000. Then ask them to estimate how many elements the answer vector has for input of 23,000,000. And finally, of course, have them run their answer function using input of 23,000,000. The answer for 23,000 is 22 elements; the answer for 23,000,000 is 36 elements.

A function meeting the problem spec is below.

```
function fibVector = Prob8_B_5(N)
% Computes a set of Fibonacci numbers such that the last
% element in the sequence is the largest Fibonacci number
% less than the input integer N. (N >1).
% INPUT: N, scalar, a positive integer
% OUTPUT: fibVector, vector, Fibonacci sequence with last
%                            element the large Fibonacci
%                            number less than N.
% sample call:
%    fibSeq = Prob8_B_5(22)

   fibVector(1) = 1;
   fibVector(2) = 1;
   for i = 3:inf
       fibVector(i) = fibVector(i-1) + fibVector(i-2);
       if fibVector(i) >= N
```

```
            fibVector(i) = [];
            return
        end
    end
```

Problem 8-B.6

Your computer science professor gives you an extra credit project. Your job is to write a function, **`Prob8_B_6`**, that determines the end-of-term grade for each student in the course. The input to your function will be an array of student grades. The professor also gives you the following sample input:

Student ID	Homework	Midterm	Term Project	Final
39	61	73	85	64
271	95	79	80	100
418	72	65	94	81
665	83	86	82	92
860	25	40	93	85

All grades will be weighted equally in determining the final grade:

$$overallGrade = \frac{homework + midterm + termpProj + final}{4}$$

The professor promises the students to take individual progress into account at the end of the term. If the term project grade is at least 15 points higher than the homework grade, and the final grade is at least 10 points higher than the midterm grade, you must calculate the overall grade by ignoring their homework and midterm scores:

$$overallGrade = \frac{termpProj + final}{2}$$

Your function takes an *N* x 5 array and outputs an *N* x 2 array. The first and second columns in the output should be the student ID and end-of-term grade, respectively. *N* is a variable, i.e., your function should work for any number of students.

```
The solution below uses the MATLAB capability to iterate over
entire columns of an array. When processing data arrays in which
an entire row or column is one "observation," this capability
can be useful.

A solution for this problem is below.

function termGrades = Prob8_B_6(studentGrades)
% Compute term grades for a group of students. Term grade
% is average of grades in homework, midterms, term
% project, and final exam. If term project is at least 15
% pts better than homework, AND the final exam is at least
% 10 pts better than the midterm, THEN ignore the homework
% and midterm scores.
% INPUT: studentGrades, an N x 5 array where each row is
%        <ID> <homework> <midterm> <term proj> <final exam>
% OUTPUT: termGrades, an N x 2 array where each row is
%        <ID> <term grade>
% sample call:
%   grades = [ 39,  61,  73,  85,  64;
%              271,  95,  79,  80, 100;
%              418,  72,  65,  94,  81;
%              665,  83,  86,  82,  92;
%              860,  25,  40,  93,  85];
%    termGrade = Prob8_B_6(grades)

  termGrades = [];
  for oneGradeSet = studentGrades'
      if (oneGradeSet(4) >= 15 + oneGradeSet(2)) & ...
         (oneGradeSet(5) >= 10 + oneGradeSet(3))
          oneTermGrade = (oneGradeSet(4) + oneGradeSet(5))/2;
      else
          oneTermGrade = sum(oneGradeSet(2:end))/4;
      end
      termGrades = [termGrades; oneGradeSet(1), oneTermGrade];
  end
```

Problem 8-B.7

A mathematical sequence, *f*, is defined this way:

$$f(n) = f(n-1) + f(n-1) \times f(n-2) \quad \text{for n} > 2,$$

$$f(1) = 1$$

$$f(2) = 1$$

Write a function, **Prob8_B_7**, that inputs a positive integer **N** as input and outputs the first **N** numbers in this sequence as a vector.

Test your function with the following:

- For an input of 1, the output should be **1**
- For an input of 2, the output should be **[1 1]**
- For an input of 3, the output should be **[1 1 2]**
- For an input of 4, the output should be **[1 1 2 4]**

A function meeting the code specs for this problem is below.

```
function f = Prob8_B_7(N)
% Create the first N terms in a sequence defined by
%   f(1) = 1, f(2) = 1,
%   f(n) = f(n-1) + f(n-1)* f(n-2) for n>2
% INPUT: N, scalar, a positive integer
% OUTPUT: seqVec, vector, first N terms in the sequence
%                         defined above
% sample call:
%    sequence = Prob8_B_7(5)
   if N == 1
      f(1) = 1;
   elseif N ==2
      f(1) = 1; f(2) = 1;
   else
      f(1) = 1; f(2) = 1;
      for i = 3:inf
         if i > N
            return
         else
            f(i) = f(i-1) + f(i-1)* f(i-2);
         end
      end
   end
```

Problem 8-B.8

The half-life of a radioisotope is the time required for half of the radioactive mass in any sample to undergo radioactive decay. In a radioactive sample, after one half-life, half of the original sample remains. After three half-lives, one-eighth of the original mass remains.

Write a MATLAB function, **Prob8_B_8**, that determines how many years it would take for **N** atoms of radioactive material, with half-life of **HL** years, to be reduced to a level at or below the target threshold of **T** atoms. (Normally, we would use the mass of the radioactive material, but here we will use the number of atoms.)

This function will have three inputs:

1. The total number of radioactive atoms we start with (integer)
2. The length of the half-life (in years)
3. The target threshold number of atoms (integer)

It will have two outputs:

1. **numYears**—The total number of years it will take for the radioactive material to decay to a level at or below the target threshold
2. **atomsLeft**—The number of atoms left after **numYears**

One assumption is made in the solution below that is not obvious: the number half-lives will be an integer. It would be a reasonable exercise to ask students to write down all assumptions they make after they have a working solution to the problem.

A function meeting the specs of this problem is below.

```
function [yrs, atomsLeft] = ...
                  Prob8_B_8(startAtoms, halfLife, tarAtoms)
% Computer the number of years for a radioactive sample to
% decay to a target level of radiation. Assumes that the
% answer will be in terms of a whole number of half lives.
% INPUT: startAtoms, scalar, number of atoms at time = 0 yrs
%        halfLife, scalar, number of yrs to halve the number
%                          of atoms
%        tarAtoms, scalar, number of atoms we must be either
%                         at or below
% OUTPUT: yrs, scalar, number of years to reach tar Atoms or
%                      below
%         atomsLeft, scalar, number of atoms left are yrs
% sample call:
%    [Y, atoms] = Prob8_B_8(10^23, 2*10^2, 10^4)

   atomsLeft = startAtoms;
   for halfLifeCounter = 1:inf
      atomsLeft = .5 * atomsLeft;
      if atomsLeft <= tarAtoms
```

```
            yrs = halfLifeCounter * halfLife;
            return
        end
    end
```

Problem 8-B.9

As the ecological supervisor for the Great Hitonkin Forest, you are contracted by a lumber company to find out the number of trees it will be allowed to cut down over the upcoming years. The number of trees a lumber company is allowed to chop in a given year can be modeled as 0.5 times the number of seedlings planted the previous year minus 0.1 times the number of trees chopped down the previous year.

The company provides you with the following data on the number of seedlings it will plant over the next 10 years:

```
seedlingPlanted = ...
[2700, 3000, 3000, 3200, 4000, 3700, 3300, 4200, 4400, 4100]
```

The lumber company tells you it plans to chop down 3,000 trees this year.

Write a MATLAB function, **`Prob8_B_9`**, that inputs the following:

- The number of trees chopped down in the first year
- A vector for the number of seedlings that will be planted in the next ten years

Your function should plot the projected number of trees the lumber company is allowed to chop down in the next 10 years. (Be sure to label your plot well because it will be part of your report to the lumber company.)

This problem is a straightforward inventory simulation problem. The main hurdle for students is to take the "word description" given in the problem spec and turn it into the core simulation line: cuttings(yr) = 0.5*plantingV(yr-1) - 0.1*cuttings(yr-1).

The solution below creates a bar graph, although a line plot is included as lines that are "commented out." The reason that a bar chart is preferable is that the cutting activity extends over an entire year rather than happening at a given time instant. Thus the bar chart is a more accurate reflection of the situation being modeled, and thus a better way to transfer the information on cuttings.

```
Most students will produce a line plot to meet the specification
for this problem. A discussion on the fundamental purpose of
graphs (to convey information) can ensue, and lead to the bar
chart as the best vehicle for this problem.

A function meeting the specification for this problem is below.


function Prob8_B_9(treesCutYr1, planting)
% Computes the number of trees to be harvested in the next
% 10 years in the Hitonkin Forest. Relation for cuttings is
%   cutting(YR) = 0.5*planting(YR-1) - 0.1*cutting(YR-1)
% INPUT: treesCutYr1, scalar, number of trees cut in Yr 1
%        plantingV, vector, 10 elements, number of trees
%                           plants in each of next 10 years.
% OUTPUT: none
% generates a graph of number of trees to harvest vs time
% sample call:
%   yr1_cut = 3000;
%   plantings = 10^3*[2.7,3,3,3.2,4,3.7,3.3,4.2,4.4,4.1];
%   Prob8_B_9(yr1_cut, plantings)

   cuttings(1) = treesCutYr1;
   for yr = 2:10;
      cuttings(yr) = 0.5*plantingV(yr-1) - 0.1*cuttings(yr-1);
   end

   figure
%     plot(1:10, cuttings, ...
%                 '--ro','LineWidth',2,...
%                 'MarkerEdgeColor','k',...
%                 'MarkerFaceColor','g',...
%                 'MarkerSize',10)
     bar(0.5:1:9.5, cuttings)
     grid on
     xlabel('year number', 'FontWeight', 'bold')
     ylabel('number of trees to harvest')
     title('Number of Trees to Harvest  vs. Year Number', ...
         'FontSize', 14)
```

Problem 8-B.10

Elaine opens a savings account with a 6% interest rate and deposits $10,000. In the next 30 years, Elaine does the following:

1. She deposits $10,000 at the beginning of every year.

2. In years when she has more than $100,000, she deposits $10,000 and then withdraws 10% of her total balance to buy mutual funds.

Write a MATLAB function, **`Prob8_B_10`**, that has no inputs or outputs. The function should plot Elaine's account balance over the next 30 years. Do not include the value of her mutual funds.

This is another bank account problem.

The problem specification is left purposefully unclear on one point: whether at the start of each year the deposit of $10,000 is made first, or the test if the current balance is over $100,000 is made first. On balance the spec seems to indicate the former. But students should be encouraged to note this kind of looseness in a problem specification, and encouraged further to seek clarification.

An interesting extension to this problem is to ask students to characterize the plot they create. In fact, you cannot see enough "data" to make that characterization definitively. However, if the number of years is changed to 100, the nature of this model because clear: there is an approach to an asymptotic value.

A function meeting the specs of the problem is below.

```
function Prob8_B_10
% plots value of bank account with:
%   6% interest rate
%   $10^4 initial deposit
%   time is over 30 yrs
%   rule for deposit/withdrawal...
%           deposit 10^4 start of each year
%           if bal > $10^5,
%            withdraw 10% balance to invest
% INPUTS: none
% OUTPUTS: none
% generates a graph per above
%    NOTE: plot not to include value of mutual funds
% sample call:
%     Prob8_B_10

   numYrs = 30;
   intRate = .06;
   yearlyDeposit = 10^4;

   startBal(1) = yearlyDeposit;
   for yrNum = 2:numYrs+1
       startBal(yrNum) = startBal(yrNum-1)*(1+intRate);
```

```
        startBal(yrNum) = startBal(yrNum) + yearlyDeposit;
        if startBal(yrNum) > 10^5
            startBal(yrNum) = startBal(yrNum)*0.9;
        end
    end

    figure
      plot(0:numYrs, startBal, '--ro', ...
             'LineWidth',2,...
             'MarkerEdgeColor','k',...
             'MarkerFaceColor','g',...
             'MarkerSize',6)
      grid on
      xlabel('number of years', 'FontWeight', 'bold')
      ylabel('account balance ($)', 'FontWeight', 'bold')
      title('Elaine''s Bank Account Over Time', 'FontSize', 14)
      disp('click to place annotation')
      gtext('markers indicate value at START of year',...
             'FontWeight', 'bold')
```

Problem 8-B.11

The Registrar's Office wants you to project the number of graduating students in the upcoming years. The total number of graduating students can be modeled this way:

> The number of senior students who did not fulfill their requirements in the previous year (we are going to assume that they finish in one additional year)
>
> PLUS
>
> The number of new enrollments in senior class in the current year
>
> MINUS
>
> The number of senior students who did not fulfill their requirements in the current year.

The Registrar's Office gives you the projected new enrollment in the senior class and the projected number of students that will not fulfill their requirements. This projected data are for each of the next 15 years:

```
projectedEnrollment = ...
[95, 81, 76, 88, 96, 121, 133, 112, 91, 89, 79, 74, 91, 121, 75]
```

```
projectedStudentsNotFulfilling = ...
[5, 6, 3, 10, 7, 4, 6, 8, 8, 3, 4, 3, 2, 10, 8]
```

The Registrar indicates that a total of 90 students graduated at the end of the first year. Write a MATLAB function, **Prob8_B_11**, that inputs the **studentsGraduatingYear1**, the **projectedEnrollment**, and the **projectedStudentsNotFulfilling**. It should have no output variables. Your function should plot the projected number of students graduating over the next 15 years.

```
This problem is similar to B.9 above. In the abstract, both can
be thought of as inventory problems. Although not the focus of
this problem, note that the function below is a solution going
beyond the problem specification in that the function will per-
form correctly on any number of years - provided the two vector
inputs are appropriate: the number of elements in each of the
input vectors is equal to the number of years over which the
simulation is to be run.

function Prob8_B_11(studentsGrad_yr1, projEnrollV, projNotFulV)
% create a plot of modeling results for students graduating
% over the next 15 years. Modeling relation is as follows.
%  numGrad(YR) = projNotFulfill(YR-1) ...
%                   + projEnrollV(YR) ...
%                   - projNotFulV(YR)
% INPUT: studentsGrad_yr1, scalar, number of Yr1 graduates
%        projEnrollV, vector, N elements, the enrollment
%                     projected for the senior class
%        projNotFulV, vector, N elements, number of
%                     students not meeting requirements
% generates a graph of number graduated for N years.
% sample call:
%  yr1_grads = 90;
%  enrollment = [95, 81, 76, 88, 96, 121, 133, ...
%                112, 91, 89, 79, 74, 91, 121, 75];
%  notFulfilling = [5, 6, 3, 10, 7, 4, 6, 8, 8, ...
%                   3, 4, 3, 2, 10, 8];
%  Prob8_B_11(yr1_grads, enrollment, notFulfilling)

       numGrad(1) = studentsGrad_yr1;
       numYears = length(projEnrollV);
       for yrCounter = 2:numYears
           numGrad(yrCounter) = projNotFulV(yrCounter - 1) ...
                              + projEnrollV(yrCounter) ...
                              - projNotFulV(yrCounter);
       end
       figure
         plot(1:numYears, numGrad, ':ro', ...
                                   'LineWidth',2,...
```

```
                                'MarkerEdgeColor','k',...
                                'MarkerFaceColor','g',...
                                'MarkerSize',6)
grid on
xlabel('number of year', 'FontWeight', 'bold')
ylabel('projected number of graduates', ...
          'FontWeight', 'bold')
title('Projected Number of Graduates Over Time',...
      'FontSize', 14)
```

Problem 8-B.12

Your engineering expertise is requested by the Vandelay Truck Company. Truck prices are projected to increase next year, so Vandelay would like to keep at least 1,000 trucks in the warehouse until the prices go up. The capacity of the warehouse is 1,500 trucks, never to be exceeded. The following model for warehouse management has been developed:

- The production schedule for the next twelve months is **`[130 90 105 152 174 146 168 194 203 185 164 191]`**. All manufactured trucks will be stored in the warehouse until they are sold.
- Vandelay is planning to sell 15% of inventory once more than 1,050 trucks are in the warehouse. Otherwise, no trucks will be sold.
- Current inventory is 850 trucks.

Vandelay would like to know if inventory will fall below 1,000 or exceed 1,500. Create a plot of inventory over the next 13 months.

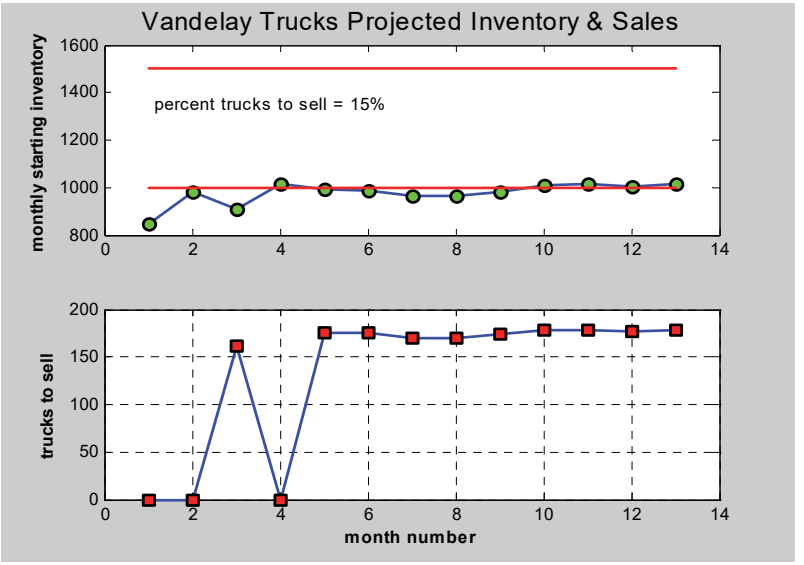
For quick and easy plot generation, write a MATLAB function, **`Prob8_B_12`**. Your function should input the percentage of trucks that will be sold (15% in this case) when inventory goes over 1,050, and output a vector of inventories. It should plot a graph of inventory for the next 13 months.

Assume that the trucks produced in one month are transferred to the warehouse at the beginning of the next month. Start with a beginning inventory of 850 and run your simulation from month 2 to month 13.

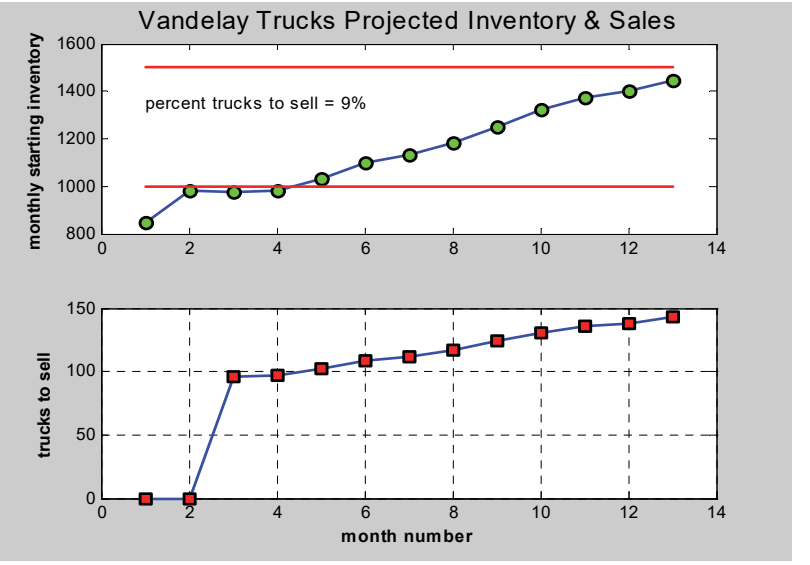
Hint: 15% of inventory will not always be an integer. Use built-in function **`round`** to round the numbers to nearest integer.

This problem on the surface is straight forward. A function that meets the problem specification is below. The function goes beyond the problem spec per se by making a subplot of both the inventory of Vandelay Trucks and the sales number (number of trucks that may be sold) of Vandelay Trucks. An instructor initiated discussion might be held on the subject of what a user really "needs" versus what a user has written in a problem specification. Note that in this problem, it might well be the case that management of Vandelay Trucks will be interested in how many cars they may sell in a given month. Note also that upper and lower bounds on the inventory as set in the problem are also shown in the graphic – again despite this feature not being an explicit requirement in the problem.

Calling the function below with the "sales threshold" noted in the problem (15%) produces the graphics window plots as follows.



A "sales threshold" of 15% holds inventory close to 1,000 trucks, although inventory takes two dips below 1,000 trucks. Suppose that management at Vandelay Trucks means that lower bound of 1,000 trucks to be more closely adhered to. A "sales threshold" of 9% produces the following graphic plots.



At a "sales threshold" of 9%, the lower bound of 1,000 trucks is more closely held; only one minor deviation is noted for months 2-4. The 9% value results in lower numbers of cars to sell, but also in an increasing inventory of cars as the year progresses. Is this what Vandelay Trucks wants? The problem spec does not specify.

The point a discussion following the above lines is to yet again encourage students to question a problem spec.

Code follows ….

```
function inventoryV = Prob8_B_12(percentTrucksSell)
% Computes the month by month starting inventory for
% Vandelay Truck by doing a simulation. Strategy for selling
% is ... SELL 15% of trucks if inventory is more than 1050
% trucks.
% INTERNALLY SET: startingInventory, scalar,
%                              the starting truck inventory
%                 productionSchedule, vector, 12 element
%                              vector representing truck
%                              production over next year
%                 thresholdForSale, scalar, threshold for
%                                              selling trucks
% INPUT: percentTrucksSell, scalar, threshold for triggering
%                                 sales - initially 15%
% OUTPUT: inventoryV, 13 element vector, the starting
%                  inventory for each of the next 13 months,
%                  with the current month being numbered 1
% creates a plot for inventoryV
% sample call:
%     inventory = Prob8_B_12(0.15)
```

```
% SET INTERNAL CONSTANTS
  startingInventory = 850;
  productionSchedule = [130 90 105 152 174 146 168 ...
                        194 203 185 164 191];
  thresholdForSale = 1050;
  lenSim = length(productionSchedule)+1;
  lowerBound = 1000;
  upperBound = 1500;

% SET UP AND PERFORM THE SIMULATION
  inventoryV(1) = startingInventory;
  for month = 2:lenSim
      onHand = inventoryV(month-1) ...
                     + productionSchedule(month-1);
      if onHand >thresholdForSale
          sellNumber(month) = percentTrucksSell * onHand;
          sellNumber(month) = round(sellNumber(month));
      else
          sellNumber(month) = 0;
      end
      inventoryV(month) = onHand - sellNumber(month);
  end

  figure
    subplot(2,1,1)
    plot(1:lenSim, inventoryV, '--bo', ...
              'LineWidth',2,...
              'MarkerEdgeColor','k',...
              'MarkerFaceColor','g',...
              'MarkerSize',10)
     hold on
     plot(1:lenSim,lowerBound*ones(1,lenSim), '-r',...
               'LineWidth', 2)
     plot(1:lenSim,upperBound*ones(1,lenSim), '-r',...
               'LineWidth', 2)
     title('Vandelay Trucks Projected Inventory & Sales',...
           'FontSize', 14)
     ylabel('monthly starting inventory', ...
                'FontWeight', 'bold')
     annotStr = ['sales threshold = ', ...
                   num2str(percentTrucksSell*100), '%'];
     disp('place an annotation')
     gtext(annotStr)

     subplot(2,1,2)
     plot(1:13, sellNumber, ':bs', ...
              'LineWidth',2,...
              'MarkerEdgeColor','k',...
              'MarkerFaceColor','r',...
              'MarkerSize',10)
      ylabel('trucks to sell', ...
                'FontWeight', 'bold')
      xlabel('month number', 'FontWeight', 'bold')
```

```
grid on
```

Problem 8-B.13

Dr. Crane opens a bank account with an initial deposit of $2,125. He will receive 6.25% interest, compounded quarterly. Right after he deposits the money, he moves to Paris and stays there for 19 years. He does not make any deposits or withdrawals during this time.

Nineteen years later he returns home and goes to collect the money from his account. How many dollars in interest has his account earned over the last 19 years? (Interest is compounded four times every year, which you can assume means that for every three months, 6.25%/4 interest is paid to Dr. Crane's account.)

Write a Matlab function that will take the initial deposit, number of years, and the interest rate (in percentage) as inputs. The only out-put will be the interest he earned over the years he was gone.

A function meeting this problem specification is below.

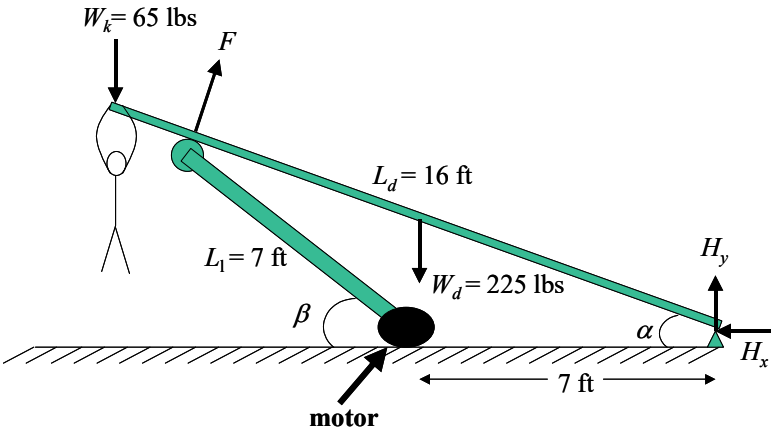
```
function interestEarned = ...
             Prob8_B_13(initDep, numYrs, interestRatePerCent)
% computes interest earned for a bank account with an
% initial deposit but now further deposits, over a set
% number of years, and with a set annual interest rate
% compounded quarterly.
% INPUT: initDep, scalar, $ of the initial deposit
%        numYrs, scalar, the number of year the account "sits"
%        interestRatePerCent, scalar, annual interest rate
% OUPPUT: interestEarned, scalar, the total amount of money
%                         earned over numYrs
% sample call:
%    earned = Prob8_B_13(2125, 19, 6.25)

   threeMonthIntRate = interestRatePerCent*.01/4;
   interestEarned = 0;
   accountBal = initDep;
   for quartNumber = 1:numYrs*4
       interestThisQuarter = accountBal * threeMonthIntRate;
       accountBal = accountBal + interestThisQuarter;
       interestEarned = interestEarned + interestThisQuarter;
```

```
end
```

Problem 8-B.14

The figure below shows an electric motor-powered underground garage door. The door weighs 225 pounds and is 16 feet long. The lever that lifts the door is 7 feet long and has negligible weight. A child hangs on the tip of the door as the door moves up.



The magnitude of torque produced by the electric motor can be calculated with the following:

$$\tau = F \times L_l \times sin\left(90 - \frac{\beta}{2}\right)$$

F is the force applied on the door by the lever and L_l is the length of the lever.

We can calculate the force F by calculating the moments around the hinge:

$$F = \frac{\left(W_d \frac{L_d}{2} + W_k L_d\right) cos^2\left(\frac{\beta}{2}\right)}{L_l \left(1 + cos(\beta)\right)}$$

W_d and L_d are the weight and length of the door, respectively, W_k is the weight of the child hanging on the door, L_l is the length of the lever, and β is the angle of the lever with the horizontal.

The electric motor rotates the lever at a rate of 3 degrees/sec. The door rotates freely ($W_k = 0$) until $t = 10$ seconds. The child clings to the door from $t = 10$ to $t = 20$ seconds. At $t = 20$ seconds, the child gets scared and lets go.

Write a MATLAB function, **`Prob8_B_14`**, that inputs the time the child first grabs the door and the time he eventually releases it. Your function should use a **FOR LOOP** to make subplots of the force applied by the lever versus time and torque produced by the electric motor versus time from **`t=0`** seconds to **`t=60`** seconds.

This problem is relatively straightforward if the problem specs are carefully followed.

Code for the problem as written is below. There is also a second version of a solution below that uses vector operations rather than a loop. A good additional exercise would be to require a vector solution of students.

```
function Prob8_B_14(timeKidOn, timeKidOff)
% Creates subplots force and torque given the problem spec
% 8-B.14. This version uses a FOR loop.
% INPUT: timeKidOn, scalar, the elapsed time at which a child
%                           jumps on the end of the door being
%                           opened
%        timeKidOn, scalar, the elapsed time at which a child
%                           lets go of the door
% OUTPUT: none
% creates a subplot of force VS time, and torgue VS time
% sample call:
%     Prob8_B_14(10, 20)
% uses Prob8_B_14_graph

  % set up internal constants
    weightDoor = 225;          % lbs
    lengthDoor = 16;           % feet
    lengthLever = 7;           % feet
    weightChild = 65;          % lbs

  i = 0;
  for time = linspace(0,60,1000)
   i = i + 1;
   t(i) = time;
   beta = 3 * (pi/180) * time;        % radians
   if ~(timeKidOn <= time & time <= timeKidOff)   % kid is off
     force(i) = (0.5*weightDoor*lengthDoor*(cos(beta/2))^2 ) ...
              / (lengthLever*(1+cos(beta))); % force with no kid
   else   % kid is on
     force(i) = ((0.5 * weightDoor * lengthDoor ...
```

```
                              +  weightChild * lengthDoor) ...
                      * (cos(beta/2))^2 ) ...
                  / (lengthLever*(1+cos(beta)));
   end

   torque(i) = force(i) * lengthLever .* sin((pi-beta)/2);
  end

  Prob8_B_14_graph(t, force, torque)


% ----------------------------------------
function Prob8_B_14_graph(time, force, torque)
% creates the subplots called for in Prob8-B.14

  figure
    subplot(2,1,1)
    plot(time,force, 'LineWidth', 3)
    title('Force/Torque vs Time for Prob 8-B.16', ...
          'FontSize', 14)
    ylabel('force (lbs)', 'FontWeight', 'bold')
    xlabel('time (sec)',  'FontWeight', 'bold')
    grid on

    subplot(2,1,2)
    plot(time,torque, 'LineWidth', 3)
    ylabel('torque (ft-lbs)', 'FontWeight', 'bold')
    xlabel('time (sec)',  'FontWeight', 'bold')
    grid on



% ----------------------------------------
function Prob8_B_14b(timeKidOn, timeKidOff)
% Creates subplots force and torgue given the problem spec
% 8-B.14. This version uses vector computations.
% INPUT: timeKidOn, scalar, the elapsed time at which a child
%                           jumps on the end of the door being
%                           opened
%        timeKidOn, scalar, the elapsed time at which a child
%                           lets go of the door
% OUTPUT: none
% creates a subplot of force VS time, and torgue VS time
% sample call:
%     Prob8_B_14b(10, 20)
% uses Prob8_B_14_graph

  % set up internal constants
    weightDoor = 225;         % lbs
    lengthDoor = 16;          % feet
    lengthLever = 7;          % feet
    weightChild = 65;         % lbs
```

```
time = linspace(0,60, 1000);
timeKidOff_ndx = find(time < timeKidOn  |  time > timeKidOff);
timeKidOn_ndx = find(timeKidOn <= time & time <= timeKidOff);

beta = 3 * (pi/180) * time;        % radians

force = ( 0.5*weightDoor*lengthDoor*(cos(beta/2)).^2 ) ...
          ./ (lengthLever*(1+cos(beta))); % force with no kid
force(timeKidOn_ndx) = ...
    ((0.5*weightDoor*lengthDoor + weightChild*lengthDoor) ...
                      * (cos(beta(timeKidOn_ndx)/2)).^2 ) ...
            ./ (lengthLever*(1+cos(beta(timeKidOn_ndx))));
                     % force with kid on
torque = force * lengthLever .* sin((pi-beta)/2);

Prob8_B_14_graph(time, force, torque)
```

Problem 8-B.15

The bisection method is a simple way of finding roots of polynomials numerically. Bisection method starts with two guesses, x_1 and x_2, such that:

$$f(x_1) < 0$$

$$f(x_2) > 0$$

In such a case, the "zero value" of the polynomial (i.e., the root), must lie between x_1 and x_2. To close in on the root, we calculate the average of x_1 and x_2:

$$x_3 = \frac{x_1 + x_2}{2}$$

and we evaluate the polynomial at x_3. According to the value of $f(x_3)$, we make one of the following decisions:

- If $f(x_3)$ is greater than zero, the root must lie between x_1 and x_3, so we replace x_2 with x_3.
- If $f(x_3)$ is less than zero, the root must lie between x_2 and x_3, so we replace x_1 with x_3.
- If $f(x_3)$ is zero, x_3 is a root, so we quit.

We continue the "average-replace" strategy until we find a root or the difference between x_1 and x_2 is below a tolerance.

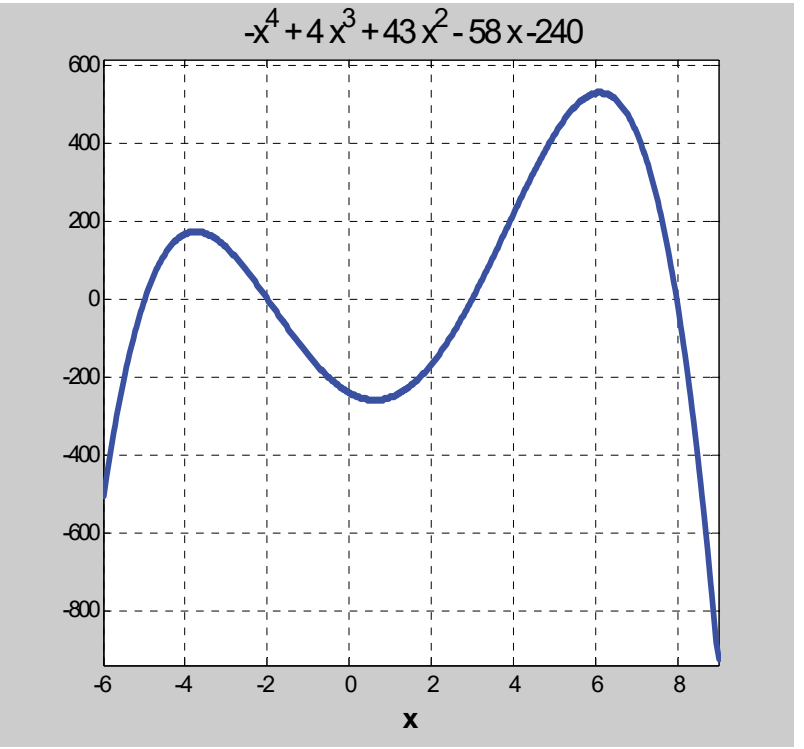
Use the Bisection Method described above in a function, **Prob8_B_15**, to find a root between **x = 0** and **x = 5** for the following polynomial. Set the tolerance in your solution to 10^{-3}.

$$f(x) = -x^4 + 4x^3 + 43x^2 - 58x - 240$$

This problem will be the most difficult in this chapter for most students. The difficulty is translating the algorithm given in the problem spec into MATLAB code. The real hurdle lies in understand the problem spec itself. Most students will need to work this problem out with paper and pencil for a simple case before they try to implement a solution in MATLAB. An additional aid is to use ezplot to create a functional graph for the polynomial specified:

```
>> eS = '-x^4 + 4*x^3 + 43*x^2 - 58*x -240';
>> ezplot(eS, [-6,9])
```

… to produce the following:



Note that the four roots of the polynomial are x=8, x=3, x=-5, and x=-2. Note also that the conditions necessary for the

bisection method as stated in the problem spec are met: If F(x) is the polynomial, then F(0) < 0, and F(5) > 0. Finally, note that there is a root in the (0,5) interval. The bisection method as given in the problem specification will find that root.

A set of two functions that meets the explicit problem specification is below. One of the two, Prob8_B_15b, evaluates a polynomial F(x) for a given value of x. The second, Prob8_B_15, implements the bisection algorithm as specified.

The bisection method given in the problem is capable of finding two of the four roots: x=3 and x=-5. The other two roots, x=8 and x=-2, cannot be found with the algorithm given; the assumptions of the method given are not met.

A challenging extra credit problem would be to enhance the bisection method as given in the problem specification so that all roots would be potentially accessible to the extended method.

The extension could distinguish three cases. If F(x) is the polynomial, the lower limit on the interval is a, and the upper limit on the interval is b, then the three cases are as follows:

1) F(a) < 0 AND F(b) > 0 … this is the case covered in the algorithm stated in the problem specification. For
F(x) = -x^4 + 4*x^3 + 43*x^2 - 58*x -240
roots at x=3 and x=5 can be found under this case.

2) F(a) > 0 AND F(b) < 0 … this is the case that can cover the two roots of the polynomial at x=8 and x=-2.

3) F(a) and F(b) are of the same sign … This means that a new pair of boundaries a and b must be chosen. The simple solution to this case is to "punt" – create an error as in the code below. A more sophisticated handling would be to generate a new guess for a and b.

MATLAB includes a rich set of built-in function that operate on polynomials. Two of particular note here are "roots," which finds all roots of a given polynomial, and "polyval," which evaluates a polynomial F(X) at a given x.

Code for Prob8_B_15 and Prob8_B_15b is below.

```
function oneRoot = Prob8_B_15(polyCoeff, tolerance, bounds)
% Compute a single zero for a polynomial in an interval using a
% bisection method. The polynomial is represented by a vector of
% its coefficients (from  is represented by a vector of its
% coefficients (from highest to lowest). Let F(x) be the
% polynomial. Assumption is that F(bounds(1) < 0, and
% F(bonuds(2) > 0.
```

```
% INPUTS polyCoeff, vector, polynomial coefficients from
%                           highest power to constant.
%        tolerance, scalar, acceptable tolerance for
%                           identifying a root
%        bounds, two element vector, lower and upper range
%                                    bound
% OUTPUT: oneRoot, scalar, one root of the polynomial in the
%                  given range
% sample call:
%    coef = [-1, 4, 43, -58, -240];
%    limits = [0, 5];
%    tol = 10^-3;
%    aRoot = Prob8_B_15(coef, tol, limits)

  x1 = bounds(1);
  x2 = bounds(2);
  x3 = mean([x1,x2]);

% make sure assumptions of method are realized
  fLower = Prob8_B_15b(polyCoeff, x1);
  fUpper = Prob8_B_15b(polyCoeff, x2);
  if (fLower >= 0) | (fUpper <= 0)
      error('  !!! assumptions of method not met')
  end

  for i = 1:inf
      f_x1 = Prob8_B_15b(polyCoeff, x1);
      f_x2 = Prob8_B_15b(polyCoeff, x2);
      f_x3 = Prob8_B_15b(polyCoeff, x3);

      [1,3,2; x1, x3, x2; f_x1 f_x3 f_x2]

      if abs(x1-x2) <  tolerance
          oneRoot = x3;
          return
      elseif f_x3 == 0
          oneRoot = x3;
          return
      elseif f_x3 > 0
          x2 = x3;
      elseif f_x3 < 0
          x1 = x3;
      end
      x3 = mean([x1,x2]);
  end

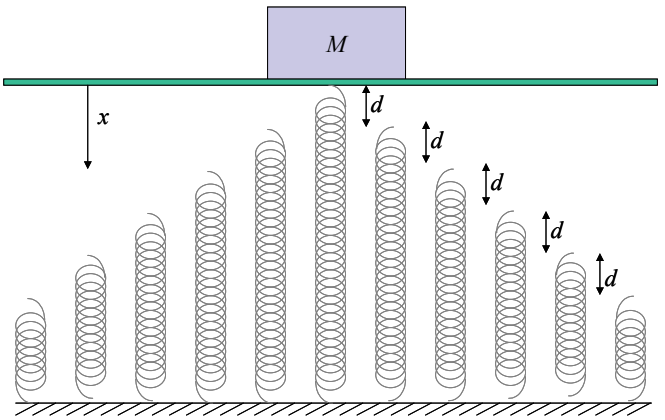

function valOfPoly = Prob8_B_15b(coef, x)
% evaluate a polynomial with known coefficients at a given
% point
% INPUT: coef, vector of coefficients from highest power to
%              the coefficient for the constant term
%        x, scalar, point at which the polynomial is
%                   to be evaluated
```

```
% OUTPUT: valOfPoly, scalar, the computed evaluation of the
%                    polynomial
% sample call:
%      coef = [-1, 4, 43, -58, -240];
%          This represents a polynomial
%                -x^4 + 4x^3 + 43x^2 -58x -240
%      x = 2;
%      valOfPoly = Prob8_B_15b(coef, x)

   for i = length(coef):-1:1
       powersX(i) = x^(i-1);
   end
   powerX = powersX(end:-1:1);
   valOfPoly = sum(coef .* powerX);
```

Problem 8-B.16

A mass-spring system is set up as shown in the picture below:



This arrangement shows 11 springs: one that touches the platform and 5 pairs of springs with *d* meters difference in heights. When mass *M* is supported by the platform, the platform will move through a distance *x* until the weight is balanced by the force of some number of the springs. The number of springs that it takes to balance the weight depends on the mass being supported by the platform (*M*).

A spring exerts a force proportional to its compression. The constant of proportionality is called the "spring constant." For a single spring of spring constant *k*, supporting a mass of *M*, the balance of forces (gravitational VS spring compressional) is as follows, given *g* as the acceleration due to gravity, 9.81 m/s^2:

$$Mg = kx \qquad if\ x \le d$$
$$or \ldots x = \frac{Mg}{k}$$

If the mass being supported requires both the center spring and the first spring pair out from the center in order to support it, then the balance of forces gives us:

$$Mg = kx + 2k(x - d) \qquad if\ d < x \le 2d$$
$$or \ldots x = \frac{Mg + 2kd}{3k}$$

Write a MATLAB function, **`Prob8_B_16_ver1`**, that inputs *M*, *d*, and *k* and outputs the loaded position of the platform (*x*). Assume only three springs are available: the center spring and the spring pair closest the center spring.

Try your **`Prob8_B_16_ver1`** with inputs *M*=10 kg, *d*=0.2 m, and *k*=200 N/m. (Your answer should be *x*=0.29667 m.)

Now, generalize your solution to *N* pairs of springs, with spring pair *n*=0 to mean the *single* center spring, then count pairs of springs out from the center. When spring pair *n* is supporting the platform, the loaded position of the platform (*x*) can be calculated using:

$$x = \frac{Mg + n(n+1)kd}{(2n+1)k} \qquad if \ldots nd < x \le (n+1)\,d$$

Convince yourself that the above general relation is correct, and be prepared to convince your instructor.

Notice that you have to know which region the platform comes to rest in before you can compute its position. This seems circular: you have to know the position to compute the position!

But think of solving the general problem as follows. Assume the platform comes to rest in the region supported by only one spring, and compute the loaded position. If the loaded position turns out to be greater than the region of one spring only, then recompute with three springs. And continue until the loaded position that you calculate meets the assumption for which region the platform will be in when loaded.

Now, starting from your function **Prob8_B_16_ver1**, create a new function **Prob8_B_16_ver2** that takes inputs of *N*, *M*, *d*, and *k* and outputs (a) the loaded position of the platform (*x*), and (b) the number of the spring pair the last to engage in supporting the platform. Be careful to understand what *N* stands for: the number of spring pairs in a system. Note again that *N*=0 represents the center spring.

Try your function with inputs that you used before for version 1; the results for version 1 should agree with the results for version 2.

Now try inputs of N=11, M=100 kg, d=0.2 m, k=200 N/m. (Your answers should be x=1.9789 m, and springNumber=9.)

Most students will find this problem challenging. The main hurdle for students is to understand the physical situation and the problem specification.

Functions implementing a solution for this problem follow.

```
function x = Prob8_B_16_ver1(M, d, k)
% For a set of 2N + 1 springs organized as show in the figure
% associated with the problem spec, determine the value of X for
% given values of (a) supported mass (M), (b) no-load end point
% separations of the spring pairs (d), and same spring constants
% for all springs (k)
% INPUT: M, scalar, the mass of the object supported (kg)
%        d, scalar, the no-load distances between spring pairs
%        k, scalar, the spring constant for all springs
% OUTPUT: x, scalar, the loaded position of the platform
% INTERNAL CONSTANT: g, scalar, accel due to gravity = 9.8 m/s^2
% sample call:
%    equibilbriumHeight = Prob8_B_16_ver1(10, 0.2, 200)

   % set local constants
     g = 9.8;              % m/s^2

   solutionWithCenterSpringOnly = weight/k;
   if solutionWithCenterSpringOnly <= d
       x = solutionWithCenterSpringOnly;
   else
       x = (M*g + 2*k*d) / (3*k);
   end
```

```
function [x, n] = Prob8_B_16_ver2(N, M, d, k)
% For a set of 2N + 1 springs organized as show in the figure
% associated with the problem spec, determine the value of X
% for given values of (a) supported mass (M), (b) no-load
% end point separations of the spring pairs (d), and same spring
% constants for all springs (k). Return as well the number of
% the spring pair last engaged to support the platform.
% INPUT: N, scalar, 0 or positive integer, num of spring pairs
%        M, scalar, the mass of the object supported (kg)
%        d, scalar, the no-load distances between spring pairs
%        k, scalar, the spring constant for all springs
% OUTPUT: x, scalar, the loaded position of the platform
```

```
%         n, scalar, the spring pair that is the last to engage
% INTERNAL CONSTANT: g, scalar, accel due to gravity = 9.8 m/s^2
% sample call:
%   [loadedX, springPairNumber] = Prob8_B_16_ver2(11,100,0.2,50)

   % set local constants
     g = 9.8;              % m/s^2

   for n = 0:N
       x = ( M*g + n*(n+1)*k*d ) / ( (2*n+1)*k );
       if x <= (n+1)*d
           return
       end
   end
```